

INFORMATIQUE • 1ÈRE TI

MODULE 2 : PROGRAMMATION

LEÇON N08

PROGRAMMATION PROCÉDURALE EN LANGAGE C

DURÉE CONSEILLÉE : 12 H

DANS CETTE LEÇON

Cette leçon initie l'élève à la programmation procédurale en C : de la structure d'un programme à la traduction d'algorithmes concrets, en passant par les types, les opérateurs, les entrées/sorties, les structures de contrôle et les boucles, le tout testé dans un IDE courant.

- À la fin de cette leçon, je sais distinguer un langage interprété d'un langage compilé et expliquer le rôle du compilateur C.
- Je sais écrire, compiler et exécuter un programme C minimal (Hello World) dans Code ::Blocks ou Dev-C++.
- Je sais déclarer des variables et des constantes en respectant les types de base (int, float, double, char) et utiliser les opérateurs arithmétiques, de comparaison et logiques.
- Je sais afficher des résultats formatés avec printf et lire des données au clavier avec scanf.
- Je sais construire des alternatives (if, if...else, switch) et des boucles (for, while, do...while) pour résoudre des problèmes algorithmiques.
- Je sais utiliser les fonctions mathématiques courantes (sqrt, abs, cos) de la bibliothèque math.h.

**TÉLÉCHARGE
ONBUCH**

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats

**PASSE À ONBUCH+**

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. 1. Langages interprétés vs compilés & structure minimale d'un programme C
2. 2. Types de base, variables, constantes et opérateurs
3. 3. Entrées/Sorties formatées avec `stdio.h` (`printf`, `scanf`)
4. 4. Structures alternatives : `if`, `if...else`, `switch`
5. 5. Boucles : `for`, `while`, `do...while` et traduction d'algorithmes itératifs
6. 6. Fonctions mathématiques (`math.h`) et exécution pas à pas (débogage)
7. 7. Mini-projet d'intégration : calculatrice de notes de classe
8. Activité d'intégration
9. Méthodes et exercices résolus
10. Exercices
11. Corrigés des exercices
12. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais distinguer un langage interprété d'un langage compilé et expliquer le rôle du compilateur C.
- Je sais écrire, compiler et exécuter un programme C minimal (Hello World) dans Code ::Blocks ou Dev-C++.
- Je sais déclarer des variables et des constantes en respectant les types de base (`int`, `float`, `double`, `char`) et utiliser les opérateurs arithmétiques, de comparaison et logiques.
- Je sais afficher des résultats formatés avec `printf` et lire des données au clavier avec `scanf`.
- Je sais construire des alternatives (`if`, `if...else`, `switch`) et des boucles (`for`, `while`, `do...while`) pour résoudre des problèmes algorithmiques.
- Je sais utiliser les fonctions mathématiques courantes (`sqrt`, `abs`, `cos`) de la bibliothèque `math.h`.
- Je sais exécuter pas à pas un programme C dans le débogueur de l'IDE pour repérer les erreurs logiques.
- Je sais traduire un algorithme écrit en langage naturel ou en pseudo-code en un programme C complet et le tester sur des cas limites.

PRÉREQUIS

- Notion d'algorithme et de pseudo-code (variables, affectation, structures de contrôle) vue en Seconde.
- Manipulation basique d'un système d'exploitation (créer un dossier, lancer un exécutable).
- Connaissances élémentaires en arithmétique (opérations, priorités) et en logique booléenne (ET, OU, NON).
- Utilisation d'un éditeur de texte simple (Notepad, gedit) pour écrire du code source.
- Comprendre la différence entre un fichier source (.c) et un fichier exécutable.

1 LANGAGES INTERPRÉTÉS VS COMPIlés & STRUCTURE MINIMALE D'UN PROGRAMME C

Dans un cybercafé de Yaoundé, le gérant, M. Etoundi, passe une bonne partie de sa journée à calculer à la main le montant total des impressions de chaque client : nombre de pages, type de papier (simple ou glacé), remise accordée dès qu'un client dépasse cent pages. Chaque soir, les erreurs de calcul se comptent en milliers de francs CFA perdus. Il se demande alors : comment transformer ce besoin quotidien en un programme informatique fiable, que je pourrais exécuter sur n'importe quel ordinateur du quartier, sans rien installer de compliqué ?

C'est exactement la promesse du langage **C** : écrire une fois un **programme**, le traduire en un fichier exécutable, puis le distribuer partout. Mais avant d'écrire la moindre ligne de code, il faut comprendre comment un texte écrit par un humain devient un programme que la machine exécute. C'est toute la problématique de cette première section : **qu'est-ce qui distingue un langage compilé d'un langage interprété, et quelle est la structure minimale d'un programme C ?**

1.1 Programme, langage de programmation : les fondations

DÉFINITION Programme et langage de programmation

Un **programme** est une suite finie et ordonnée d'instructions, écrites dans un **langage de programmation**, qu'un ordinateur peut exécuter pour résoudre un problème donné. Un **langage de programmation** est un langage artificiel doté d'un vocabulaire (mots-clés) et d'une grammaire (syntaxe) précis, permettant à un humain de communiquer des ordres à la machine.

L'ordinateur, lui, ne comprend qu'une seule langue : le **langage machine**, c'est-à-dire des suites de 0 et de 1 directement exécutables par le processeur. Or aucun humain raisonnable n'écrit 10110000 01100001 pour afficher un message. Tout l'enjeu est donc la traduction : comment passer du texte écrit par le programmeur (le **code source**) au code machine ? Il existe deux grandes stratégies, qui partagent tous les langages de programmation en deux familles.

1.2 Langages interprétés et langages compilés

DÉFINITION Langage interprété

Un **langage interprété** est un langage dont le code source est lu, traduit et exécuté instruction par instruction, à la volée, par un programme spécial appelé **interpréteur**. Exemples : Python, JavaScript, PHP, BASIC. À chaque exécution, l'interpréteur relit tout le code source : il n'existe pas de fichier exécutable autonome.

DÉFINITION Langage compilé

Un **langage compilé** est un langage dont le code source est traduit une seule fois, intégralement et avant l'exécution, en code machine par un programme appelé **compilateur**. Le résultat est un **fichier exécutable** (par exemple `impression.exe` sous Windows) qui peut ensuite être lancé autant de fois qu'on veut, sans le compilateur et sans le code source. Exemples : C, C++, Pascal, Rust.

L'intuition est simple : un interpréteur, c'est un traducteur qui traduit un discours phrase par phrase pendant qu'on le prononce ; un compilateur, c'est un traducteur qui traduit tout le livre une fois pour toutes, puis on distribue le livre traduit.

Critère	Langage compilé (C)	Langage interprété (Python)
Traduction	Une seule fois, avant l'exécution	À chaque exécution, ligne par ligne
Vitesse d'exécution	Très rapide (code machine direct)	Plus lente (interpréteur intermédiaire)
Détection des erreurs	À la compilation, avant exécution	Pendant l'exécution
Distribution	Exécutable autonome (.exe)	Nécessite l'interpréteur installé

Pour M. Etoundi, le choix du C est donc idéal : une fois son programme compilé en `impression.exe`, il le copie sur une clé USB et le lance sur n'importe quel poste du cybercafé, même sans rien installer.

1.3 Le flux de compilation d'un programme C

Contrairement à une idée répandue, « compiler » un programme C n'est pas une seule étape, mais une chaîne de quatre étapes que l'IDE enchaîne automatiquement.

PROPRIÉTÉ Les quatre étapes de la chaîne de compilation C

1. Le **prétraitement** (préprocesseur) traite les directives (lignes commençant par #) : il remplace notamment `#include <stdio.h>` par le contenu réel du fichier d'en-tête, et `#define` par les valeurs définies. Il produit un code source « expansé » (fichier `.i`).
2. La **compilation** (compilateur) traduit ce code source expansé en **code assembleur** (fichier `.s`), langage symbolique propre à l'architecture du processeur.
3. L'**assemblage** (assembleur) convertit le code assembleur en **code objet** (fichier `.o` ou `.obj`) : du code machine presque complet, mais avec des références non résolues (par exemple l'adresse réelle de la fonction `printf`).
4. L'**édition de liens** (linker) assemble le ou les fichiers objets avec les bibliothèques (celle qui contient le code machine de `printf`, par exemple) et produit le **fichier exécutable**.

cutable (.exe sous Windows).



La chaîne de compilation en C : du code source au fichier exécutable (quatre étapes).

À RETENIR Ce qu'il faut retenir

Le C est un langage **compilé** : le code source (.c) est traduit une fois pour toutes en un exécutable autonome, en passant par le prétraitement, la compilation, l'assemblage puis l'édition de liens. Toute erreur de syntaxe est détectée avant l'exécution, à la compilation.

LESAVAIS-TU ? Le C, père des systèmes modernes

Le langage C a été créé en 1972 par Dennis Ritchie dans les laboratoires Bell, pour réécrire le système d'exploitation **UNIX**, jusqu'alors écrit en assembleur. Cinquante ans plus tard, le C reste la base de la quasi-totalité des systèmes modernes : le noyau Linux (qui fait tourner Android, les serveurs de CAMTEL et la plupart des sites web), une grande partie de Windows et de macOS sont écrits en C. Apprendre le C en Première TI, c'est apprendre la langue dans laquelle le monde numérique est construit.

1.4 Structure minimale d'un programme C

Tout programme C, du plus petit au plus grand, respecte la même ossature. La voici, réduite à son strict minimum :

```
#include <stdio.h>

int main(void)
{
    return 0;
}
```

Analysons chaque élément, car chacun est obligatoire et a un rôle précis :

- `#include <stdio.h>` : **directive du préprocesseur**. Elle demande d'insérer le fichier d'en-tête `stdio.h` (Standard Input Output), qui contient les déclarations des fonctions d'entrée/-

sortie comme `printf` et `scanf`. Sans elle, le compilateur ne connaît pas `printf`.

- `int main(void)` : la **fonction principale**. Tout programme C commence son exécution par `main`. Le mot `int` signifie que cette fonction retourne un entier au système d'exploitation à la fin de son exécution. `void` indique explicitement que la fonction ne prend aucun paramètre (bonnes pratiques en C).
- Les accolades `{ }` : elles délimitent le **corps** de la fonction, c'est-à-dire le bloc d'instructions à exécuter. Toute accolade ouvrante doit avoir sa fermante.
- `return 0 ;` : l'instruction qui termine la fonction `main` en renvoyant la valeur 0 au système d'exploitation. Par convention, 0 signifie « le programme s'est terminé sans erreur ».

ATTENTION L'erreur n°1 du débutant : le point-virgule oublié

En C, toute instruction se termine par un point-virgule `;`. Oublier le `;` après `return 0` ou après un `printf` provoque une erreur de compilation du type « error : expected ';' before ... ». Le compilateur signale souvent l'erreur sur la ligne suivante, ce qui déroute les débutants. Autres pièges classiques : oublier une accolade fermante, écrire `Main` avec une majuscule (le C **distingue les majuscules des minuscules** : `main` et `Main` sont deux noms différents), ou écrire `include` sans le `#`.

1.5 Premier programme : « Hello World » analysé ligne par ligne

La tradition veut que le premier programme écrit dans un langage affiche un message de salutation. Voici le nôtre, complet et commenté :

EXEMPLE Programme « Hello World » complet et commenté

```
#include <stdio.h> /* Directive : on importe les entrees/sorties */

int main(void) /* Fonction principale : point d'entree du programme */
{
    /* Affichage d'un message a l'ecran.
       \n provoque un retour a la ligne apres le message. */
    printf("Bonjour la classe de Premiere TI !\n");
    printf("Bienvenue dans le monde du langage C.\n");

    return 0; /* On rend la main au systeme : tout s'est bien passe */
}
```

Détaillons l'exécution ligne par ligne :

1. Le préprocesseur remplace `#include <stdio.h>` par la déclaration de la fonction `printf`.
2. L'exécution démarre à `main`. Le premier `printf` affiche `Bonjour la classe de Premiere TI !` puis le caractère spécial `\n` fait passer le curseur à la ligne suivante.

3. Le second `printf` affiche la deuxième phrase, suivie d'un autre retour à la ligne.
4. `return 0` ; termine le programme et informe le système que tout s'est bien passé.

À l'écran, on obtient exactement :

```
Bonjour la classe de Premiere TI !  
Bienvenue dans le monde du langage C.
```

ATTENTION Les séquences d'échappement

Dans une chaîne de caractères C, le caractère `\` (antislash) introduit une **séquence d'échappement** : `\n` = retour à la ligne, `\t` = tabulation, `\"` = guillemet, `\\` = antislash. Écrire `printf("C:\nouveau")` affiche C: suivi d'un retour à la ligne puis ouveau ! Pour afficher un vrai antislash, il faut le doubler : `"C:\\nouveau"`.

1.6 Créer, compiler et exécuter un projet dans Code ::Blocks

Un **IDE** (Environnement de Développement Intégré) regroupe dans un seul logiciel : l'éditeur de code, le compilateur (ici GCC), le débogueur et l'exécuteur. Les deux IDE recommandés au programme sont **Code ::Blocks** et **Dev-C++**.

MÉTHODE Créer un projet « Console Application » dans Code ::Blocks

1. Lance Code ::Blocks, puis menu File → New → Project...
2. Dans la fenêtre des modèles, choisis Console application, clique sur Go, puis Next.
3. Sélectionne le langage C (et non C++), puis Next.
4. Donne un titre au projet, par exemple `HelloTI`, choisis un dossier de destination (évite les espaces et les accents dans les noms de dossiers), puis Next.
5. Laisse le compilateur GNU GCC Compiler sélectionné, clique sur Finish.
6. Dans l'arborescence à gauche, déplie Sources et double-clique sur `main.c` : Code ::Blocks a déjà généré un squelette de programme.
7. Remplace ou complète le code, enregistre avec Ctrl+S.

PROPRIÉTÉ Raccourcis clavier essentiels

- F9 (Build and Run) : compile, édite les liens et exécute le programme en une seule action. C'est le raccourci que tu utiliseras le plus.

- Ctrl+F9 : compile seulement (Build), pour vérifier l'absence d'erreurs sans exécuter.
- Ctrl+F10 : exécute le dernier programme compilé (Run), sans recompiler.

Dans Dev-C++, l'équivalent de F9 est F11 (Compiler & Exécuter).

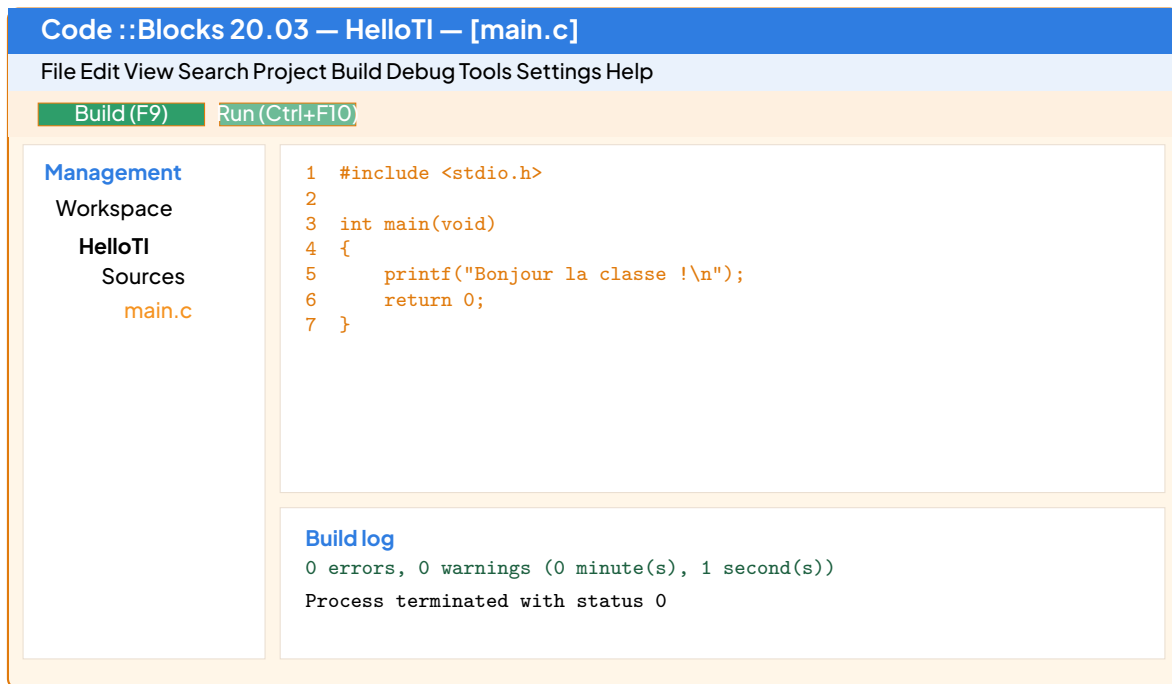


Schéma de l'interface de Code ::Blocks : panneau des projets à gauche, éditeur de code au centre, journal de compilation en bas.

EXPÉRIENCE TP guidé : ton premier programme C exécuté

Objectif : créer, compiler et exécuter un programme qui affiche ton nom et ta classe.

Matériel : un ordinateur avec Code ::Blocks (ou Dev-C++) installé.

Manipulation :

1. Crée un projet Console application en langage C nommé `PremierProg` (méthode ci-dessus).
2. Ouvre `main.c` et écris le code suivant :

```
#include <stdio.h>

int main(void)
{
    printf("Je m'appelle Amina.\n");
    printf("Je suis en Première TI.\n");
    printf("Mon premier programme C fonctionne !\n");
}
```

```
    return 0;  
}
```

3. Appuie sur F9. Observe le journal de compilation en bas : 0 errors, 0 warnings.
4. Une console noire s'ouvre et affiche les trois lignes. Note le message final « Process returned 0 » : c'est la preuve que ton `return 0` ; a bien été transmis au système.
5. Provoque volontairement une erreur : supprime le point-virgule de la ligne 5, appuie sur F9, et lis le message d'erreur dans le journal. Corrige, puis recompile.

Observations : sans le ; , la compilation échoue et aucun exécutable n'est produit — c'est la caractéristique d'un langage compilé. Interprétation : le compilateur est ton premier correcteur ; apprend à lire ses messages au lieu de les craindre.

EXERCICE RÉSOLU 1**Traduction d'un besoin en programme C**

Énoncé. M. Etoundi veut un premier programme qui affiche simplement, à l'ouverture de la caisse, l'en-tête suivant sur deux lignes : CYBERCAFE ETOUNDI - YAOUNDE puis Bienvenue !. Écris le programme C complet.

SOLUTION

Solution détaillée. Il faut : (1) inclure `stdio.h` pour utiliser `printf` ; (2) écrire la fonction `main` ; (3) deux appels à `printf` avec `\n` pour le passage à la ligne ; (4) terminer par `return 0` ; .

```
#include <stdio.h>  
  
int main(void)  
{  
    printf("CYBERCAFE ETOUNDI - YAOUNDE\n");  
    printf("Bienvenue !\n");  
    return 0;  
}
```

Vérification mentale ligne par ligne : la directive `#include` est présente ; `main` retourne un `int` ; chaque instruction se termine par ; ; les accolades sont équilibrées (une ouvrante, une fermante). À l'exécution (F9), la console affiche exactement les deux lignes demandées, puis Process returned 0. Le programme est correct.

EXERCICE 1 À toi de jouer

1. Explique en deux phrases la différence entre un langage compilé et un langage interprété, en citant un exemple de chaque.

2. Remplace dans l'ordre les étapes de la chaîne de compilation : éditeur de liens, préprocesseur, exécutable, compilateur, code objet, assembleur.
3. Écris un programme C qui affiche ton nom, ta classe et ton lycée sur trois lignes distinctes.
4. Que se passe-t-il si l'on écrit `int Main()` au lieu de `int main(void)` ? Justifie.

CORRIGÉ Exercice

1. Un langage compilé (C) est traduit une fois pour toutes en code machine avant l'exécution, produisant un exécutable autonome. Un langage interprété (Python) est traduit et exécuté instruction par instruction, à chaque lancement, par un interpréteur.
2. Ordre correct : préprocesseur → compilateur → assembleur → code objet → éditeur de liens → exécutable.

```
#include <stdio.h>

int main(void)
{
    printf("Nom : Ngo Bell Sandrine\n");
    printf("Classe : Premiere TI\n");
    printf("Lycee : Lycee de Nkol-Eton\n");
    return 0;
}
```

4. Le C distingue les majuscules des minuscules : `Main` n'est pas reconnu comme le point d'entrée du programme. L'éditeur de liens échoue avec une erreur du type « undefined reference to 'main' », car il ne trouve aucune fonction `main` (en minuscules).

À RETENIR Bilan de la section

- Le C est un langage **compilé** : source → prétraitement → compilation → assemblage → édition de liens → exécutable.
- La structure minimale d'un programme C comprend : la directive `#include <stdio.h>`, la fonction `int main(void)`, ses accolades et l'instruction `return 0 ;`.
- Toute instruction se termine par un `;` ; le C est sensible à la casse (majuscules/minuscules).
- Dans Code :: Blocks : projet Console Application en C, puis F9 pour compiler et exécuter, Ctrl+F10 pour exécuter sans recompiler.

Dans la prochaine section, nous donnerons de la mémoire à nos programmes : types de

base, variables, constantes et opérateurs — de quoi commencer à calculer les impressions de M. Etoundi.

2 TYPES DE BASE, VARIABLES, CONSTANTES ET OPÉRATEURS

Dans la section précédente, nous avons découvert la différence entre langage compilé et langage interprété, puis écrit notre premier programme C : la structure minimale avec `main()` et un premier affichage grâce à `printf`. Mais un programme qui ne fait qu'afficher un message fixe reste très limité. Les vrais programmes manipulent des données : des notes d'élèves, des prix en FCFA, des températures, des quantités en stock. Pour cela, il faut savoir **stocker** ces données en mémoire (variables et constantes), choisir le bon **type** pour chacune, et les **transformer** grâce aux opérateurs. C'est l'objet de cette section, véritable fondation de tout ce qui suivra.

2.1 Notion de variable : une boîte nommée en mémoire

Imagine la mémoire vive (RAM) de l'ordinateur comme une immense armoire remplie de casiers numérotés. Quand tu écris un programme, tu n'as pas envie de retenir des numéros de casiers : tu veux donner un nom à chaque casier et y ranger une valeur que tu pourras lire et modifier à volonté. C'est exactement ce qu'est une variable.

DÉFINITION Variable

Une **variable** est une zone mémoire nommée dont la valeur peut changer pendant l'exécution du programme. Elle est caractérisée par trois éléments : son nom (identificateur), son type (nature des valeurs qu'elle peut contenir) et sa valeur (contenu actuel de la zone mémoire).

En C, langage typé statiquement, toute variable doit être **déclarée avant d'être utilisée** : on annonce au compilateur son type et son nom, afin qu'il réserve la bonne quantité de mémoire. La syntaxe générale est :

```
type nom;           // déclaration simple
type nom = valeur;  // déclaration avec initialisation
type a = 1, b = 2;   // plusieurs variables du meme type
```

EXEMPLE Déclarations de variables

```
int age = 16;           // age d'un eleve
float moyenne = 12.75;  // moyenne generale
```

```
char genre = 'F';      // une lettre : note les apostrophes
int nombreEleves;      // declaree mais pas encore initialisee
nombreEleves = 45;     // affectation ulterieure
```

ATTENTION Règles de nommage et variable non initialisée

Un identificateur C commence par une lettre ou un `_` (jamais un chiffre), ne contient que lettres, chiffres et `_`, et ne doit pas être un mot réservé (`int`, `return`, `if`...). Le C distingue majuscules et minuscules : `Note` et `note` sont deux variables différentes. Autre piège classique : une variable déclarée mais jamais initialisée contient une valeur imprévisible (un « résidu » de la mémoire). Initialise toujours tes variables avant de les lire.

2.2 Les types de base du langage C

Pourquoi plusieurs types ? Parce que la mémoire coûte cher et que toutes les données n'ont pas les mêmes besoins. Stocker l'âge d'un élève (entre 0 et 120) ne demande pas la même place qu'un calcul scientifique de haute précision. Le type détermine la **taille occupée en mémoire** et l'**intervalle de valeurs** représentables.

Les types entiers. Ils représentent des nombres sans partie décimale : `short` (entier court), `int` (entier standard, le plus utilisé) et `long` (entier long). On peut les préfixer de `unsigned` pour n'accepter que des valeurs positives, ce qui double la borne supérieure.

Les types réels (flottants). Ils représentent des nombres à virgule : `float` (simple précision, environ 7 chiffres significatifs) et `double` (double précision, environ 15 chiffres significatifs). En pratique, on préfère souvent `double` pour les calculs.

Le type caractère. `char` stocke un seul caractère, codé sur 1 octet. En réalité, un `char` contient un nombre : le code **ASCII** du caractère. Ainsi 'A' vaut 65, 'a' vaut 97, '0' vaut 48. C'est pourquoi on peut écrire `char c = 'A';` puis `c = c + 1;` ; `c` contient alors 'B'. Pour les textes accentués (français, langues nationales), on utilise aujourd'hui le codage **UTF-8**, qui code les caractères courants sur 1 octet (compatible ASCII) et les caractères spéciaux (é, è, ç...) sur 2 à 4 octets.

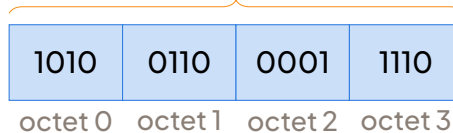
PROPRIÉTÉ Taille et plage du type int

En C, un `int` occupe généralement 4 octets (32 bits) sur les machines actuelles, soit une plage de valeurs de -2 147 483 648 à 2 147 483 647 (c'est-à-dire -2^{31} à $2^{31} - 1$). Dépasser ces bornes provoque un dépassement de capacité (overflow) : la valeur « boucle » et devient fausse sans message d'erreur. La taille exacte dépend de l'architecture ; utilise `sizeof(int)` pour la connaître à l'exécution.

Type	Taille typique	Intervalle de valeurs
char	1 octet	-128 à 127 (ou 0 à 255 selon implémentation)
short	2 octets	-32 768 à 32 767
int	4 octets	-2 147 483 648 à 2 147 483 647
long	4 ou 8 octets	au moins -2^{31} à $2^{31} - 1$
unsigned int	4 octets	0 à 4 294 967 295
float	4 octets	environ $\pm 3,4 \times 10^{38}$ (7 chiffres significatifs)
double	8 octets	environ $\pm 1,7 \times 10^{308}$ (15 chiffres significatifs)

Un `int` en mémoire : 4 octets contigus

32 bits = la variable `int n`;



Occupation mémoire d'une variable de type `int` : 4 octets (32 bits) réservés par le compilateur.

2.3 Les constantes : des valeurs qui ne changent jamais

Certaines valeurs sont fixées une fois pour toutes : π , le taux de TVA (19,25 % au Cameroun), le nombre de trimestres dans l'année. Les réécrire « en dur » partout dans le code est dangereux : si la valeur change, il faut la modifier à chaque endroit, sans en oublier. La solution : la **constante**.

Le C offre deux mécanismes :

1. Le mot-clé `const` placé devant une déclaration : la variable devient **en lecture seule**. Toute tentative de modification provoque une erreur de compilation. La constante possède un type, donc le compilateur vérifie son usage.
2. La directive `#define NOM valeur` : le préprocesseur remplace textuellement chaque occurrence du nom par la valeur avant la compilation. Pas de type, pas de vérification : c'est un simple copier-coller automatique.

MÉTHODE Déclarer une constante avec `const`

1. Choisis un nom explicite, par convention en majuscules : `PI`, `TAUX_TVA`.
2. Déclare-la avec son type, précédé de `const`, et initialise-la immédiatement : `const double PI = 3.14159;`
3. Place-la au début du programme (avant `main`) pour qu'elle soit visible partout (portée globale).
4. Utilise ce nom dans tous les calculs au lieu de la valeur numérique.

Préfère `const` à `#define` : le typage permet au compilateur de détecter les erreurs, et le débogueur affiche le nom de la constante.

```
#include <stdio.h>

#define NB_TRIMESTRES 3 // macro : remplacement textuel
const double PI = 3.14159; // constante typée (recommandée)
const int EFFECTIF_MAX = 60; // autre constante typée

int main() {
    printf("Pi vaut %lf\n", PI);
    // PI = 3.14; // ERREUR de compilation : modification interdite
    return 0;
}
```

LESAVAIS-TU ? Et `constexpr` dans tout ça ?

Tu entendras peut-être parler du mot-clé `constexpr`, introduit par le C++ en 2011 (C++11) pour définir des constantes évaluées à la compilation. Sache qu'il n'existe pas en langage C : en C, on se contente de `const` ou des macros `#define`. C'est une des nombreuses différences entre le C et son « descendant » C++.

2.4 Les opérateurs : faire calculer le programme

Les opérateurs sont les verbes d'action du langage : ils transforment les valeurs. On distingue trois familles au programme.

Opérateurs arithmétiques. + (addition), - (soustraction), * (multiplication), / (division) et % (modulo : reste de la division entière, réservé aux entiers). Attention à la **division entière** : si les deux opérandes sont des `int`, le résultat est tronqué : `7 / 2` donne 3, et non 3,5. Pour obtenir un réel, il faut qu'au moins un opérande soit réel : `7.0 / 2` donne 3,5. Le modulo est très utile : `n % 2 == 0` teste si `n` est pair.

Opérateurs de comparaison. Ils comparent deux valeurs et renvoient 1 (vrai) ou 0 (faux) : `==` (égal), `!=` (différent), `<`, `>`, `<=`, `>=`. Le C ne possède pas de type booléen natif (avant C99/`stdbool.h`) : le faux est 0, et toute valeur non nulle est considérée comme vraie.

Opérateurs logiques. Ils combinent des conditions : `&&` (ET logique : vrai si les deux conditions sont vraies), `||` (OU logique : vrai si au moins une condition est vraie), `!` (NON logique : inverse la condition). Exemple typique : un élève est admis si sa moyenne est supérieure ou égale à 10 et sa conduite correcte.

```
int a = 7, b = 2;
printf("%d\n", a / b); // affiche 3 (division entière !)
printf("%d\n", a % b); // affiche 1 (reste de 7 divisé par 2)

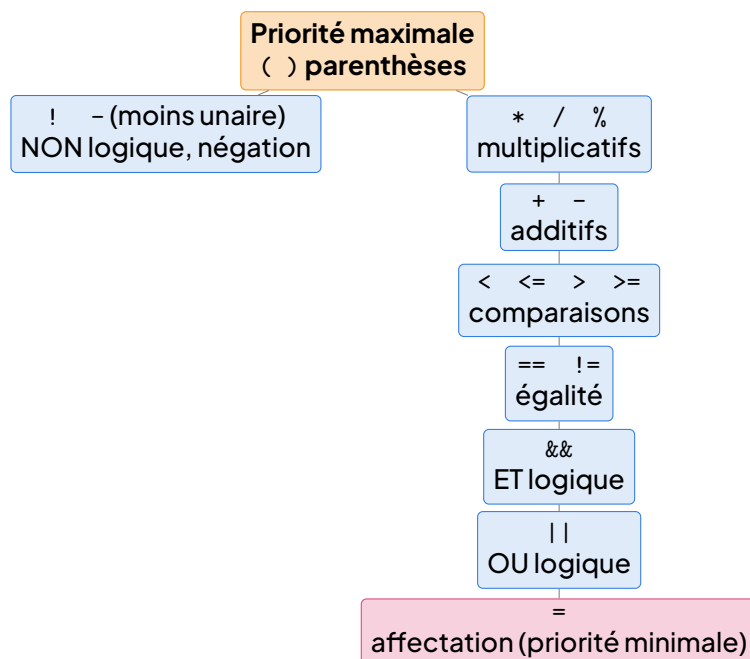
int moyenne = 12;
int admis = (moyenne >= 10) && (moyenne < 20);
printf("%d\n", admis); // affiche 1 (vrai)
```

ATTENTION Le piège n°1 du débutant : `=` contre `==`

`=` est l'affectation (range une valeur), `==` est le test d'égalité. Écrire `if (note = 10)` au lieu de `if (note == 10)` ne provoque pas d'erreur de compilation : la condition affecte 10 à `note` puis teste cette valeur non nulle, donc elle est toujours vraie. Bug silencieux et redoutable ! Astuce de vieux programmeur : écrire `if (10 == note)` — si tu oublies un `=`, le compilateur refusera `10 = note`.

2.5 Priorités et associativité des opérateurs

Que vaut $2 + 3 * 4$? Comme en mathématiques, la multiplication est prioritaire : le résultat est 14, pas 20. Le C définit une **hiérarchie de priorité** stricte. À priorité égale, l'**associativité** tranche : les opérateurs arithmétiques s'évaluent de gauche à droite ($10 - 4 - 3$ vaut 3), tandis que l'affectation s'évalue de droite à gauche (`a = b = 5` affecte 5 aux deux).



Arbre de priorité des opérateurs en C, du plus prioritaire (haut) au moins prioritaire (bas).

À RETENIR Règle d'or des parenthèses

En cas de doute sur la priorité, mets des parenthèses. Elles sont gratuites, ne ralentissent pas le programme et rendent le code lisible et sans ambiguïté : écris `(a + b) * c` plutôt que de parier sur ta mémoire. Une expression parenthésée ne trompe jamais.

EXEMPLE Exemple chiffré : aire d'un cercle

On veut calculer l'aire $A = \pi \times r^2$ d'un cercle dont le rayon est saisi au clavier (par exemple pour paver un rond-point à Yaoundé).

```
#include <stdio.h>

const double PI = 3.14159; // constante typee

int main() {
    double rayon, aire;

    printf("Entrez le rayon (m) : ");
    scanf("%lf", &rayon); // %lf pour un double, & pour l'adresse

    aire = PI * rayon * rayon; // A = PI * r * r

    printf("Aire = %lf m2\n", aire);
    return 0;
}
```

Pour un rayon de 5 m, le programme affiche une aire de 78,539750 m². Remarque l'usage de la constante PI : si l'on veut plus de précision, on ne modifie qu'une seule ligne. Notez le & devant rayon dans scanf : il indique l'adresse mémoire où stocker la valeur lue.

EXERCICE RÉSOLU 2 Moyenne de deux notes et mention

Énoncé : écris un programme C qui déclare deux notes (float), calcule leur moyenne, puis affiche 1 si l'élève est admis (moyenne ≥ 10) et 0 sinon, en utilisant un opérateur de comparaison.

SOLUTION

Solution détaillée :

```
#include <stdio.h>

int main() {
    float note1 = 13.5, note2 = 8.0; // declarations initialisees
    float moyenne;
    int admis;

    moyenne = (note1 + note2) / 2; // parentheses inutiles mais claires
    // note1 + note2 est un float : division reelle, pas de troncature

    admis = (moyenne >= 10); // comparaison : 1 ou 0

    printf("Moyenne = %f\n", moyenne); // affiche 10.750000
    printf("Admis : %d\n", admis); // affiche 1
    return 0;
}
```

Analyse : la somme $13,5 + 8 = 21,5$ est divisée par 2, ce qui donne 10,75 (division réelle car les opérandes sont des `float`). La comparaison `moyenne >= 10` est vraie, donc `admis` reçoit 1. Si l'on avait déclaré les notes en `int`, $(13 + 8) / 2$ aurait donné 10 par troncature : le choix du type compte !

EXERCICE 2 À toi de jouer

1. Déclare une constante entière `TAUX_REMISE` valant 5 et une variable `prix` de type `double`. Calcule et affiche le prix après remise de 5 %.
2. Que valent les expressions suivantes en C : `9 / 4`, `9 % 4`, `9.0 / 4`, `(5 > 3) && (2 == 2)`, `!(7 <= 7)` ?
3. Écris une expression C qui vaut vrai si un entier `n` est compris entre 10 et 20 inclus.

CORRIGÉ Exercice 1

1. `const int TAUX_REMISE = 5; et double prix = 10000; puis double final = prix - prix * TAUX_REMISE / 100; soit 9 500 FCFA.`
2. `9 / 4` vaut 2 (division entière); `9 % 4` vaut 1; `9.0 / 4` vaut 2,25; `(5 > 3) && (2 == 2)` vaut 1; `!(7 <= 7)` vaut 0.
3. `(n >= 10) && (n <= 20)` — impossible d'écrire `10 <= n <= 20` en C : cela évaluerait `(10 <= n) (0 ou 1)` puis comparerait ce résultat à 20, toujours vrai !

Tu maîtrises désormais les briques de données du langage C : types, variables, constantes et opérateurs. Dans la prochaine section, nous apprendrons à dialoguer avec l'utilisateur de manière précise et élégante grâce aux entrées/sorties formatées de `stdio.h` : `printf` et `scanf` n'auront plus aucun secret pour toi.

3 ENTRÉES/SORTIES FORMATÉES AVEC STDIO.H (PRINTF, SCANF)

Dans la section précédente, nous avons appris à déclarer des variables et des constantes, à choisir le bon type de base (`int`, `float`, `double`, `char`) et à combiner des valeurs à l'aide des opérateurs arithmétiques, de comparaison et logiques. Mais un programme qui calcule en silence ne sert à rien : il faut qu'il dialogue avec l'utilisateur. Imagine la calculatrice du proviseur qui doit afficher la moyenne d'un élève, ou le programme du cybercafé qui demande la durée de connexion puis affiche le montant à payer. Sans instructions d'entrée (lire ce que tape l'utilisateur) et de sortie (afficher un résultat à l'écran), aucun de ces programmes n'existe. C'est pré-

cisément le rôle de la bibliothèque `stdio.h` et de ses deux fonctions vedettes : `printf` et `scanf`. Cette section te rendra capable de faire communiquer n'importe quel programme C avec son utilisateur, de manière propre et formatée.

3.1 La bibliothèque `stdio.h` et les flux standard

DÉFINITION Bibliothèque `stdio.h`

`stdio.h` (Standard Input/Output) est la bibliothèque standard du langage C qui fournit les fonctions d'**entrées/sorties** : lire des données au clavier, afficher des résultats à l'écran, manipuler des fichiers. Pour l'utiliser, on écrit en tout début de programme :

```
#include <stdio.h>
```

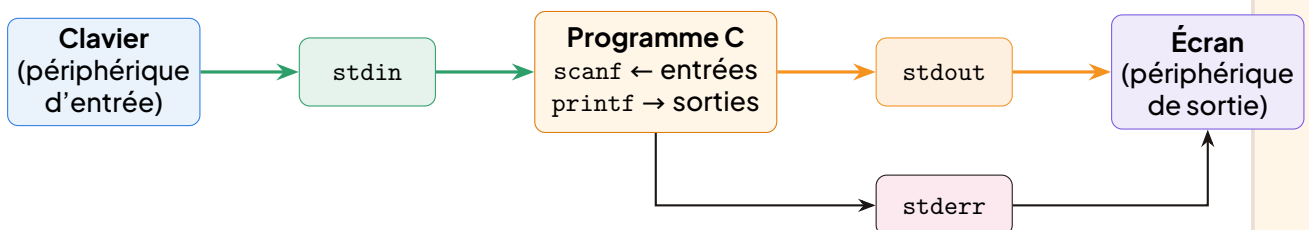
Sans cette ligne, le compilateur ne connaît ni `printf` ni `scanf` et refusera de compiler.

DÉFINITION Flux standard

Un **flux** (stream) est un canal de communication entre le programme et un périphérique. Tout programme C dispose automatiquement de trois flux standard :

- **`stdin`** (standard input) : le flux d'entrée, relié par défaut au clavier ;
- **`stdout`** (standard output) : le flux de sortie, relié par défaut à l'écran ;
- **`stderr`** (standard error) : le flux des messages d'erreur, relié aussi à l'écran.

Concrètement : quand tu tapes 25 au clavier, ces caractères voyagent dans `stdin` jusqu'au programme qui les récupère avec `scanf` ; quand le programme affiche un résultat avec `printf`, les caractères voyagent dans `stdout` jusqu'à l'écran.



Les trois flux standard : les données tapées au clavier entrent par `stdin`, les résultats sortent par `stdout` vers l'écran, et les erreurs par `stderr`.

À RETENIR L'idée essentielle

`printf` écrit dans `stdout` (vers l'écran) ; `scanf` lit dans `stdin` (depuis le clavier). L'utilisateur ne voit que l'écran et le clavier : c'est le programme qui organise le dialogue.

3.2 Afficher avec `printf` : formatage de la sortie

DÉFINITION Fonction `printf`

`printf` (print formatted, « afficher avec format ») affiche à l'écran un texte dans lequel on peut insérer les valeurs de variables. Sa syntaxe générale est :

```
printf("chaîne de format", expression1, expression2, ...);
```

La **chaîne de format** contient du texte ordinaire et des **spécificateurs de format** (qui commencent par %), remplacés à l'affichage par les valeurs des expressions, dans l'ordre.

PROPRIÉTÉ Spécificateurs de format usuels

Spécificateur	Type affiché	Exemple de résultat (sortie C)
%d	entier (<code>int</code>)	42
%f	réel (<code>float</code> , <code>double</code>)	3.141593 (6 décimales par défaut)
%.2f	réel avec 2 décimales	3.14
%c	caractère (<code>char</code>)	A
%s	chaîne de caractères	Bonjour
%p	adresse mémoire (pointeur)	0x7ffd5e8a3b24
%%	le caractère % lui-même	%

On y ajoute les **séquences d'échappement** : `\n` (saut de ligne), `\t` (tabulation).

PROPRIÉTÉ Drapeaux : largeur et précision

Entre le % et la lettre, on peut préciser la mise en forme : %[largeur] . [précision][lettre].

- Largeur : nombre minimal de caractères affichés, aligné à droite. `%5d` affiche un entier sur 5 colonnes.
- Précision : pour un réel, nombre de décimales (`%.2f`) ; pour une chaîne, nombre maxi-

mal de caractères.

- Le drapeau – aligne à gauche : `%-10s`.

C'est l'outil idéal pour afficher des tableaux alignés (bulletins de notes, factures).

MÉTHODE Afficher un réel avec deux décimales

Pour afficher proprement une aire, une moyenne ou un prix :

1. Déclarer la variable réelle : `float aire;`
2. Calculer sa valeur ;
3. Afficher avec le spécificateur `%.2f` :

```
printf("Aire = %.2f\n", aire);
```

Le `.2` impose exactement deux chiffres après la virgule (point en notation C) ; le `\n` renvoie à la ligne pour la suite de l'affichage.

EXEMPLE Affichage aligné : mini-bulletin

```
#include <stdio.h>

int main(void) {
    int age = 16;
    float moyenne = 13.4567;
    char mention = 'B';

    printf("Age : %d ans\n", age);
    printf("Moyenne : %.2f / 20\n", moyenne); /* 13.46 */
    printf("Mention : %c\n", mention);
    printf("%-12s %8s\n", "Matiere", "Note"); /* en-têtes alignés */
    printf("%-12s %8.2f\n", "Informatique", 15.5);
    printf("%-12s %8.2f\n", "Mathematiques", 12.25);
    printf("Reussite a %.0f %%\n", 85.0); /* %% affiche % */
    return 0;
}
```

Résultat à l'écran : les notes sont alignées en colonnes grâce à la largeur 8 et à l'alignement à gauche -12 des noms de matières.

ATTENTION Spécificateur incompatible = comportement imprévisible

Si le spécificateur ne correspond pas au type de la variable (`printf("%d", 3.14);`), le ré-

sultat est imprévisible : valeur aberrante, voire plantage. Vérifie toujours la correspondance : `int` avec `%d`, `float/double` avec `%f`, `char` avec `%c`. Autre piège : oublier le `\n` final fait que l'invite de commande s'affiche sur la même ligne que le résultat.

3.3 Lire au clavier avec `scanf`

DÉFINITION Fonction `scanf`

scanf (scan formatted) lit des données tapées au clavier et les range dans des variables. Sa syntaxe générale est :

```
scanf("chaîne de format", &variable1, &variable2, ...);
```

Le symbole **&** (esperluette) placé devant chaque variable désigne son adresse mémoire : `scanf` a besoin de savoir où écrire la valeur lue.

PROPRIÉTÉ Règles d'utilisation de `scanf`

- Les spécificateurs sont les mêmes qu'avec `printf` : `%d` (entier), `%f` (float), `%lf` (double), `%c` (caractère).
- Jamais de `\n` dans le format de `scanf`.
- Toujours le **&** devant la variable (sauf pour les tableaux de caractères, vus plus tard).
- L'utilisateur valide sa saisie avec la touche Entrée.

MÉTHODE Lire deux entiers séparés par un espace

Pour lire plusieurs valeurs sur une même ligne :

1. Déclarer les variables : `int a, b;`
2. Afficher une consigne claire avec `printf` (l'utilisateur doit savoir quoi taper!);
3. Lire avec un format contenant les deux spécificateurs :

```
printf("Entrez deux entiers separes par un espace : ");  
scanf("%d %d", &a, &b);
```

L'utilisateur tape par exemple `12 7` puis Entrée : `a` reçoit 12 et `b` reçoit 7. L'espace dans le format accepte d'ailleurs un ou plusieurs espaces, tabulations ou retours à la ligne entre les valeurs.

ATTENTION Oublier le & : l'erreur classique du débutant

Écrire `scanf("%d", a);` au lieu de `scanf("%d", &a);` est l'erreur la plus fréquente en Première. Le compilateur ne signale parfois rien, mais le programme plante à l'exécution (erreur de segmentation) ou corrompt la mémoire, car `scanf` écrit n'importe où. Réflexe à acquérir : « `scanf` ⇒ & ».

3.4 Le buffer d'entrée et ses pièges

Quand l'utilisateur tape 25 puis Entrée, trois caractères arrivent dans le **buffer d'entrée** (une mémoire tampon associée à `stdin`) : '2', '5' et '\n' (le caractère newline produit par la touche Entrée). Or `scanf("%d", &a)` ne consomme que les chiffres : le '\n' **reste dans le buffer**.

ATTENTION `scanf` laisse le newline dans le buffer

Conséquence redoutable : un `scanf("%c", &reponse);` placé après un `scanf("%d", &a);` ne laisse pas le temps à l'utilisateur de taper : il lit immédiatement le '\n' resté dans le buffer ! Le programme semble « sauter » une question. Deux solutions :

1. Mettre un espace avant %c dans le format : `scanf(" %c", &reponse);` — l'espace absorbe tous les caractères blancs en attente ;
2. Vider le buffer avec `getchar();` juste avant la lecture du caractère.

EXEMPLE Nettoyage du buffer en pratique

```
int age;
char reponse;

printf("Votre age : ");
scanf("%d", &age);      /* le '\n' reste dans le buffer */

getchar();              /* on absorbe le '\n' résiduel */

printf("Continuer (o/n) ? ");
scanf("%c", &reponse); /* lit bien le caractère tape */
```

Sans le `getchar();`, la variable `reponse` contiendrait '\n' et la question serait ignorée.

3.5 Exemple résolu et observation en console

EXERCICE RÉSOLU 3 Périmètre et surface d'un terrain rectangulaire

Énoncé. Un menuisier de Bafoussam veut un programme qui demande la longueur et la largeur d'un terrain (en mètres), puis affiche son périmètre et sa surface avec deux décimales. Écris ce programme en C.

SOLUTION

Solution détaillée. On déclare quatre réels, on lit les deux dimensions avec un seul `scanf`, on applique les formules $P = 2 \times (L + l)$ et $S = L \times l$, puis on affiche avec `%.2f`.

```
#include <stdio.h>

int main(void) {
    float longueur, largeur, perimetre, surface;

    printf("Entrez la longueur et la largeur (en m) : ");
    scanf("%f %f", &longueur, &largeur);

    perimetre = 2 * (longueur + largeur);
    surface = longueur * largeur;

    printf("Perimetre = %.2f m\n", perimetre);
    printf("Surface = %.2f m2\n", surface);
    return 0;
}
```

Trace d'exécution avec les valeurs 12,5 et 8 tapées par l'utilisateur : $P = 2 \times (12,5 + 8) = 41,00$ m et $S = 12,5 \times 8 = 100,00$ m². Le programme affiche exactement `Perimetre = 41.00` m puis `Surface = 100.00` m².

Console — terrain.exe

```
Entrez la longueur et la largeur (en m) :
12.5 8                                     ← saisie de l'utilisateur (scanf)

Perimetre = 41.00 m
Surface = 100.00 m2                       ← sorties formatées (printf)

Appuyez sur une touche pour continuer...
```

Console d'exécution : l'utilisateur tape ses données (lues par `scanf`), le programme affiche les résultats alignés (écrits par `printf`).

EXPÉRIENCE TP guidé : tester printf et scanf dans Code ::Blocks

Objectif : observer concrètement le dialogue entrée/sortie et le piège du buffer. Matériel : un PC avec Code ::Blocks (ou Dev-C++) installé.

1. Crée un nouveau projet Console Application en langage C, nommé `dialogue`.
2. Recopie le programme « Périmètre et surface » de l'exemple résolu, puis compile (Build) et exécute (Run).
3. Teste avec les valeurs `12.5 8`, puis avec `7 3.2`. Observe l'alignement des résultats.
4. Modifie le format en `%10.2f` et relance : constate l'alignement à droite sur 10 colonnes.
5. Retire volontairement le `&` d'un `scanf`, compile et exécute : observe le plantage, puis corrige.
6. Ajoute à la fin une question `scanf("%c", &reponse);` sans `getchar()` : constate que la question est « sautée », puis corrige avec `getchar();`.

Interprétation : tu viens de vérifier expérimentalement le rôle du `&`, le formatage par largeur/précision et le comportement du buffer d'entrée.

LESAVAIS-TU ? Le spécificateur `%p`, espion de la mémoire

Le spécificateur `%p` affiche l'adresse mémoire d'une variable, en notation hexadécimale : `printf("%p", &a);` affiche par exemple `0x7ffd5e8a3b24`. C'est un outil précieux de débogage : il permet de vérifier où les variables sont rangées et de comprendre pourquoi `scanf` exige le `&`. Les professionnels s'en servent aussi pour traquer les erreurs de pointeurs. À ton niveau, retiens simplement : chaque variable possède une adresse unique en mémoire, et `&a` signifie littéralement « l'adresse de `a` ».

EXERCICE 3 Facture de cybercafé

Un cybercafé de Yaoundé facture la connexion à 250 FCFA l'heure. Écris un programme C qui demande le nom du client (un seul mot, lu avec `%s` dans un `char nom[30]` ; — sans `&` car un tableau est déjà une adresse) et la durée de connexion en heures (réel), puis affiche une facture alignée : nom du client, durée, montant à payer avec deux décimales.

CORRIGÉ Exercice : Facture de cybercafé

```
#include <stdio.h>
```

```
int main(void){
    char nom[30];
    float duree, montant;

    printf("Nom du client : ");
    scanf("%s", nom);          /* pas de & : tableau */
    printf("Duree de connexion (h) : ");
    scanf("%f", &duree);

    montant = duree * 250;

    printf("\n----- FACTURE ----- \n");
    printf("%-12s : %s\n", "Client", nom);
    printf("%-12s : %.2f h\n", "Duree", duree);
    printf("%-12s : %.2f FCFA\n", "Montant", montant);
    return 0;
}
```

Points de vigilance : `%s` ne lit qu'un mot (il s'arrête au premier espace); le `&` est présent pour `duree` mais pas pour `nom`; l'alignement est obtenu par `%-12s`.

À RETENIR Bilan de la section

- `stdio.h` donne accès aux flux `stdin` (clavier), `stdout` et `stderr` (écran).
- `printf("format", valeurs)` affiche; les spécificateurs `%d`, `%f`, `%c`, `%s`, `%p` et les drapeaux de largeur/précision (`%.2f`, `%10d`) contrôlent la mise en forme.
- `scanf("format", &variables)` lit au clavier; le `&` est obligatoire devant chaque variable simple.
- Après un `scanf`, le `'\n'` reste dans le buffer : on le nettoie avec `getchar()`; ou un espace dans le format (`" %c"`).

Tu sais désormais faire dialoguer un programme avec son utilisateur. La prochaine section te donnera le pouvoir de faire choisir le programme entre plusieurs chemins : les structures alternatives `if`, `if...else` et `switch`.

4 STRUCTURES ALTERNATIVES : IF, IF...ELSE, SWITCH

Dans la section précédente, nous avons appris à dialoguer avec l'utilisateur grâce à `printf` et `scanf` : nos programmes savent désormais lire des données et afficher des résultats. Mais un programme qui se contente d'enchaîner toujours les mêmes instructions, quelles que soient les données saisies, reste très limité. Imagine le logiciel de gestion des notes du lycée : il doit afficher « Admis » si la moyenne est supérieure ou égale à 10, et « Ajourné » sinon. Le programme doit donc prendre une décision en fonction d'une condition. C'est exactement le rôle des **struc-**

structures alternatives que nous étudions maintenant : tu les connais déjà en algorithmique sous la forme *Si ... Alors ... Sinon ... FinSi* ; il s'agit ici de les traduire fidèlement en langage C.

4.1 L'instruction `if` simple

DÉFINITION Structure alternative

Une **structure alternative** (ou instruction conditionnelle) est une structure de contrôle qui permet d'exécuter un bloc d'instructions uniquement si une condition est vérifiée. En C, une condition est une expression dont la valeur est interprétée comme vraie (toute valeur différente de 0) ou fausse (la valeur 0).

L'intuition est simple : « Si la condition est vraie, ALORS fais ceci ». La syntaxe exacte en C est la suivante :

```
if (condition){  
    /* instructions executees si la condition est vraie */  
}
```

La condition est **obligatoirement entourée de parenthèses**. Les instructions à exécuter sont regroupées dans un **bloc** délimité par des accolades `{ }`. Si le bloc ne contient qu'une seule instruction, les accolades sont facultatives, mais nous te conseillons vivement de les mettre systématiquement : cela évite de nombreux bugs lorsqu'on ajoute plus tard une deuxième instruction.

EXEMPLE Contrôle d'accès à un examen

```
#include <stdio.h>  
  
int main(void) {  
    int age;  
    printf("Entrez votre age : ");  
    scanf("%d", &age);  
  
    if (age >= 16) {  
        printf("Vous pouvez vous presenter au Probatoire.\n");  
    }  
    printf("Fin du programme.\n");  
    return 0;  
}
```

Si l'utilisateur saisit 17, le message d'autorisation s'affiche. S'il saisit 14, le bloc est simplement sauté et seul « Fin du programme. » apparaît.

ATTENTION Le piège du point-virgule après le `if`

Ne place jamais de point-virgule juste après la parenthèse du `if` :

```
if (age >= 16); {           /* ERREUR : le ; termine le if ! */
    printf("Autorise.\n"); /* s'exécute TOUJOURS */
}
```

Le `;` est interprété comme une instruction vide : la condition ne contrôle plus rien, et le bloc s'exécute dans tous les cas. C'est l'un des bugs les plus classiques du débutant en C.

4.2 L'instruction `if ... else`

Très souvent, on veut faire une chose si la condition est vraie, et une autre chose si elle est fausse. C'est le « Si ... Alors ... Sinon ... FinSi » de l'algorithmique :

```
if (condition){
    /* instructions si la condition est vraie */
} else {
    /* instructions si la condition est fausse */
}
```

EXEMPLE Exemple chiffré : Pair ou Impair

Ce programme lit un entier et affiche sa parité. Rappel : un entier n est pair si et seulement si le reste de sa division par 2 vaut 0, ce qui s'écrit $n \% 2 == 0$ en C.

```
#include <stdio.h>

int main(void){
    int n;
    printf("Entrez un entier : ");
    scanf("%d", &n);

    if (n % 2 == 0){
        printf("%d est Pair\n", n);
    } else {
        printf("%d est Impair\n", n);
    }
    return 0;
}
```

Trace d'exécution : pour la saisie 14, on calcule $14 \bmod 2 = 0$, la condition est vraie, le programme affiche « 14 est Pair ». Pour la saisie 7, on calcule $7 \bmod 2 = 1$, la condition est fausse, c'est le bloc du `else` qui s'exécute : « 7 est Impair ».

ATTENTION Confondre = et ==

L'opérateur de comparaison est == (double égal). Le simple = est l'affectation. Écrire `if (n = 0)` ne teste pas si `n` vaut 0 : cela affecte 0 à `n`, et comme 0 signifie « faux », le bloc n'est jamais exécuté. Le compilateur ne signale parfois qu'un avertissement discret. Relis toujours tes conditions !

4.3 Opérateurs logiques dans les conditions : &&, ||, !

Une condition peut combiner plusieurs tests grâce aux **opérateurs logiques** :

Opérateur	Nom	Signification
&&	ET logique	vrai si les deux conditions sont vraies
	OU logique	vrai si au moins une condition est vraie
!	NON logique	inverse la valeur de vérité

Par exemple, pour vérifier qu'une note saisie est valide (comprise entre 0 et 20, comme au lycée) :

```
if (note >= 0 && note <= 20) {
    printf("Note valide.\n");
} else {
    printf("Erreur : une note doit etre entre 0 et 20.\n");
}
```

Concernant les **priorités** : le NON ! est prioritaire sur le ET &&, qui est lui-même prioritaire sur le OU ||. Les opérateurs de comparaison (<, >=, ==, ...) sont quant à eux prioritaires sur tous les opérateurs logiques. En cas de doute, mets des parenthèses : c'est gratuit et cela rend ton code plus lisible.

PROPRIÉTÉ Évaluation en court-circuit de && et ||

En C, l'opérateur && n'évalue le second opérande que si le premier est vrai : si le premier est faux, le résultat est forcément faux, inutile d'aller plus loin. De même, || n'évalue le second opérande que si le premier est faux. On parle d'**évaluation paresseuse** ou en court-circuit. Conséquence pratique : dans `if (n != 0 && 100 / n > 5)`, la division `100 / n` n'est jamais tentée lorsque `n` vaut 0, ce qui évite un plantage par division par zéro.

4.4 Imbrication d'alternatives et indentation

Un bloc `if` ou `else` peut lui-même contenir d'autres alternatives : c'est l'**imbrication**. C'est indispensable dès qu'il y a plus de deux cas. Par exemple, déterminer le signe d'un nombre (trois cas : positif, négatif, nul) :

```
if (n > 0) {
```

```
printf("Positif\n");
} else {
    if (n < 0) {
        printf("Négatif\n");
    } else {
        printf("Nul\n");
    }
}
```

MÉTHODE Indenter correctement un code imbriqué

1. À chaque ouverture d'accolade {, décale tout le bloc d'un niveau vers la droite (4 espaces ou une tabulation).
2. Aligne chaque accolade fermante } avec le mot-clé (if, else) qui a ouvert le bloc.
3. Place le else au même niveau d'indentation que son if.
4. Vérifie en relisant que chaque { possède exactement une } correspondante.

Un code bien indenté se lit comme un plan : on voit immédiatement quelles instructions dépendent de quelle condition. Code ::Blocks et Dev-C++ indentent automatiquement quand tu appuies sur Entrée — profite-en !

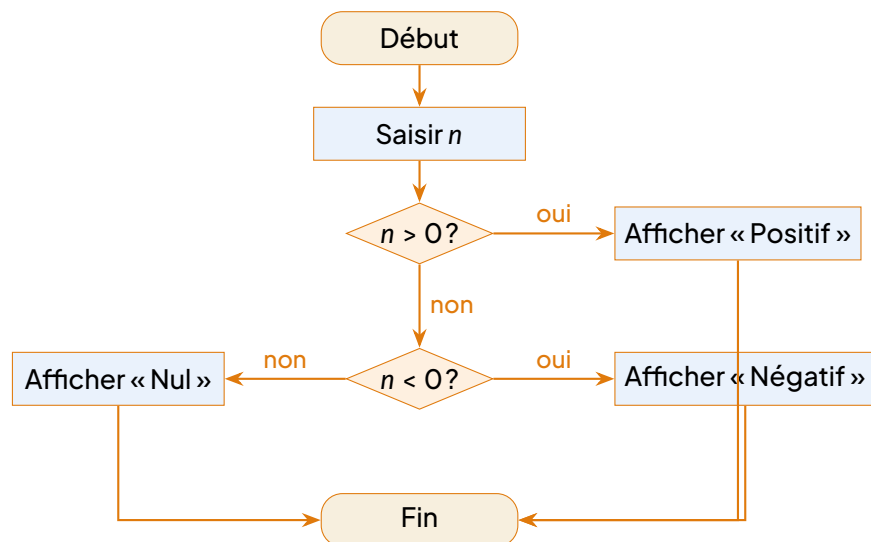


Diagramme de flux d'une structure if...else imbriquée : détermination du signe d'un nombre.

4.5 L'instruction switch

Quand une même variable doit être comparée à **plusieurs valeurs entières constantes** (un choix de menu, un mois de l'année, une mention...), enchaîner les if...else if devient lourd. Le switch

offre une écriture plus claire :

```
switch (expression) {  
    case valeur1 :  
        /* instructions */  
        break;  
    case valeur2 :  
        /* instructions */  
        break;  
    default :  
        /* instructions si aucun case ne correspond */  
}
```

L'expression doit être de type **entier** (int, char, ...): le switch ne fonctionne **pas** avec des nombres réels ni avec des chaînes de caractères. Chaque case énumère une valeur constante possible; default (facultatif) traite tous les autres cas.

ATTENTION L'oubli du break : le fall-through

L'instruction break fait sortir du switch. Si tu l'ometts, l'exécution continue dans le case suivant (on parle de fall-through, « traversée ») :

```
switch (choix) {  
    case 1 :  
        printf("Ajouter\n");  
        /* break oubliée : on tombe dans le case 2 ! */  
    case 2 :  
        printf("Supprimer\n");  
        break;  
}
```

Si choix vaut 1, ce code affiche « Ajouter » puis « Supprimer ». Ce comportement est parfois utilisé volontairement (plusieurs case vides empilées pour un même traitement), mais chez le débutant c'est presque toujours un oubli. Règle d'or : un break à la fin de chaque case.

MÉTHODE Écrire un switch pour un menu à choix multiples

1. Affiche le menu avec printf (une ligne par option numérotée).
2. Lis le choix de l'utilisateur dans une variable entière avec scanf("%d", &choix).
3. Ouvre le switch (choix) et crée un case par option du menu.
4. Termine chaque case par break;.
5. Ajoute un default qui signale poliment tout choix invalide.

EXEMPLE Menu d'une application de gestion de bibliothèque scolaire

```
#include <stdio.h>

int main(void) {
    int choix;
    printf("=== BIBLIOTHEQUE DU LYCEE ===\n");
    printf("1. Ajouter un livre\n");
    printf("2. Supprimer un livre\n");
    printf("3. Quitter\n");
    printf("Votre choix : ");
    scanf("%d", &choix);

    switch (choix) {
        case 1:
            printf("Ajout d'un livre ... \n");
            break;
        case 2:
            printf("Suppression d'un livre... \n");
            break;
        case 3:
            printf("Au revoir !\n");
            break;
        default:
            printf("Choix invalide : tapez 1, 2 ou 3.\n");
    }
    return 0;
}
```

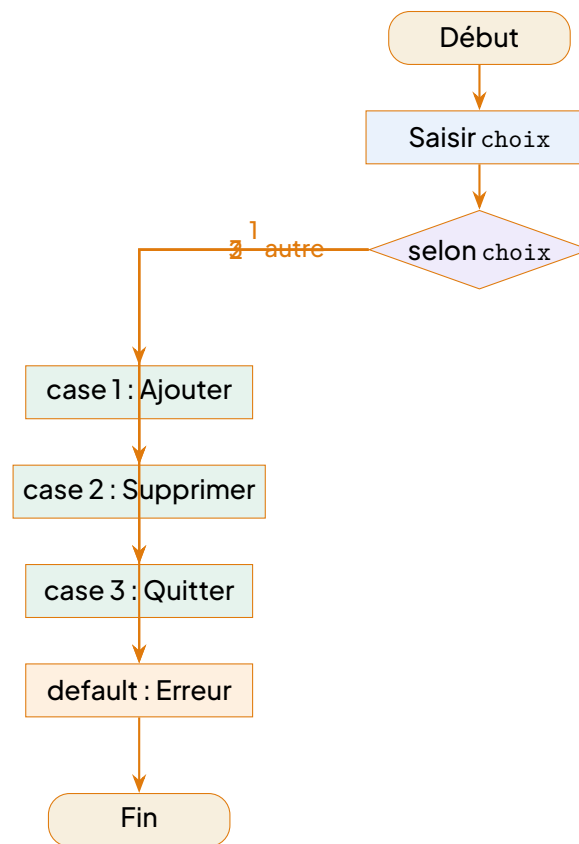


Diagramme de flux d'un `switch` : le choix aiguille vers le case correspondant, chaque `break` ramène vers la fin.

ATTENTION Le `switch` ne compare pas des chaînes de caractères

Contrairement à certains langages, le C n'autorise pas `switch (nom)` où `nom` serait une chaîne comme "Yaounde". Pour comparer des chaînes, il faut utiliser la fonction `strcmp` de `string.h` dans des `if...else if` (tu verras cela plus tard). À ce stade, retiens : `switch` = entiers et caractères uniquement.

4.6 L'opérateur ternaire ? :

Pour les alternatives très courtes dont le seul but est d'affecter une valeur, le C propose un raccourci élégant :

```
variable = (condition) ? valeur_si_vrai : valeur_si_faux;
```

LESAVAIS-TU ? Le ternaire, condensé d'un `if...else`

L'opérateur ternaire permet d'écrire une affectation conditionnelle en une seule ligne. Le maximum de deux nombres s'obtient ainsi :

```
max = (a > b) ? a : b;
```

Cette unique ligne remplace quatre lignes de `if...else`. C'est le seul opérateur du C qui prend trois opérandes, d'où son nom. Utilise-le pour des cas simples ; au-delà, préfère un `if...else` classique, plus lisible.

EXERCICE RÉSOLU 4 Mention d'une note au bulletin

Énoncé. Écris un programme C qui demande la moyenne générale d'un élève (réel entre 0 et 20) et affiche : « Ajourné » si la moyenne est inférieure à 10, « Passable » si elle est entre 10 et 11,99, « Assez Bien » entre 12 et 13,99, « Bien » entre 14 et 15,99 et « Très Bien » à partir de 16. Le programme refusera toute note hors de l'intervalle [0; 20].

SOLUTION

Solution détaillée. On commence par valider la saisie avec un OU logique, puis on enchaîne les cas par `if...else if` imbriqués (un `switch` est impossible ici car la moyenne est un réel).

```
#include <stdio.h>

int main(void) {
    float moy;
    printf("Entrez la moyenne generale : ");
    scanf("%f", &moy);

    if (moy < 0 || moy > 20) {
        printf("Erreur : moyenne hors de [0 ; 20].\n");
    } else if (moy < 10) {
        printf("Mention : Ajourne\n");
    } else if (moy < 12) {
        printf("Mention : Passable\n");
    } else if (moy < 14) {
        printf("Mention : Assez Bien\n");
    } else if (moy < 16) {
        printf("Mention : Bien\n");
    } else {
        printf("Mention : Tres Bien\n");
    }
    return 0;
}
```

Trace pas à pas pour la saisie 13,5 : le premier test ($13,5 < 0$ ou $13,5 > 20$) est faux ; le test $13,5 < 10$ est faux ; le test $13,5 < 12$ est faux ; le test $13,5 < 14$ est vrai : le programme affiche « Mention : Assez Bien » et saute tous les autres cas. Remarque l'astuce : grâce à l'enchaî-

nement des `else if`, chaque borne inférieure est déjà garantie par les tests précédents — inutile d'écrire `moy >= 12 && moy < 14`.

À RETENIR L'essentiel des structures alternatives

- `if (condition) { ... }` exécute un bloc seulement si la condition est vraie; `else` traite le cas contraire.
- Une condition combine des comparaisons (`==`, `!=`, `<`, `>=...`) avec les opérateurs logiques `&&` (ET), `||` (OU), `!` (NON), évalués en court-circuit.
- `switch` aiguille selon la valeur d'une expression entière; chaque `case` se termine par `break`, et `default` gère les autres cas.
- Les alternatives s'imbriquent librement; une indentation rigoureuse est indispensable pour s'y retrouver.
- L'opérateur ternaire `(cond) ? a : b` condense une affectation conditionnelle en une ligne.

EXERCICE 4 À toi de jouer

1. Écris un programme qui lit deux entiers *a* et *b* et affiche le plus grand des deux, d'abord avec un `if...else`, puis avec l'opérateur ternaire.
2. Écris un programme qui lit le numéro d'un mois (1 à 12) et affiche son nom en français à l'aide d'un `switch`. Prévois un `default` pour les numéros invalides.
3. Un cybercafé facture l'heure de connexion 300 FCFA, avec une réduction de 20 % au-delà de 5 heures. Écris le programme qui calcule le montant à payer à partir du nombre d'heures saisi.

CORRIGÉ Exercice 1

```
#include <stdio.h>

int main(void) {
    int a, b, max;
    printf("Entrez deux entiers : ");
    scanf("%d %d", &a, &b);

    /* Version 1 : if ... else */
    if (a > b) {
```

```
    max = a;
} else {
    max = b;
}
printf("Version if ... else : max = %d\n", max);

/* Version 2 : operateur ternaire */
max = (a > b) ? a : b;
printf("Version ternaire : max = %d\n", max);
return 0;
}
```

Pour $a = 7$ et $b = 12$, la condition $a > b$ est fausse : dans les deux versions, `max` reçoit la valeur de `b`, soit 12. Les deux écritures sont strictement équivalentes ; le ternaire est simplement plus compact.

CORRIGÉ Exercice 2

```
#include <stdio.h>

int main(void) {
    int mois;
    printf("Entrez le numero du mois (1 a 12) : ");
    scanf("%d", &mois);

    switch (mois) {
        case 1: printf("Janvier\n"); break;
        case 2: printf("Fevrier\n"); break;
        case 3: printf("Mars\n"); break;
        case 4: printf("Avril\n"); break;
        case 5: printf("Mai\n"); break;
        case 6: printf("Juin\n"); break;
        case 7: printf("Juillet\n"); break;
        case 8: printf("Aout\n"); break;
        case 9: printf("Septembre\n"); break;
        case 10: printf("Octobre\n"); break;
        case 11: printf("Novembre\n"); break;
        case 12: printf("Decembre\n"); break;
        default: printf("Numero de mois invalide.\n");
    }
    return 0;
}
```

Le `break` est indispensable après chaque `case` pour éviter l'effet de cascade (fall-through). Le `default` capture toute valeur hors intervalle $[1; 12]$.

CORRIGÉ Exercice 3

```
#include <stdio.h>

int main(void) {
    int heures;
    float montant;
    const float TARIF = 300.0f;
    const float TAUX_REDUCTION = 0.20f; /* 20 % */

    printf("Entrez le nombre d'heures de connexion : ");
    scanf("%d", &heures);

    if (heures < 0) {
        printf("Erreur : nombre d'heures negatif.\n");
    } else if (heures <= 5) {
        montant = heures * TARIF;
        printf("Montant a payer : %.0f FCFA\n", montant);
    } else {
        /* Premières 5h au tarif plein, le reste avec réduction */
        montant = 5 * TARIF + (heures - 5) * TARIF * (1 - TAUX_REDUCTION);
        printf("Montant a payer (avec reduction) : %.0f FCFA\n", montant);
    }
    return 0;
}
```

Explication : Pour 7 heures, le calcul est $5 \times 300 + (7 - 5) \times 300 \times 0,8 = 1500 + 480 = 1980$ FCFA. L'utilisation de constantes (`const`) rend le code maintenable : si le tarif change, on modifie une seule ligne.

Tu sais désormais faire prendre des décisions à tes programmes. Mais un programme qui décide doit souvent aussi répéter : saisir à nouveau tant que la note est invalide, parcourir les 60 élèves d'une classe, additionner des notes une par une... C'est l'objet de la prochaine section : les boucles `for`, `while` et `do...while`.

5 BOUCLES : FOR, WHILE, DO...WHILE ET TRADUCTION D'ALGORITHMES ITÉRATIFS

Dans la section précédente, nous avons vu comment faire choisir le programme entre plusieurs chemins d'exécution grâce aux structures alternatives (`if`, `switch`). Mais un ordinateur tire sa vraie puissance de sa capacité à répéter une tâche des milliers de fois sans se fatiguer et sans erreur. C'est le rôle des **boucles** (ou itérations).

Imaginez que vous deviez saisir les notes de 50 élèves pour calculer la moyenne de la classe. Sans boucle, vous écririez 50 fois `scanf` et 50 fois l'addition. Avec une boucle, vous écrivez la logique une seule fois et vous dites : « Répète cela tant qu'il reste des élèves ». C'est l'automatisation par excellence.

5.1 Qu'est-ce qu'une boucle ? Définition et invariant

DÉFINITION Boucle (itération)

Une **boucle** est une structure de contrôle qui permet d'exécuter de manière répétée un bloc d'instructions (le **corps de la boucle**) tant qu'une **condition de continuation** (ou condition d'arrêt) reste vraie (ou tant qu'elle n'est pas devenue fausse).

Chaque passage dans le corps de la boucle s'appelle une **itération**. Pour qu'une boucle soit correcte et termine, trois éléments doivent être maîtrisés :

1. **L'initialisation** : Mise en état des variables avant la première itération (ex : compteur à 0, somme à 0).
2. **Le test** : Évaluation de la condition avant (ou après) chaque itération.
3. **La modification (pas)** : Mise à jour des variables de contrôle à l'intérieur du corps (ou dans l'en-tête pour `for`) pour faire progresser la boucle vers sa fin.

À RETENIR Invariant de boucle (notion intuitive)

Un **invariant de boucle** est une propriété (relation entre les variables) qui est vraie avant la première itération, restée vraie après chaque itération, et donc vraie à la sortie de la boucle. C'est l'outil mental du programmeur pour raisonner sur la correction de son code sans l'exécuter.

Exemple : Pour calculer la somme $S = 1 + 2 + \dots + n$ avec un compteur i .

- **Invariant** : « À l'entrée de l'itération i , la variable `somme` vaut $1 + 2 + \dots + (i - 1)$ ».
- **Initialisation** : $i = 1$, `somme`=0. La somme des 0 premiers entiers est bien 0. L'invariant tient.
- **Corps** : `somme += i`; `i++`; Après, `somme` vaut $1 + \dots + (i - 1) + i = 1 + \dots + i$ et i devient $i + 1$. L'invariant tient pour l'itération suivante.
- **Arrêt** : Quand $i = n + 1$, la condition $i \leq n$ devient fausse. L'invariant dit : `somme` vaut $1 + \dots + n$. C'est le résultat cherché.

5.2 La boucle `for` : l'itération pilotée par un compteur

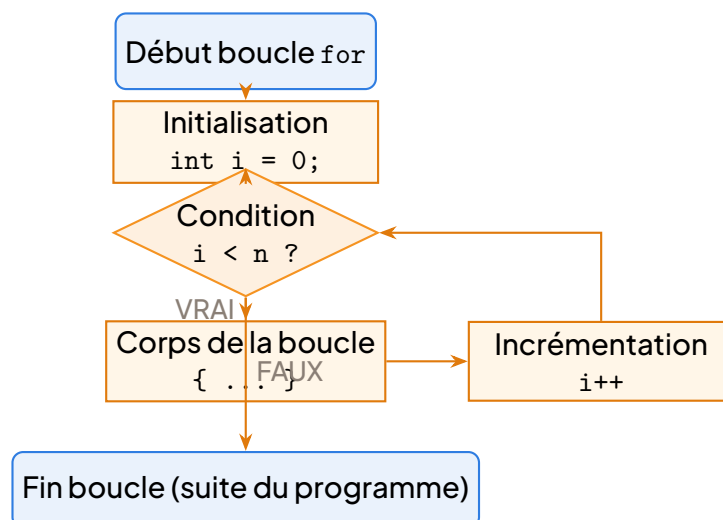
C'est la boucle la plus utilisée quand le nombre d'itérations est connu à l'avance (ou facile à calculer). Elle regroupe les trois étapes (init, test, pas) sur une seule ligne.

PROPRIÉTÉ Syntaxe complète de la boucle `for`

```
for ( initialisation ; condition ; incrementation ) {  
    // Corps de la boucle (instruction simple ou bloc { ... })  
}
```

Sémantique exacte (ordre d'exécution) :

1. Exécuter `initialisation` une seule fois au tout début.
2. Évaluer `condition`. Si fausse (0) → sortir de la boucle (aller à l'instruction d'après). Si vraie (non-nul) → continuer.
3. Exécuter le `corps`.
4. Exécuter `incrementation`.
5. Revenir à l'étape 2.



PROPRIÉTÉ Les trois parties sont optionnelles

En C, les trois champs de l'en-tête `for` sont optionnels. On peut écrire :

- `for ( ; condition ; ) { ... } ≡ while (condition) { ... }` (initialisation et incrémentations faites ailleurs).
- `for ( ; ; ) { ... }` → Boucle infinie volontaire (équivalent à `while(1)`).
- `for (int i=0, j=10 ; i<j ; i++, j--) { ... }` → Initialisation et incrémentations multiples séparées par des virgules.

ATTENTION Variable de contrôle modifiée dans le corps

Ne jamais modifier la variable de boucle (ex : `i`) à l'intérieur du corps d'un `for` (sauf cas experts très rares). Cela casse la lisibilité, l'invariant et peut créer des boucles infinies ou des sauts d'itérations imprévisibles. Utilisez une variable distincte si vous avez besoin d'un autre compteur.

EXEMPLE Somme des n premiers entiers avec `for`

```
#include <stdio.h>

int main() {
    int n, i;
    long long somme = 0; // long long pour éviter débordement si n grand

    printf("Entrez un entier positif n : ");
    scanf("%d", &n);

    // Traduction directe de : Pour i de 1 à n faire somme <- somme + i
    for (i = 1; i <= n; i++) { // i=1 (init) ; i<=n (cond) ; i++ (incr)
        somme += i;           // Corps : accumulation
        // Invariant : ici somme == 1+2+...+i
    }

    printf("La somme des %d premiers entiers est : %lld\n", n, somme);
    return 0;
}
```

Structure classique `for`. La variable `i` est déclarée avant la boucle (standard C89/C90) ou dans l'init (C99 : `for(int i=1;...)`). Préférez C99 pour limiter la portée.

5.3 La boucle `while` : test avant l'entrée (pré-conditionnelle)

La boucle `while` est idéale quand le nombre d'itérations n'est **pas connu à l'avance** (ex : lecture jusqu'à fin de fichier, saisie jusqu'à valeur sentinelle, calcul jusqu'à précision souhaitée).

PROPRIÉTÉ Syntaxe et sémantique `while`

initialisation ;	// AVANT la boucle
<code>while</code> (condition) {	// TEST au DÉBUT de chaque tour
// Corps	
modification ;	// DEDANS le corps (souvent à la fin)
}	

Si `condition` est fausse dès le premier test, le corps n'est jamais exécuté (0 itération pos-

sible).

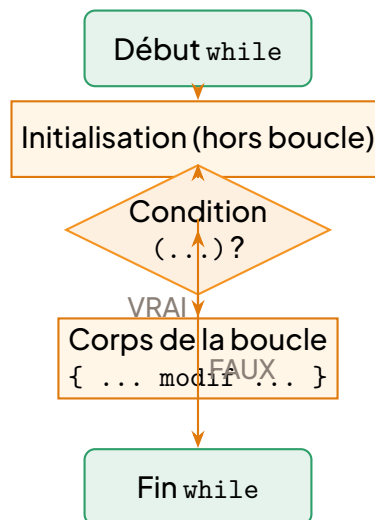


Schéma while : le test est au-dessus du corps. Flèche de retour du corps vers le test.

EXEMPLE Saisie de notes jusqu'à note négative (sentinelle)

```
#include <stdio.h>

int main() {
    float note, somme = 0.0f;
    int count = 0;

    printf("Entrez les notes (note negative pour arreter) :\n");
    // Initialisation : première lecture AVANT le test
    scanf("%f", &note);

    while (note >= 0.0f) {           // Test : continuer tant que valide
        somme += note;               // Traitement
        count++;                   // Comptage
        scanf("%f", &note);        // Modification : lecture SUIVANTE (indispensable !)
    }

    if (count > 0)
        printf("Moyenne de la classe : %.2f\n", somme / count);
    else
        printf("Aucune note saisie.\n");
    return 0;
}
```

Pattern classique while : Read-Process-Loop (Lire, Traiter, Relire). Oublier le `scanf` dans le corps → boucle infinie.

5.4 La boucle `do...while` : test après l'exécution (post-conditionnelle)

Parfois, on **veut** exécuter le corps au moins une fois avant de vérifier la condition. C'est le cas typique des menus interactifs ou de la validation de saisie.

PROPRIÉTÉ Syntaxe et sémantique `do...while`

```
do {  
    // Corps (exécuté AU MOINS UNE FOIS)  
    // ... modification ...  
} while (condition); // <-- Point-virgule OBLIGATOIRE ici !
```

Le test a lieu après le corps. Si `condition` est vraie, on recommence. Si fausse, on sort.

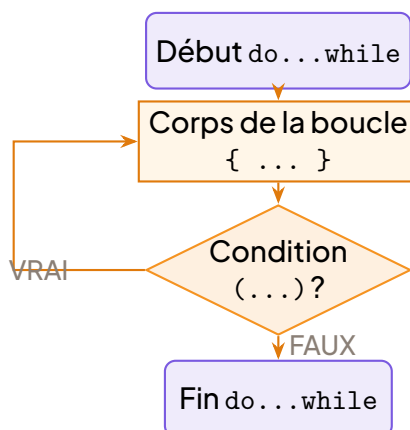


Schéma `do...while` : le corps arrive avant le test. La flèche de retour part du test vers le haut du corps.

ATTENTION Le point-virgule terminal

Oublier le `;` après `while(condition);` est une erreur de syntaxe fréquente. Le compilateur le signalera, mais parfois il peut le confondre avec une instruction vide suivante si on n'est pas vigilant. Mettez-le systématiquement.

EXEMPLE Menu interactif : faire au moins une fois

```
#include <stdio.h>  
  
int main() {  
    int choix;  
    do {
```

```

printf("\n--- MENU PRINCIPAL ---\n");
printf("1. Gestion eleves\n");
printf("2. Gestion notes\n");
printf("3. Quitter\n");
printf("Votre choix : ");
scanf("%d", &choix);

switch(choix){
    case 1: printf("-> Module eleves...\n"); break;
    case 2: printf("-> Module notes...\n"); break;
    case 3: printf("Au revoir !\n"); break;
    default: printf("Choix invalide.\n");
}
} while (choix != 3); // On ressort SI choix == 3
return 0;
}

```

Le menu s'affiche au moins une fois. La condition de `while` est la condition de continuation (on continue tant que ce n'est PAS 3).

5.5 `break` et `continue` : contrôle fin du flux

Parfois, la condition de l'en-tête ne suffit pas pour exprimer une sortie ou un saut complexes.

DÉFINITION `break` et `continue`

- `break`; : Quitte immédiatement la boucle la plus proche (`for`, `while`, `do...while` ou `switch`). L'exécution continue à l'instruction après la boucle.
- `continue`; : Termine l'itération courante immédiatement et passe à l'itération suivante (exécute l'incrément dans `for`, réévalue la condition dans `while`/`do...while`).

ATTENTION Piège `continue` dans `while` / `do...while`

Dans un `for`, `continue` exécute bien l'incrément (`i++`). Dans un `while`, `continue` saute directement au `while(cond)` sans exécuter le reste du corps. Si votre incrément est après le `continue`, elle ne sera pas faite → boucle infinie assurée.

```

// DANGEREUX :
while (i < 10) {
    if (i == 5) continue; // Saute l'incrément !
    i++; // Jamais atteint si i==5 -> Boucle infinie sur i=5
}

```

EXEMPLE Recherche du premier multiple de 7 supérieur à 100 (break)

```
#include <stdio.h>

int main() {
    int i;
    for (i = 101; ; i++){ // for(;;) boucle infinie volontaire
        if (i % 7 == 0) {
            printf("Premier multiple de 7 > 100 : %d\n", i);
            break; // Sortie immédiate de la boucle for
        }
    }
    // Suite du programme ici
    return 0;
}
```

Utilisation de `for(;;)` avec `break` pour une recherche à borne inconnue.

EXEMPLE Somme des pairs seulement (continue)

```
#include <stdio.h>

int main() {
    int i, somme = 0;
    for (i = 1; i <= 10; i++){
        if (i % 2 != 0) // Si impair
            continue; // Passe direct à i++ (itération suivante)
        somme += i; // N'atteint que pour les pairs
    }
    printf("Somme des pairs 1..10 = %d\n", somme); // 2+4+6+8+10 = 30
    return 0;
}
```

5.6 Traduction systématique : de l'algorithme (français) au C

Voici la **méthode pas à pas** pour traduire n'importe quel algorithme itératif (écrit en pseudo-code style Bac/Probatoire) en C valide.

MÉTHODE Traduction Algorithme → C (Boucles)

1. Identifier le type de boucle :

- « Pour i de a à b faire » → `for (int i=a; i<=b; i++)` (ou $i < b$ selon borne).
- « Tant que condition faire » → `while (condition) { ... }`.

- « Répéter ... Jusqu'à condition » → `do { ... } while (!(condition));` (Attention : condition d'arrêt en algo = condition de continuation en C ⇒ nier la condition).
2. Repérer l'initialisation : Variables à initialiser avant la boucle (compteurs, accumulateurs, drapeaux/flags, première lecture pour `while`).
 3. Repérer la modification (pas) : Comment les variables évoluent dans le corps. Placer au bon endroit (en-tête `for` ou fin de corps `while`).
 4. Traduire le corps : Chaque instruction algo → instruction C (affectation → `=`, affichage → `printf`, saisie → `scanf`, si → `if`).
 5. Vérifier l'invariant mental : « Si j'entre dans le corps avec ces valeurs, en ressortirai-je avec les bonnes valeurs pour le tour d'après ? »
 6. Tester les bornes : Cas $n = 0$, $n = 1$, $n = MAX$. La boucle s'exécute-t-elle le bon nombre de fois ?

EXERCICE RÉSOLU 5**Traduction : Calcul de la factorielle $n!$**

Énoncé : Écrire un programme C qui demande un entier $n \geq 0$ et affiche $n!$ (factorielle).

Rappel : $0! = 1$, $n! = 1 \times 2 \times \dots \times n$. Utiliser `unsigned long long` pour aller jusqu'à $20!$.

Analyse algorithmique :

- Entrée : n (entier ≥ 0).
- Sortie : $fact$ (entier grand).
- Algo : $fact \leftarrow 1$; Pour i de 1 à n faire $fact \leftarrow fact \times i$; Fin Pour.

Traduction C commentée :

```
#include <stdio.h>

int main() {
    int n, i;
    unsigned long long fact = 1ULL; // 1. Initialisation accumulateur (type large)

    printf("Entrez n (0 <= n <= 20) : ");
    scanf("%d", &n);

    // 2. Validation entrée (défense)
    if (n < 0 || n > 20) {
        printf("Erreur : n hors limites (0-20).\n");
        return 1;
    }

    // 3. Boucle FOR : traduction directe "Pour i de 1 à n"
    for (i = 1; i <= n; i++) {
        fact = fact * i; // ou fact *= i;
        // Invariant : fact == i! à la fin de cette itération
    }
}
```

```

    }

    printf("%d! = %llu\n", n, fact); // %llu pour unsigned long long
    return 0;
}

```

Tests de non-régression (à faire mentalement ou en TP) :

n	Parcours boucle	Résultat
0	Condition $i \leq 0$ fausse dès le début ($i=1$). Corps ja- mais exécuté.	fact reste 1. Correct ($0!=1$).
1	1 itération : $i = 1 \rightarrow fact = 1 \times 1 = 1$.	1. Correct.
5	$i = 1..5 : 1, 2, 6, 24, 120$.	120. Correct.

SOLUTION

Solution détaillée : Le code ci-dessus est la traduction fidèle. Points clés :

- Type `unsigned long long` (au moins 64 bits) : $20! \approx 2,4 \times 10^{18}$ tient dans 64 bits ($2^{64} - 1 \approx 1,8 \times 10^{19}$). $21!$ déborde.
- Initialisation `fact=1` avant la boucle (élément neutre de la multiplication).
- Borne $i \leq n$ inclusive car algo dit « de 1 à n ».
- Pas d'instruction `continue` ni `break` nécessaire ici, logique linéaire simple.

5.7 Boucles imbriquées : tables de multiplication et motifs

Une boucle dans une autre. La boucle **extérieure** contrôle les « lignes » (ou les grands cycles), la boucle **intérieure** contrôle les « colonnes » (ou les répétitions rapides). Le compteur intérieur doit être **réinitialisé** à chaque tour de la boucle extérieure.

EXEMPLE Table de multiplication de 1 à 10 (boucles imbriquées)

```

#include <stdio.h>

int main() {
    int i, j; // i = multiplicateur (ligne), j = multiplicande (colonne)

    printf(" "); // Décalage pour en-tête colonnes
    for (j = 1; j <= 10; j++) printf("%4d", j);
    printf("\n");

    for (i = 1; i <= 10; i++) {          // BOUCLE EXTERIEURE : Lignes 1..10
        printf("%2d |", i);              // En-tête de ligne
        for (j = 1; j <= 10; j++) {      // BOUCLE INTERIEURE : Colonnes 1..10
            printf("%4d", i * j);        // Produit aligné sur 4 chars
        }
    }
}

```

```
        printf("\n");           // Retour à la ligne APRES la boucle intérieure
    }
    return 0;
}
```

Sortie : grille 10×10 alignée. Notez %4d pour l'alignement. La variable `j` est réinitialisée à 1 à chaque itération de `i`.

ATTENTION Variables de boucle distinctes

N'utilisez jamais la même variable de contrôle (`i`) pour la boucle extérieure et la intérieure. La boucle intérieure écrase la valeur de `i`, la boucle extérieure ne terminera jamais (ou sautera des valeurs). Utilisez `i`, `j`, `k` (convention mathématique) ou des noms explicites (`ligne`, `col`).

5.8 Boucles infinies volontaires et arrêt contrôlé

Parfois, on veut une boucle qui tourne « pour toujours » (serveur, menu principal, jeu vidéo) et on sort de l'intérieur via `break`, `return` ou `exit()`.

PROPRIÉTÉ Idiomes C pour boucle infinie

- `while (1) { ... }` : Classique, clair. 1 est vrai.
- `for (;;) { ... }` : Idiomatique C, historiquement plus rapide (pas de test), équivalent aujourd'hui.
- `do { ... } while (1);` : Rare, corps exécuté au moins une fois.

EXPÉRIENCE TP Guidé : Calculatrice de notes robuste (Intégration)

Objectif : Écrire un programme qui demande des notes (0 à 20) jusqu'à saisie d'une note négative, calcule la moyenne, le maximum, le minimum et le nombre de notes ≥ 10 (admis). Gérer le cas « aucune note ». Outils : Code::Blocks ou Dev-C++ (compilateur GCC), terminal. Étapes :

1. Créer un projet « Console Application ».
2. Déclarer : `somme=0.0`, `max=-1.0`, `min=21.0`, `count=0`, `admis=0`, `note`.
3. Boucle `do...while` (au moins une saisie tentée) ou `while` avec lecture initiale.
4. Dans la boucle : `if (note < 0) break;` (sortie sentinelle).

5. **Validation** : `if (note > 20) { printf("Note invalide >20\n"); continue; } (re-saisie).`
6. **Mise à jour stats** : `somme+=note; count++; if(note>max) max=note; if(note<min) min=note; if(note>=10) admis++;`.
7. **Après boucle** : `if(count>0) { moy=somme/count; printf(...); } else { printf("Aucune note\n"); }.`
8. **Tester** : Cas normaux, `note > 20` (continue), note négative immédiate (`count=0`), note exactement 0 et 20.

Observation attendue : Le `continue` permet de rejeter les notes `> 20` sans casser les stats ni compter l'erreur. Le `break` sort proprement. Interprétation : C'est le pattern « Boucle infinie + validation + break/continue » utilisé dans tout logiciel interactif.

LE SAVAIS-TU ? Le mot-clé `register` : vestige du passé

Dans les vieux codes C (années 80–90), on voyait `register int i;`. Cela suggérait au compilateur : « Essaie de mettre `i` dans un registre du processeur (CPU) au lieu de la RAM, car je l'utilise tout le temps (compteur de boucle) ». Aujourd'hui, les optimiseurs (GCC -O2, -O3, Clang, MSVC) font un bien meilleur travail que le programmeur pour l'allocation des registres. Le mot-clé `register` est obsolète (déprécié en C11, supprimé en C23 pour les objets, gardé pour `register storage class` seulement). Ne l'utilisez plus. Écrivez du code clair ; le compilateur optimisera.

EXERCICE 5 Entraînement : Boucles et invariants

1. Écrire une boucle `while` qui affiche les puissances de 2 (1, 2, 4, 8, ...) tant qu'elles sont ≤ 1000 . Utiliser une variable `puissance` initialisée à 1 et multipliée par 2 à chaque tour.
2. Traduire en C (avec `for`) l'algorithme suivant :

```
S <- 0
Pour k de 1 à 10 faire
  Si k est pair alors
    S <- S + k
  Fin Si
Fin Pour
Afficher S
```

Quel est l'invariant de boucle sur `S` ?

3. Écrire un programme qui demande un entier `N` et affiche un triangle rectangle de hauteur `N` avec des `*` (ex : `N=4` → `*`, `,`, `*`, `****`). Utiliser deux boucles `for` imbriquées.

4. Piège : Que fait ce code ? Combien de fois « Bonjour » s'affiche-t-il ?

```
int i = 0;
while (i < 5) {
    if (i == 2) continue;
    printf("Bonjour %d\n", i);
    i++;
}
```

CORRIGÉ Exercice 1

1. Puissances de 2 :

```
int puissance = 1;
while (puissance <= 1000) {
    printf("%d", puissance);
    puissance *= 2; // Modification INDISPENSABLE dans le corps
}
// Affiche : 1 2 4 8 16 32 64 128 256 512 (1024 > 1000 stop)
```

2. Somme des pairs de 1 à 10 :

```
int S = 0, k;
for (k = 1; k <= 10; k++) {
    if (k % 2 == 0) S += k;
}
printf("S = %d\n", S); // 30
```

Invariant : À l'entrée de l'itération k , S vaut la somme des entiers pairs strictement inférieurs à k .

3. Triangle :

```
int N, lig, col;
scanf("%d", &N);
for (lig = 1; lig <= N; lig++) {
    for (col = 1; col <= lig; col++) printf("★");
    printf("\n");
}
```

4. Piège continue : Boucle infinie.

- $i = 0$: affiche « Bonjour 0 », i devient 1.
- $i = 1$: affiche « Bonjour 1 », i devient 2.
- $i = 2$: if vrai → continue → saute $i++$ → retour while($2 < 5$) VRAI.

- *i* reste 2 pour toujours. « Bonjour 2 » jamais affiché. Programme planté (Ctrl+C pour arrêter).

Correction : Placer `i++` avant `continue` ou utiliser `for`.

6 FONCTIONS MATHÉMATIQUES (MATH.H) ET EXÉCUTION PAS À PAS (DÉBOGAGE)

Après avoir maîtrisé les boucles `for`, `while` et `do...while` qui permettent de répéter des traitements, nous abordons maintenant deux compétences indispensables pour tout programmeur C : l'utilisation de la bibliothèque mathématique standard pour effectuer des calculs scientifiques (racines, puissances, trigonométrie) et la maîtrise du débogueur intégré à l'IDE (Code::Blocks ou Dev-C++) pour traquer les erreurs logiques invisibles à la compilation. Ces deux outils transforment un « code qui compile » en un « programme fiable et vérifié ».

6.1 La bibliothèque mathématique `<math.h>` : inclusion et édition de liens

En algorithmique, nous écrivions `racine(x)` ou `cos(x)` comme si ces fonctions existaient par magie. En C, elles sont regroupées dans la bibliothèque standard `math.h`. Contrairement à `stdio.h` (entrées/sorties) qui est liée automatiquement, la bibliothèque mathématique nécessite une action explicite lors de l'édition de liens (**linking**), étape où l'exécutable final est assemblé à partir du code objet et des bibliothèques précompilées.

PROPRIÉTÉ Inclusion et liaison de `math.h`

Pour utiliser les fonctions mathématiques en C :

1. Inclure l'en-tête au début du fichier source : `#include <math.h>`.
2. Lors de la compilation avec `gcc` (ligne de commande), ajouter l'option `-lm` (minuscule L, m pour math) après le nom du fichier source :
`gcc programme.c -o programme -lm`.
3. Dans Code::Blocks : Project → Build options → Linker settings → Add → taper `m` (ou cocher `Link math library` si disponible).
4. Dans Dev-C++ : Project → Project Options → Parameters → Linker → ajouter `-lm`.

Oublier `-lm` provoque une erreur de liaison du type `undefined reference to 'sqrt'` (ou `cos`, `pow`, etc.), alors que la compilation (création du `.o`) a réussi.

MÉTHODE Compiler un programme utilisant `math.h` avec GCC (terminal)

1. Écrire le code dans `calcul.c` (avec `#include <math.h>`).
2. Ouvrir un terminal dans le dossier du fichier.
3. Taper : `gcc calcul.c -o calcul -lm` (`-o calcul` nomme l'exécutable, `-lm` lie la lib `math`).
4. Exécuter : `./calcul` (Linux/macOS) ou `calcul.exe` (Windows).

ATTENTION Piège fréquent : `sqrt` attend un `double`

Les fonctions de `math.h` travaillent en double précision (`double`). Si vous passez un `int` (ex. `sqrt(25)`), une conversion implicite `int` → `double` a lieu : c'est correct pour de petites valeurs. Mais pour de très grands entiers (proches de `INT_MAX` $\approx 2 \times 10^9$), la conversion en `double` peut perdre de la précision car le `double` IEEE 754 ne code exactement les entiers que jusqu'à 2^{53} .

- Bonne pratique : caster explicitement si nécessaire : `sqrt((double)n)` ou écrire `25.0` au lieu de `25`.
- `fabs` (valeur absolue flottante) s'utilise pour `float`/`double`; pour les entiers, utiliser `abs` (dans `stdlib.h`) ou `labs` (`long`).

LESAVAIS-TU ? La constante `M_PI` n'est pas standard C !

La norme ISO C (C99, C11, C18) ne définit pas `M_PI`. C'est une extension GNU (Linux/-MinGW) activée si la macro `_GNU_SOURCE` (Linux) ou `_USE_MATH_DEFINES` (Windows/MSVC, avant `#include <math.h>`) est définie.

```
#define _USE_MATH_DEFINES // Obligatoire sous Windows (MSVC, MinGW) AVANT #include <math.h>
#include <math.h>
#include <stdio.h>

int main() {
    printf("PI = %.10f\n", M_PI); // Affiche 3.1415926536
    return 0;
}
```

En examen ou pour du code portable, définissez votre propre constante : `#define PI 3.14159265358979323846`.

6.2 Fonctions usuelles de `math.h` et conversion degrés/radians

Voici les fonctions les plus utilisées au programme de Première TI. Toutes prennent et retournent

des `double` (angles en radians pour la trigonométrie).

Fonction	Rôle	Exemple d'utilisation
<code>sqrt(x)</code>	Racine carrée de x ($x \geq 0$)	<code>d = sqrt(25.0); // 5.0</code>
<code>fabs(x)</code>	Valeur absolue ($ x $) flottante	<code>ecart = fabs(a - b);</code>
<code>pow(x, y)</code>	Puissance x^y	<code>volume = pow(rayon, 3);</code>
<code>cos(x)</code>	Cosinus de x (radians)	<code>c = cos(PI/3); // 0.5</code>
<code>sin(x)</code>	Sinus de x (radians)	<code>s = sin(PI/2); // 1.0</code>
<code>tan(x)</code>	Tangente de x (radians)	<code>t = tan(PI/4); // 1.0</code>
<code>exp(x)</code>	Exponentielle e^x	<code>y = exp(1.0); // e</code>
<code>log(x)</code>	Logarithme népérien $\ln(x)$ ($x > 0$)	<code>l = log(10.0);</code>
<code>log10(x)</code>	Logarithme base 10 $\log_{10}(x)$	<code>db = 20 * log10(ratio);</code>

Conversion degrés $\rightarrow$ radians : radians = degrés $\times \frac{\pi}{180}$.

EXEMPLE Calcul de distance euclidienne entre deux points

On veut la distance entre $A(x_A, y_A)$ et $B(x_B, y_B)$: $d = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$.

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846 // Portable

int main() {
    double xA, yA, xB, yB, dx, dy, distance;

    printf("Coordonnees de A (x y) : ");
    scanf("%lf %lf", &xA, &yA);
    printf("Coordonnees de B (x y) : ");
    scanf("%lf %lf", &xB, &yB);

    dx = xB - xA;
    dy = yB - yA;
    // pow(dx, 2) est plus lisible que dx*dx pour des formules complexes
    distance = sqrt(pow(dx, 2) + pow(dy, 2));

    printf("Distance AB = %.3f\n", distance);
    return 0;
}
```

Test : $A(0,0)$, $B(3,4) \rightarrow dx=3$, $dy=4$, $3^2 + 4^2 = 25$, $\sqrt{25} = 5.000$.

EXEMPLE Trigonométrie : calcul de la hauteur d'un arbre

Depuis un point au sol, on mesure l'angle d'élévation $\alpha = 35^\circ$ vers la cime d'un arbre. Distance horizontale $D = 12$ m. Hauteur $h = D \times \tan(\alpha)$.

```
#include <stdio.h>
#include <math.h>
#define PI 3.14159265358979323846

int main() {
    double angle_deg = 35.0;
    double distance = 12.0;
    double angle_rad, hauteur;

    // Conversion degrés -> radians
    angle_rad = angle_deg * PI / 180.0;

    hauteur = distance * tan(angle_rad); // tan attend des radians !

    printf("Angle : %.1f deg -> %.4f rad\n", angle_deg, angle_rad);
    printf("Hauteur de l'arbre : %.2f m\n", hauteur);
    return 0;
}
```

Résultat : $35 \times \pi / 180 \approx 0,6109$ rad, $\tan(0,6109) \approx 0,7002$, $h \approx 8,40$ m.

6.3 Exécution pas à pas et débogage avec Code ::Blocks

Un programme qui compile sans erreur n'est pas forcément correct. Les **erreurs logiques** (mauvaise formule, condition inversée, boucle infinie, indice de tableau hors limites) ne sont détectées qu'à l'exécution. Le **débogueur** (debugger) permet d'observer le programme « en direct » : arrêter l'exécution à une ligne précise, examiner le contenu des variables, avancer instruction par instruction.

DÉFINITION Point d'arrêt (Breakpoint)

Un point d'arrêt (**breakpoint**) est un marqueur placé sur une ligne de code source qui indique au débogueur : « Stoppe l'exécution avant d'exécuter cette ligne ». L'exécution passe en mode pas à pas (**step-by-step**). On le pose en cliquant dans la marge gauche de l'éditeur (ou F9 dans Code ::Blocks) : un point rouge apparaît.

PROPRIÉTÉ Commandes principales du débogueur (Code ::Blocks / GDB)

Une fois l'exécution arrêtée sur un breakpoint (cible Debug active) :

- Continuer (F8 par défaut) : Reprend l'exécution normale jusqu'au prochain breakpoint ou à la fin.
- Pas à pas détaillé / Step Over (Shift+F7 par défaut) : Exécute la ligne courante et s'arrête à la suivante. N'entre pas dans les fonctions appelées (traite `printf`, `scanf`, `sqrt` comme une seule instruction).

- Pas à pas / Step Into (F7 par défaut) : Exécute la ligne courante ; si c'est un appel de fonction utilisateur, entre dans cette fonction et s'arrête à sa première ligne. Utile pour déboguer vos propres fonctions.
- Sortir de fonction / Step Out (Ctrl+F7 par défaut) : Termine l'exécution de la fonction courante et revient à l'appelant.
- Fenêtre **Watches** (Observateurs) : Permet d'ajouter des expressions (ex. `i`, `delta`, `x1`, `tab[j]`) pour surveiller leur valeur en temps réel.
- Fenêtre **Call Stack** (Pile d'appels) : Montre la chaîne d'appels de fonctions active (qui a appelé qui).
- Fenêtre **Locals** : Affiche automatiquement toutes les variables locales de la fonction en cours avec leur valeur et type.

Note : Les raccourcis clavier peuvent varier selon la configuration ; utilisez le menu **Debug** pour être sûr.

Éditeur : debug_demo.c (Ligne 10 active → flèche jaune)

```
// debug_demo.c
#include <stdio.h>
#include <math.h>

int main()
{
    double a=2, b=5, c=2;
    10 delta = b*b - 4*a*c;
```

Variables locales (après exécution ligne 10)

Watch / Locals		
a	2.000000	double
b	5.000000	double
c	2.000000	double
delta	9.000000	double
x1	<non initialisé>	double

Console / Logs du débogueur (GDB)

```
Building to ensure sources are up-to-date...
Selecting target: Debug
Adding source file: debug_demo.c
Starting debugger: done
Setting breakpoints
Debugger name and version: GNU gdb (GDB) 12.1
Breakpoint 1, main () at debug_demo.c:10
10 delta = b*b - 4*a*c;
(gdb) _
```

Interface du débogueur Code ::Blocks (mode Debug) : éditeur avec break-point (point rouge + flèche jaune), fenêtre **Watches** montrant les valeurs actuelles des variables (`delta=9`), pile d'appels (**Call Stack**) et console GDB.

EXPÉRIENCE TP guidé : Débogage pas à pas d'un calcul de discriminant

Objectif : Observer l'évolution des variables dans une structure `if/else` et vérifier la racine carrée. **Logiciel :** Code ::Blocks (projet `Console application`, C). **Étapes :**

1. Créer un nouveau projet `DebugDemo`. Copier le code de l'exemple `debug_demo.c` ci-dessus (équation $2x^2 + 5x + 2 = 0$, $\Delta = 9$).
2. Compiler en mode Debug (menu `Build` → `Build` ou `Ctrl+F9`) : vérifiez que la cible active est Debug (pas Release), sinon les informations de débogage sont absentes.
3. Cliquer dans la marge gauche de la ligne `delta = b*b - 4*a*c;` pour poser un breakpoint (point rouge).
4. Lancer le débogueur : `Debug` → `Start/Continue` (ou `F8`). Le programme s'arrête avant cette ligne. La flèche jaune est sur le breakpoint.
5. Ouvrir `Debug` → `Debugging windows` → `Watches` et `Call Stack`.
6. Dans `Watches`, ajouter `delta`, `x1`, `x2` (clic droit → `Add watch`).
7. Appuyer sur `Shift+F7` (Step Over) : la ligne `delta = ...` s'exécute. Regarder `Watches` : `delta` passe de <non initialisé> à `9.000000`.
8. Appuyer encore sur `Shift+F7` : on passe le `if (delta < 0)` (faux), on entre dans le `else`.
9. Appuyer sur `Shift+F7` sur `x1 = ...` : `x1` devient `-0.500000`.
10. Appuyer sur `Shift+F7` sur `x2 = ...` : `x2` devient `-2.000000`.
11. Appuyer sur `F8` (Continue) : le programme se termine normalement.

Observation : Le mode pas à pas révèle l'ordre exact d'exécution et les valeurs intermédiaires. C'est la seule méthode fiable pour comprendre une boucle complexe ou une condition mal écrite. Interprétation : Si `delta` restait 0 ou devenait négatif alors qu'il devrait être positif, l'erreur est dans la formule `b*b - 4*a*c` (ex. priorité des opérateurs, types entiers vs flottants).

6.4 Exemple complet : Résolution d'une équation du second degré

Cet exemple synthétise : `math.h` (`sqrt`), structures alternatives (`if/else if/else`), entrées/sorties formatées, et algorithme classique du discriminant.

EXERCICE RÉSOLU 6 Équation du second degré $ax^2 + bx + c = 0$

Énoncé : Écrire un programme C qui lit trois coefficients réels a, b, c ($a \neq 0$), calcule le discriminant $\Delta = b^2 - 4ac$, et affiche les solutions réelles s'il y en a, ou un message adapté.

- Si $\Delta > 0$: deux solutions distinctes $x_1 = \frac{-b-\sqrt{\Delta}}{2a}$, $x_2 = \frac{-b+\sqrt{\Delta}}{2a}$.
- Si $\Delta = 0$: une solution double $x_0 = \frac{-b}{2a}$.
- Si $\Delta < 0$: pas de solution réelle (on affiche « Pas de racine réelle »).

SOLUTION

Analyse et algorithme :

1. Déclarer `a`, `b`, `c`, `delta`, `x1`, `x2`, `x0` en double.
2. Lire `a`, `b`, `c` (`scanf` avec `%lf`).
3. Vérifier `a != 0` (sinon équation du 1er degré, hors sujet ici).
4. Calculer `delta = b*b - 4*a*c`.
5. Structure alternative sur le signe de `delta`.
6. Afficher les résultats avec `printf` et format `%.3f`.

Code source complet (`eq_second_deg.c`) :

```
#include <stdio.h>
#include <math.h> // Indispensable pour sqrt()

#define PI 3.14159265358979323846 // Non utilisé ici, mais bonne habitude

int main() {
    double a, b, c;    // Coefficients
    double delta;      // Discriminant
    double x1, x2, x0; // Racines

    // 1. Saisie des coefficients
    printf("Resolution de l'equation a*x^2 + b*x + c = 0\n");
    printf("Entrez a (non nul), b, c (reels) : ");
    // %lf pour lire un double
    if (scanf("%lf %lf %lf", &a, &b, &c) != 3) {
        printf("Erreur de saisie : trois nombres reels attendus.\n");
        return 1; // Code de retour erreur
    }

    // 2. Vérification a != 0
    if (fabs(a) < 1e-12) { // Comparaison flottante avec epsilon
        printf("Coefficient a nul : ce n'est pas une equation du second degre.\n");
        return 1;
    }

    // 3. Calcul du discriminant
    delta = b * b - 4.0 * a * c; // 4.0 force le calcul en double

    // 4. Traitement selon le signe de delta
    if (delta > 1e-12) { // Delta strictement positif (avec marge epsilon)
        x1 = (-b - sqrt(delta)) / (2.0 * a);
        x2 = (-b + sqrt(delta)) / (2.0 * a);
        printf("Delta = %.3f > 0 : Deux solutions reelles distinctes.\n", delta);
        printf("x1 = %.3f\n", x1);
        printf("x2 = %.3f\n", x2);
    }
    else if (fabs(delta) <= 1e-12) { // Delta nul (quasi nul)
        x0 = -b / (2.0 * a);
        printf("Delta = %.3f = 0 : Une solution double.\n", delta);
        printf("x0 = %.3f\n", x0);
    }
}
```

```

    }
    else { // Delta < 0
        printf("Delta = %.3f < 0 : Pas de solution reelle.\n", delta);
        // Optionnel : afficher parties réelles/imaginaires
        // double re = -b/(2*a), im = sqrt(-delta)/(2*a);
        // printf("Solutions complexes : %.3f +/- %.3f i\n", re, im);
    }

    return 0; // Succès
}

```

Explications détaillées :

- `#include <math.h>` : ligne 2, active `sqrt`.
- `scanf(%lf ...)` : `%lf` est le format pour double (`%f` est pour float en `scanf`, alors qu'en `printf %f` marche pour les deux).
- `fabs(a) < 1e-12` : Ne jamais tester `a == 0.0` en flottant ! À cause des erreurs d'arrondi, on teste si la valeur absolue est inférieure à un tout petit seuil (**epsilon**).
- `4.0 * a * c` : écrire 4.0 (et non 4) garantit que la multiplication se fait en double dès le départ, évitant un débordement temporaire en int si `a, b, c` étaient des entiers (ici ce sont des double, mais c'est une habitude de rigueur).
- `sqrt(delta)` : `delta` est double, `sqrt` attend double, c'est parfait. Si `delta < 0`, `sqrt` retourne NaN (Not a Number) et positionne `errno`; notre `if` évite cet appel.
- `return 1;` : code de sortie non-nul signalant une erreur au système d'exploitation (utile dans les scripts).

Tests de validation :

1. `a=2, b=5, c=2` → $\Delta = 25 - 16 = 9 > 0$ → $x_1 = -2.000, x_2 = -0.500$.
2. `a=1, b=-4, c=4` → $\Delta = 16 - 16 = 0$ → $x_0 = 2.000$.
3. `a=1, b=2, c=5` → $\Delta = 4 - 20 = -16 < 0$ → « Pas de solution réelle ».
4. `a=0, b=2, c=3` → Message d'erreur « Coefficient a nul ».

6.5 Exercices d'application

EXERCICE 6 Calcul du volume d'un cylindre

Écrire un programme qui demande le rayon r et la hauteur h d'un cylindre (réels positifs), calcule et affiche son volume $V = \pi r^2 h$ et sa surface totale $S = 2\pi r(r + h)$. Utiliser `M_PI` (avec `#define _USE_MATH_DEFINES` si nécessaire) ou votre constante `PI`. Vérifier que $r > 0$ et $h > 0$ sinon afficher une erreur.

EXERCICE 7 Conversion polaire → cartésien

Un point est donné par ses coordonnées polaires : rayon $\rho \geq 0$ et angle θ en degrés. Écrire un programme qui lit ρ et θ , convertit θ en radians, calcule les coordonnées cartésiennes $x = \rho \cos \theta$, $y = \rho \sin \theta$ et les affiche. Tester avec $\rho = 10$, $\theta = 30^\circ$ ($x \approx 8,66$, $y = 5,00$).

EXERCICE 8 Débogage : Suite de Héron (racine carrée par approximation)

Le programme ci-dessous cherche à calculer $\sqrt{A}$ par la méthode de Héron (ou Babylone) : $u_0 = A$, $u_{n+1} = \frac{1}{2}(u_n + \frac{A}{u_n})$. Il contient trois erreurs logiques. Utilisez le débogueur (breakpoint dans la boucle, Watches sur u , n , A) pour les identifier et les corriger.

```
#include <stdio.h>
#include <math.h>

int main() {
    double A, u, prec = 1e-6;
    int n = 0;
    printf("Entrez A > 0 : ");
    scanf("%f", &A); // Erreur 1
    u = A;
    while (fabs(u - A/u) > prec) { // Erreur 2 : condition de convergence
        u = 0.5 * (u + A / u);
        n++;
        printf("Iteration %d : u = %f\n", n, u);
    }
    printf("Racine approx = %f, sqrt(A) = %f\n", u, sqrt(A));
    return 0;
}
```

Indice : Regardez le format `scanf`, la condition d'arrêt (on compare u_n et u_{n+1} ou u_n^2 et A), et l'initialisation de u .

CORRIGÉ Exercice 3 : Débogage Suite de Héron

Erreur 1 : `scanf("%f", &A) → %f attend float*, A est double. Correction :` `scanf("%lf", &A)`. **Erreur 2 :** Condition `fabs(u - A/u) > prec`. Héron converge vers $\sqrt{A}$. Quand $u \approx \sqrt{A}$, $A/u \approx \sqrt{A}$, donc $u - A/u \approx 0$. Mais si $u_0 = A$ est grand, $A/u_0 = 1$, la différence est grande. La condition standard est `fabs(u*u - A) > prec` (différence au carré) ou `fabs(u - u_prev) > prec` (variation entre itérations). **Erreur 3 (subtile) :** Si A est très grand, $u = A$ peut provoquer un débordement ou une lenteur. Mieux : $u = A/2.0$ ou $u = 1.0$. Code corrigé (extrait) :

```
scanf("%lf", &A); // %lf pour double
u = (A > 1) ? A/2.0 : 1.0; // Meilleure estimation initiale
double u_prev;
do {
    u_prev = u;
```

```
u = 0.5 * (u + A / u);  
n++;  
} while (fabs(u - u_prev) > prec); // Convergence sur la variation
```

À RETENIR Bilan de la section 6

- `#include <math.h>` + compilation avec `-lm` (GCC) ou option « Link math library » (IDE).
- Fonctions clés : `sqrt`, `fabs`, `pow`, `cos`, `sin`, `tan` (angles en radians : $\text{rad} = \text{deg} \times \pi / 180$).
- `M_PI` non standard → définir `#define PI 3.14159...`
- Débogueur : Breakpoint (F9), Step Over (Shift+F7), Step Into (F7), Watch variables, Call Stack. Mode Debug obligatoire.
- Comparaison flottante : toujours avec `epsilon` (`fabs(x-y) < 1e-9`), jamais `==`.
- `scanf : %lf` pour double ; `printf : %f` (ou `%.3f`) pour double.

7 MINI-PROJET D'INTÉGRATION : CALCULATRICE DE NOTES DE CLASSE

Après avoir maîtrisé les fonctions mathématiques de `math.h` et l'exécution pas à pas dans un débogueur (section 6), nous sommes prêts à assembler toutes les briques apprises — variables, tableaux, chaînes de caractères, structures conditionnelles, boucles, fonctions utilisateur, entrées/sorties formatées — pour réaliser un **mini-projet complet et réaliste**. Ce projet synthétise l'essentiel du programme d'algorithmique et de programmation C de Première TI et constitue un excellent entraînement pour l'épreuve pratique du Baccalauréat.

7.1 Cahier des charges et analyse du problème

EXEMPLE Spécifications fonctionnelles

On souhaite écrire un programme en C qui permet à un enseignant de saisir les notes d'une classe et d'en obtenir un bulletin synthétique. Le programme doit :

1. Demander le nombre d'élèves de la classe (entier positif, maximum 50 par exemple).
2. Pour chaque élève :
 - Saisir son nom complet (chaîne de caractères, espaces autorisés).
 - Saisir ses trois notes (entiers ou réels, comprises entre 0 et 20). Toute note hors

intervalle doit être rejetée et redemandée.

3. Calculer la moyenne des trois notes pour chaque élève.
4. Déterminer la mention selon le barème officiel camerounais :
 - [10; 12[: Passable
 - [12; 14[: Assez bien
 - [14; 16[: Bien
 - [16; 20] : Très bien
 - < 10 : Échec (ou "Ajourné")
5. Afficher un tableau récapitulatif aligné avec les colonnes : Nom, Note 1, Note 2, Note 3, Moyenne, Mention.
6. Gérer les cas limites : 0 élève (affichage message et fin), notes négatives ou > 20 (rejet et ressaisie), moyenne exactement sur un seuil (ex : 12.00 → Assez bien).

MÉTHODE Décomposer un problème complexe en sous-problèmes (fonctions)

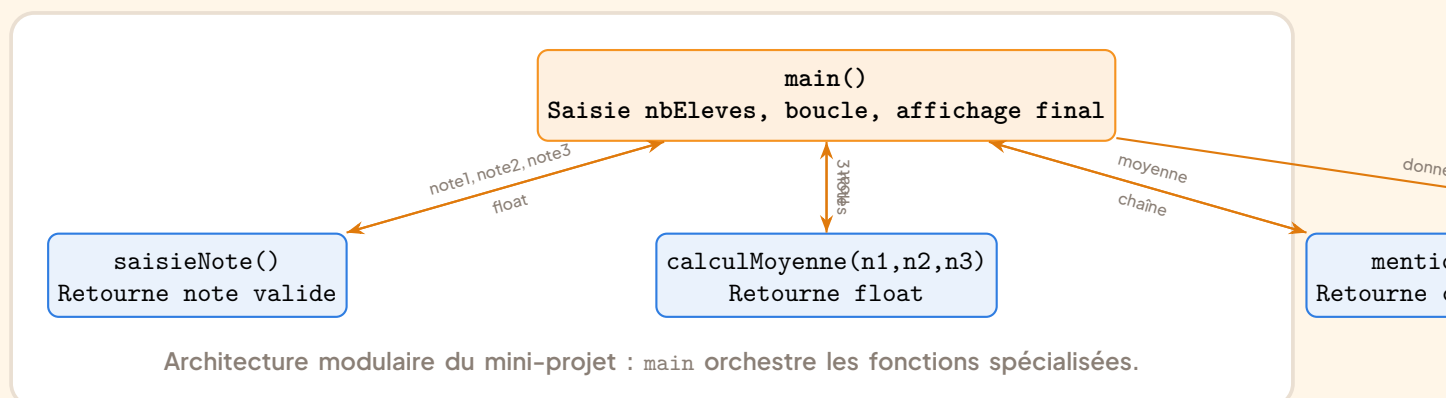
Avant d'écrire la moindre ligne de code dans `main`, un bon programmeur procède par décomposition descendante (approche modulaire) :

1. Identifier les tâches répétitives ou logiquement distinctes : ici, la saisie sécurisée d'une note, le calcul de la moyenne, la conversion moyenne → mention, l'affichage d'une ligne du tableau.
2. Concevoir une fonction par tâche : chaque fonction a une seule responsabilité, des paramètres d'entrée clairs et une valeur de retour (ou un effet de bord maîtrisé).
3. Définir les signatures (prototypes) : nom, type de retour, liste des paramètres. Cela fige l'interface entre `main` et les modules.
4. Coder `main` comme un chef d'orchestre : il déclare les variables, appelle les fonctions dans l'ordre, gère la boucle sur les élèves.
5. Tester chaque fonction isolément (tests unitaires) avant d'intégrer.

Avantages : lisibilité, réutilisabilité, débogage facilité, travail en équipe possible.

7.2 Conception modulaire : structure du programme

Le mini-projet s'organise autour de quatre fonctions auxiliaires et du `main`. Le schéma ci-dessous illustre l'architecture modulaire (flèches = appels, données échangées en paramètres/retour).



7.3 Choix des structures de données

PROPRIÉTÉ Tableaux et chaînes pour stocker les données d'un élève

Pour un élève unique traité à la volée dans la boucle, pas besoin de tableau global d'élèves. On utilise des variables locales dans `main` :

- `char nom[50]` ; — tableau de caractères pour le nom complet (espace inclus). Taille 50 pour marge de sécurité.
- `float notes[3]` ; — tableau de 3 réels pour les trois notes. Indices 0, 1, 2.
- `float moyenne` ; — variable simple pour le résultat du calcul.
- `const char *mentionStr` ; — pointeur sur chaîne littérale retournée par `mention()`.

Si le cahier des charges exigeait de stocker toute la classe pour un tri ultérieur, on aurait déclaré un tableau de structures (`struct Eleve tab[50]` ;), mais ce n'est pas requis ici.

ATTENTION Piège `scanf` vs `fgets` pour les noms composés

`scanf("%s", nom)` s'arrête au premier espace blanc. Pour lire "NGONO Jean Pierre", il faut utiliser `fgets(nom, sizeof(nom), stdin)`. Procédure correcte :

1. Vider le tampon d'entrée résiduel (le `\n` laissé par un `scanf` précédent) : `int c; while((c=getchar())!='\n' && c!=EOF);`.
2. Lire la ligne : `fgets(nom, sizeof(nom), stdin);`.
3. Retirer le `\n` final si présent : `nom[strcspn(nom, "\n")] = '\0';` (nécessite `<string.h>`).

7.4 Implémentation détaillée des fonctions

Nous présentons chaque fonction avec son prototype, son algorithme commenté et son code C complet.

EXEMPLE Fonction `saisieNote` — saisie sécurisée d'une note entre 0 et 20

Rôle : Demander une note à l'utilisateur, rejeter les valeurs hors [0, 20], boucler jusqu'à saisie valide, retourner la note (float).

```
#include <stdio.h>

float saisieNote(int numeroNote){
    float note;
    int ok;
    do{
        printf(" Note %d (0-20) : ", numeroNote);
        ok = scanf("%f", &note);
        // Vider le tampon (entrée invalide ou \n résiduel)
        int c; while ((c = getchar()) != '\n' && c != EOF);
        if (ok != 1 || note < 0.0f || note > 20.0f){
            printf(" -> Erreur : valeur invalide. Recommencez.\n");
            ok = 0; // Force la boucle
        }
    } while (!ok);
    return note;
}
```

Note : `scanf` retourne le nombre d'éléments lus avec succès (1 si un float lu). La boucle `while(getchar()...)` vide le buffer clavier.

EXEMPLE Fonction `calculMoyenne` — moyenne arithmétique de trois notes

Rôle : Recevoir trois float, retourner leur moyenne (float). Fonction pure (pas d'effet de bord).

```
float calculMoyenne(float n1, float n2, float n3){
    return (n1 + n2 + n3) / 3.0f;
}
```

EXEMPLE Fonction `mention` — conversion moyenne → libellé mention (switch)

Rôle : Recevoir la moyenne (float), retourner un pointeur sur chaîne de caractères constante (const char*). Utilisation de switch sur la partie entière ((int)moyenne) pour illustrer la structure alternative multiple demandée au programme.

```
const char* mention(float moy){
    // On travaille sur la partie entière pour le switch
    // Attention : moyenne 12.0 -> int 12 -> case 12 -> "Assez bien" (correct, borne incluse)
    // Moyenne 11.9 -> int 11 -> case 11 -> "Passable" (correct)
    switch ((int)moy){
        case 20 : case 19 : case 18 : case 17 : case 16 :
            return "Tres bien";
    }
```

```

    case 15 : case 14 :
        return "Bien";
    case 13 : case 12 :
        return "Assez bien";
    case 11 : case 10 :
        return "Passable";
    default : // 0 à 9
        return "Echec";
}
}

```

À RETENIR Seuils et switch sur (int)

Le switch ne teste que l'égalité stricte sur des entiers. En castant (int)moy, on discrète la moyenne.

- Moyenne 12.0 → (int)12 → "Assez bien" (seuil inclus).
- Moyenne 11.999 → (int)11 → "Passable".
- Cela respecte l'intervalle [12, 14[pour "Assez bien".

Pour des seuils non entiers ou plus complexes, une chaîne if/else if avec comparaisons >= serait préférable.

EXEMPLE Fonction afficherLigne — affichage formaté d'une ligne du tableau

Rôle : Recevoir le nom, le tableau de 3 notes, la moyenne, la mention ; afficher une ligne alignée avec printf et spécificateurs de largeur.

```

void afficherLigne(const char *nom, float notes[3], float moy, const char *ment) {
    // %-30s : gauche, largeur 30 | %5.2f : droite, largeur 5, 2 décimales
    // %-12s : gauche, largeur 12 pour la mention
    printf("| %-30s | %5.2f | %5.2f | %5.2f | %6.2f | %-12s |\n",
        nom, notes[0], notes[1], notes[2], moy, ment);
}

```

7.5 Fonction principale main : orchestration complète

Voici le programme complet, compilable sous Code ::Blocks, Dev-C++ ou GCC (gcc -std=c99 -Wall -o notes notes.c).

EXERCICE RÉSOLU 7 Programme complet : Calculatrice de notes de classe

Énoncé : Écrire le programme C complet selon le cahier des charges de la section 7.1.

SOLUTION

Solution détaillée :

```

/* Mini-projet 1ere TI : Calculatrice de notes de classe
   Auteur : [Votre Nom] – Lycée [Nom] – Année 2024–2025
   Compilation : gcc -std=c99 -Wall -o notes notes.c
*/
#include <stdio.h>
#include <string.h> // pour strcpy, strlen, strcmp

// --- Prototypes des fonctions auxiliaires ---
float saisieNote(int numeroNote);
float calculMoyenne(float n1, float n2, float n3);
const char* mention(float moy);
void afficherLigne(const char *nom, float notes[3], float moy, const char *ment);
void viderBuffer(); // Utilitaire pour nettoyer stdin

int main() {
    int nbEleves = 0;
    int i;

    // 1. Saisie du nombre d'élèves (avec contrôle)
    printf("=== CALCULATRICE DE NOTES DE CLASSE ===\n");
    do {
        printf("Nombre d'eleves (0 pour quitter, max 50) : ");
        if (scanf("%d", &nbEleves) != 1) {
            viderBuffer();
            nbEleves = -1; // Force erreur
        } else {
            viderBuffer(); // Nettoyer le \n restant
        }
        if (nbEleves < 0 || nbEleves > 50)
            printf("-> Erreur : entrez un nombre entre 0 et 50.\n");
    } while (nbEleves < 0 || nbEleves > 50);

    // Cas limite : 0 élève
    if (nbEleves == 0) {
        printf("Aucun eleve a traiter. Au revoir !\n");
        return 0;
    }

    // 2. En-tête du tableau final
    printf("\n");
    printf("
+-----+-----+-----+-----+-----+
n");
    printf(" | %-30s | %5s | %5s | %5s | %6s | %-12s |\n",
           "Nom", "N1", "N2", "N3", "Moy.", "Mention");
    printf("
+-----+-----+-----+-----+-----+
n");

    // 3. Boucle principale sur les élèves
    for (i = 1; i <= nbEleves; i++) {

```

```

char nom[50];
float notes[3];
float moy;
const char *ment;

printf("\n--- Eleve %d / %d ---\n", i, nbEleves);

// Saisie du nom complet (avec espaces)
printf("Nom complet : ");
if (fgets(nom, sizeof(nom), stdin) != NULL) {
    strcpy(nom, "Inconnu"); // Fallback
} else {
    // Retirer le \n final
    nom[strcspn(nom, "\n")] = '\0';
}
// Si nom vide (juste Entrée), mettre un défaut
if (strlen(nom) == 0) strcpy(nom, "Sans nom");

// Saisie des 3 notes via la fonction sécurisée
notes[0] = saisieNote(1);
notes[1] = saisieNote(2);
notes[2] = saisieNote(3);

// Calculs
moy = calculMoyenne(notes[0], notes[1], notes[2]);
ment = mention(moy);

// Affichage immédiat de la ligne dans le tableau
afficherLigne(nom, notes, moy, ment);
}

// 4. Pied du tableau
printf("
+-----+-----+-----+-----+-----+
n");
printf("Total : %d eleve(s) traite(s).\n", nbEleves);

return 0;
}

// --- Définitions des fonctions ---

void viderBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

float saisieNote(int numeroNote) {
    float note;
    int ok;
    do {
        printf(" Note %d (0-20) : ", numeroNote);
        ok = scanf("%f", &note);
        viderBuffer(); // Essentiel après scanf
        if (ok != 1 || note < 0.0f || note > 20.0f) {
            printf(" -> Erreur : note invalide (doit etre entre 0 et 20).\n");
            ok = 0;
        }
    } while (ok == 0);
}

```

```
    }  
    } while (!ok);  
    return note;  
}  
  
float calculMoyenne(float n1, float n2, float n3){  
    return (n1 + n2 + n3) / 3.0f;  
}  
  
const char* mention(float moy){  
    switch((int)moy){  
        case 20 : case 19 : case 18 : case 17 : case 16 : return "Tres bien";  
        case 15 : case 14 : return "Bien";  
        case 13 : case 12 : return "Assez bien";  
        case 11 : case 10 : return "Passable";  
        default : return "Echec";  
    }  
}  
  
void afficherLigne(const char *nom, float notes[3], float moy, const char *ment){  
    printf(" | %-30s | %5.2f | %5.2f | %5.2f | %6.2f | %-12s |\n",  
        nom, notes[0], notes[1], notes[2], moy, ment);  
}
```

7.6 Test, exécution et capture d'écran simulée

Testons le programme avec un jeu de données représentatif couvrant les cas limites : 3 élèves, notes sur les seuils, nom composé, note invalide rejetée.

```

=== CALCULATRICE DE NOTES DE CLASSE ===
Nombre d'eleves (0 pour quitter, max 50) : 3

--- Eleve 1 / 3 ---
Nom complet :  NGONO Jean Pierre
Note 1 (0-20) : 14
Note 2 (0-20) : 15
Note 3 (0-20) : 16

--- Eleve 2 / 3 ---
Nom complet :  MBOUOMBOUO Aline
Note 1 (0-20) : 25
-> Erreur : note invalide (doit etre entre 0 et 20).
Note 1 (0-20) : 10
Note 2 (0-20) : 10
Note 3 (0-20) : 10

--- Eleve 3 / 3 ---
Nom complet :  KOUAM
Note 1 (0-20) : 12
Note 2 (0-20) : 12
Note 3 (0-20) : 12

+-----+-----+-----+-----+-----+-----+
| Nom | N1 | N2 | N3 | Moy. | Mention |
+-----+-----+-----+-----+-----+-----+
| NGONO Jean Pierre | 14.00 | 15.00 | 16.00 | 15.00 | Bien |
| MBOUOMBOUO Aline | 10.00 | 10.00 | 10.00 | 10.00 | Passable |
| KOUAM | 12.00 | 12.00 | 12.00 | 12.00 | Assez bien |
+-----+-----+-----+-----+-----+-----+
Total : 3 eleve(s) traite(s).

```

Simulation de l'exécution dans la console (Code ::Blocks / terminal). On observe le rejet de la note 25, la saisie de noms composés, et l'alignement parfait du tableau final.

EXEMPLE Analyse des résultats du test

- Élève 1 (NGONO Jean Pierre) : Notes 14, 15, 16 → Moyenne 15.00 → switch sur 15 → "Bien". Correct.
- Élève 2 (MBOUOMBOUO Aline) : Première saisie 25 rejetée (boucle do...while dans saisieNote). Notes finales 10, 10, 10 → Moyenne 10.00 → switch sur 10 → "Passable". Seuil bas inclus. Correct.
- Élève 3 (KOUAM) : Notes 12, 12, 12 → Moyenne 12.00 → switch sur 12 → "Assez bien". Seuil haut inclus pour la mention supérieure. Correct.
- Alignement : Les spécificateurs %30s, %5.2f, %6.2f, %12s garantissent un tableau propre même avec des noms de longueurs variables.

7.7 Exécution pas à pas (débogage) : points d'arrêt stratégiques

Pour valider la logique sans recompiler, placez des **points d'arrêt (breakpoints)** dans votre IDE (Code :: Blocks : F5 pour lancer en mode debug, clic sur la marge gauche pour poser le point rouge) aux emplacements suivants :

MÉTHODE

Points d'arrêt recommandés pour ce mini-projet

1. Ligne `scanf("%d", &nbEleves);` dans `main` : vérifier que la saisie clavier est bien stockée dans `nbEleves` (fenêtre Watch).
2. Entrée de la fonction `saisieNote` : observer la valeur du paramètre `numeroNote` et la variable locale `note` après le `scanf`. Tester une entrée invalide (lettre) pour voir `ok=0` et le nettoyage du buffer.
3. Ligne `return (n1+n2+n3)/3.0f;` dans `calculMoyenne` : vérifier le calcul en virgule flottante.
4. Ligne `switch ((int)moy)` dans `mention` : observer la valeur de `(int)moy` et la branche empruntée (ex : 12 → case 12).
5. Juste avant `afficherLigne` dans la boucle `for` : inspecter le tableau `notes[3]`, `moy`, `ment` et `nom` pour l'élève courant.

Utilisez F7 (Step Into) pour entrer dans les fonctions, F8 (Step Over) pour les exécuter sans entrer, F9 (Continue) pour aller au point d'arrêt suivant.

7.8 Extensions possibles (pour aller plus loin)

LESAVAIS-TU ?

Ce mini-projet au cœur du Bac Pratique TI

L'épreuve pratique du Baccalauréat Camerounais (Première et Terminale TI) demande souvent d'écrire un programme complet en 2h–3h. Ce mini-projet en est une réplique fidèle : saisie contrôlée, tableaux, chaînes, fonctions, boucles, `switch`, affichage formaté. Le maîtriser, c'est s'assurer 15–20 points sur 20 à la partie "Programmation C". Entraînez-vous à le recoder de mémoire en 45 minutes chrono !

Une fois le programme de base validé, vous pouvez l'enrichir (excellent exercice de révision) :

- **Statistiques de classe** : Calculer et afficher la moyenne générale de la classe, la meilleure et la pire moyenne, le nombre d'admis ($moy \geq 10$).
- **Tri** : Trier le tableau des élèves par moyenne décroissante (tri à bulles ou sélection sur un tableau de `struct`).
- **Fichiers** : Sauvegarder le bulletin dans un fichier `bulletin.txt` (`fopen`, `fprintf`, `fclose`) — notion de Terminale mais accessible.

- **Interface menu** : Boucler sur un menu `switch` (1. Saisir, 2. Afficher, 3. Sauvegarder, 4. Quitter).

EXERCICE 9 Entraînement personnel

Modifiez le programme pour :

1. Ajouter une fonction `statistiquesClasse` qui reçoit un tableau de moyennes et le nombre d'élèves, et affiche : moyenne de la classe, max, min, % d'admis.
2. Remplacer le `switch` de mention par une chaîne `if/else if` utilisant des comparaisons `>=` sur `float` (ex: `if (moy >= 16) ... else if (moy >= 14) ...`). Vérifiez que les seuils sont identiques.
3. Gérer le cas où l'utilisateur tape une lettre au lieu d'un nombre pour le nombre d'élèves (le programme ne doit pas planter en boucle infinie).

CORRIGÉ Exercice 1 — Pistes

1. Il faut stocker les moyennes dans un tableau `float moyennes[50]` dans `main` (remplir `moyennes[i-1] = moy;` dans la boucle), puis passer ce tableau à la fonction.
2. `if (moy >= 16.0f) return "Tres bien"; else if (moy >= 14.0f) return "Bien"; ... else return "Echec";`. Attention à l'ordre décroissant des tests.
3. La fonction `viderBuffer` appelée après `scanf` gère déjà le cas lettre (`scanf` retourne 0, buffer vidé, boucle `do...while` du `main` recommence). C'est robuste.

À RETENIR Bilan du Mini-Projet — Compétences validées

Ce mini-projet a mobilisé l'ensemble des savoir-faire du Module 2 (Programmation C) :

- Algorithmique : traduction naturelle d'un algorithme structuré (variables, boucles, alternatives) en C.
- Types & Variables : `int`, `float`, `char[]`, `float[]`, `const char*`.
- Opérateurs : arithmétiques (+, /), comparaison (<, >, ==), logiques (&&, || dans les conditions de boucle).
- Bibliothèques : `stdio.h` (`printf`, `scanf`, `fgets`), `string.h` (`strcspn`, `strlen`, `strcpy`).
- Structures de contrôle : `do...while` (saisie contrôlée), `for` (boucle élèves), `switch` (mention), `if/else` (validation nb élèves).
- Modularité : 4 fonctions auxiliaires + `main`, passage par valeur, retour de valeur, passage de tableau (pointeur).

- Débogage : exécution pas à pas, points d'arrêt, inspection variables.
- Rigueur : gestion des buffers, bornes, cas limites, affichage formaté professionnel.

C'est la synthèse opérationnelle attendue en fin de Première TI.

8 ACTIVITÉ D'INTÉGRATION

9 LEÇON 08 : PROGRAMMATION PROCÉDURALE EN LANGAGE C — MINI-PROJET D'INTÉGRATION : GESTION D'UN CYBERCAFÉ

9.1 Objectifs de l'activité

Cette activité d'intégration vise à mobiliser l'ensemble des savoir-faire acquis dans la leçon sur la programmation procédurale en langage C. À l'issue de ce travail, vous devez être capable de :

- Concevoir l'algorithme complet de résolution d'un problème de gestion commerciale simple.
- Traduire cet algorithme en un programme C structuré, compilable et exécutable sous Code::Blocks ou Dev-C++.
- Manipuler les **types de base** (int, float, char), les **variables**, les **constantes** et les **opérateurs** arithmétiques et logiques.
- Implémenter des **entrées/sorties formatées** avec scanf et printf (bibliothèque stdio.h).
- Structurer le flux d'exécution à l'aide d'**instructions alternatives** (if, else if, switch) et de **boucles** (while pour la validation, for pour la simulation de plusieurs clients).
- Utiliser la bibliothèque math.h (ex. floorf pour l'arrondi monétaire) et le **débogueur** pas à pas pour corriger les erreurs logiques.
- Présenter une solution technique claire (code commenté + captures d'écran de l'exécution).

9.2 Situation

EXEMPLE Contexte : Le Cyber « Connexion Yaoundé »

Monsieur Essomba gère le cybercafé « Connexion Yaoundé », situé au quartier Mvog-Ada. Son activité principale est l'impression de documents pour les étudiants de l'Université de Yaoundé I et les fonctionnaires des ministères environnants. Actuellement, le calcul du coût se fait à la main sur un cahier, ce qui engendre des erreurs fréquentes (oubli de la TVA, mauvaise application de la remise fidélité, confusion entre les types de papier). Il vous demande de développer un **programme C** en mode console qui automatisera la facturation. Le programme doit permettre au gérant ou à son apprenti d'enregistrer une commande d'impression, de calculer instantanément le montant à payer et d'afficher un ticket récapitulatif clair.

Grille tarifaire officielle du cybercafé (coût unitaire par page) :

Type de papier	Description	Prix unitaire (FCFA)
1	A4 Standard (80 g/m ² , blanc)	50
2	A4 Glossy (photo brillant 180 g/m ²)	150
3	A3 Standard (80 g/m ² , blanc)	100

Règles de gestion à implémenter :

1. Le client choisit le type de papier (1, 2 ou 3), saisit le nombre de pages du document original (ex : un mémoire de 45 pages) et le nombre de copies souhaitées (ex : 3 exemplaires).
2. Le nombre total de pages imprimées = pages × copies.
3. Le coût total HT (Hors Taxes) = total pages × prix unitaire.
4. Remise commerciale : Si le total des pages imprimées dépasse 100 pages, une remise de 5% est appliquée sur le coût total HT.
5. La TVA est fixée à 19,25% (taux en vigueur au Cameroun). Elle s'applique sur le montant HT après remise.
6. Le Total TTC = HT (après remise) + TVA.
7. L'affichage final doit présenter : le libellé du papier, le détail du calcul (HT, Remise, TVA, TTC) avec des montants arrondis à l'entier inférieur (pas de centimes de FCFA).
8. Le programme doit pouvoir traiter plusieurs clients de suite (boucle principale) jusqu'à ce que l'utilisateur décide de quitter.
9. La saisie doit être sécurisée : rejeter les nombres négatifs ou nuls, et redemander le type de papier si le code n'est pas 1, 2 ou 3.

9.3 Consignes

Travaillez par binômes. Chaque binôme rend un fichier `cyber_gestion.c` compilé sans avertissement (`-Wall`) et un compte-rendu PDF (captures d'écran + réponses aux questions).

1. **Analyse et algorithme** : Sur papier, écrivez l'algorithme complet en pseudo-code (variables, constantes, structures `Si/Selon`, `TantQue/Pour`). Identifiez les variables de type `int` et `float`.
2. **Implantation C (Structure)** : Créez le projet dans l'IDE. Écrivez la structure minimale (`#include`, `main`, `return 0`). Déclarez les **constantes** pour les prix et la TVA (`const float TAUX_TVA = 0.1925f;`).
3. **Saisie sécurisée** : Écrivez une fonction (ou un bloc dans `main`) qui demande le type de papier et reboucle (`do...while`) tant que l'entrée n'est pas 1, 2 ou 3. Faites de même pour les pages et copies (strictement positives).
4. **Calculs** : Implémentez la logique de calcul (total pages, HT, remise conditionnelle, TVA, TTC). Utilisez `math.h` et `floorf` pour tronquer les décimales en FCFA.

5. **Affichage formaté** : Produisez un ticket aligné (utilisez `%10.0f` ou `%d` pour les entiers FCFA). Affichez le libellé du papier via un `switch`.
6. **Boucle principale** : Enveloppez le traitement d'un client dans une boucle `while` (ou `do...while`) contrôlée par une variable `continuer` (1 = oui, 0 = non).
7. **Test et débogage** : Testez avec les jeux de données ci-dessous. Utilisez le **débogueur** (points d'arrêt, pas à pas, surveillance des variables `totalPages`, `montantHT`, `remise`) pour corriger toute anomalie.
8. **Jeux de test obligatoires** :
 - Test A : Type 1 (A4 Std), 10 pages, 5 copies → 50 pages, pas de remise.
 - Test B : Type 2 (Glossy), 20 pages, 6 copies → 120 pages, remise 5%.
 - Test C : Type 3 (A3), 1 page, 1 copie → 1 page, pas de remise.
 - Test D : Saisie erronée (type 4, puis 1 ; pages -5, puis 10 ; copies 0, puis 2).
9. **Présentation orale (5 min)** : Expliquez le choix du `switch` vs `if/else`, la gestion de la remise, et montrez une exécution pas à pas dans le débogueur sur le Test B.

9.4 Ressources à mobiliser

PROPRIÉTÉ Syntaxe clé du C pour cette activité

Structure minimale :

```
#include <stdio.h>
#include <math.h> // pour floorf

int main() {
    // déclarations
    // instructions
    return 0;
}
```

Constantes et variables :

```
const float TAUX_TVA = 0.1925f;
const int PRIX_A4_STD = 50;
const int PRIX_A4_GLOSSY = 150;
const int PRIX_A3_STD = 100;
const float TAUX_REMISE = 0.05f;
const int SEUIL_REMISE = 100;

int typePapier, nbPages, nbCopies, totalPages;
float prixUnitaire, montantHT, remise, montantApresRemise, tva, totalTTC;
int continuer = 1; // 1 = Oui, 0 = Non
```

Saisie sécurisée (`do...while`) :

```
do {
    printf("Type papier (1=A4 Std, 2=A4 Glossy, 3=A3) : ");
    scanf("%d", &typePapier);
}
```

```
while(getchar() != '\n'); // vider buffer
} while (typePapier < 1 || typePapier > 3);
```

Choix prix unitaire (switch) :

```
switch (typePapier) {
    case 1 : prixUnitaire = PRIX_A4_STD; break;
    case 2 : prixUnitaire = PRIX_A4_GLOSSY; break;
    case 3 : prixUnitaire = PRIX_A3_STD; break;
}
```

Calcul remise conditionnelle :

```
if (totalPages > SEUIL_REMISE) {
    remise = floorf(montantHT * TAUX_REMISE);
} else {
    remise = 0.0f;
}
montantApresRemise = montantHT - remise;
tva = floorf(montantApresRemise * TAUX_TVA);
totalTTC = floorf(montantApresRemise + tva);
```

Affichage aligné :

```
printf("\n+-----+\n");
printf("|    CYBER CONNEXION YDE    |\n");
printf("+-----+\n");
printf("| Papier : %-20s|\n", libellePapier);
printf("| Total pages : %8d    |\n", totalPages);
printf("| Montant HT : %8.0f FCFA |\n", montantHT);
printf("| Remise (5%%) : %8.0f FCFA |\n", remise);
printf("| TVA (19.25%%) : %8.0f FCFA |\n", tva);
printf("| TOTAL TTC : %8.0f FCFA |\n", totalTTC);
printf("+-----+\n");
```

9.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 8

Corrigé complet commenté : cyber_gestion.c

Énoncé : Écrire le programme C complet répondant à la situation du cybercafé « Connexion Yaoundé ».

SOLUTION

Solution détaillée :

1. Analyse algorithmique (rappel) : On identifie trois phases : Initialisation des constantes, Boucle principale clients, Traitement un client (Saisie → Calcul → Affichage). La saisie utilise une validation `do...while`. Le choix du papier utilise `switch`. La remise utilise `if/else`. L'arrondi monétaire utilise `floorf` de `math.h`.

2. Code source complet et commenté :

```

/*
 * cyber_gestion.c
 * Gestion des impressions – Cyber "Connexion Yaounde"
 * Compilation : gcc -Wall -std=c99 cyber_gestion.c -o cyber_gestion -lm
 * Execution : ./cyber_gestion (Linux/Mac) ou cyber_gestion.exe (Windows)
 */
#include <stdio.h>
#include <math.h> // Necessary for floorf (arrondi à l'entier inferieur)

int main() {
    // --- 1. DECLARATION DES CONSTANTES (Savoir-faire : constantes) ---
    // Utilisation de 'const' et suffixe 'f' pour les float (bonnes pratiques)
    const float TAUX_TVA = 0.1925f; // 19.25%
    const float TAUX_REMISE = 0.05f; // 5%
    const int SEUIL_REMISE = 100; // Pages
    const int PRIX_A4_STD = 50;
    const int PRIX_A4_GLOSSY = 150;
    const int PRIX_A3_STD = 100;

    // --- 2. DECLARATION DES VARIABLES ---
    int typePapier = 0; // 1, 2 ou 3
    int nbPages = 0; // Pages du document original
    int nbCopies = 0; // Nombre d'exemplaires
    int totalPages = 0; // Calculé
    float prixUnitaire = 0.0f; // Selon type
    float montantHT = 0.0f; // Avant remise
    float remise = 0.0f; // Calculée (arrondie)
    float netHT = 0.0f; // HT - Remise
    float tva = 0.0f; // Calculée sur netHT (arrondie)
    float totalTTC = 0.0f; // Final (arrondi)
    char libellePapier[20]; // Chaine pour affichage (ex : "A4 Standard")
    int continuer = 1; // 1 = Oui, 0 = Non (Booléen simulé par int)

    // --- 3. BOUCLE PRINCIPALE : TRAITEMENT DE PLUSIEURS CLIENTS ---
    // do...while garantit au moins un passage (au moins un client)
    do {
        printf("\n===== \n");
        printf(" CYBER CONNEXION YAOUNDE - NOUVEAU CLIENT \n");
        printf("===== \n");

        // --- 3.1 SAISIE SECURISEE DU TYPE DE PAPIER ---
        // Structure alternative multiple : do...while + switch
        do {
            printf("\nChoisissez le type de papier : \n");
            printf(" 1 - A4 Standard (50 FCFA/page) \n");
            printf(" 2 - A4 Glossy (150 FCFA/page) \n");
            printf(" 3 - A3 Standard (100 FCFA/page) \n");
            printf("Votre choix (1-3) : ");
            // Verification du retour de scanf (bonne pratique robustesse)
            if (scanf("%d", &typePapier) != 1) {
                typePapier = 0; // Force erreur si saisie non numerique
            }
            // Vider le buffer clavier (gestion du '\n' restant)
            while (getchar() != '\n');
        } while (typePapier == 0);
    } while (continuer == 1);
}

```

```

        if (typePapier < 1 || typePapier > 3){
            printf("\n>>> ERREUR : Choix invalide. Veuillez saisir 1, 2 ou 3. <<<\n");
        }
    } while (typePapier < 1 || typePapier > 3);

    // Affectation prix et libellé selon choix (Switch : plus lisible qu'if/else chaîné)
    switch (typePapier) {
        case 1: prixUnitaire = PRIX_A4_STD; printf(libellePapier, "A4 Standard"); break;
        case 2: prixUnitaire = PRIX_A4_GLOSSY; printf(libellePapier, "A4 Glossy"); break;
        case 3: prixUnitaire = PRIX_A3_STD; printf(libellePapier, "A3 Standard"); break;
        default: prixUnitaire = 0; printf(libellePapier, "Inconnu"); break; // Securite
    }

    // --- 3.2 SAISIE SECURISEE NOMBRE DE PAGES (> 0) ---
    do {
        printf("Nombre de pages du document (>= 1) : ");
        if (scanf("%d", &nbPages) != 1) nbPages = 0;
        while (getchar() != '\n');
        if (nbPages <= 0) printf(">>> ERREUR : Le nombre de pages doit etre positif. <<<\n");
    } while (nbPages <= 0);

    // --- 3.3 SAISIE SECURISEE NOMBRE DE COPIES (> 0) ---
    do {
        printf("Nombre de copies souhaitees (>= 1) : ");
        if (scanf("%d", &nbCopies) != 1) nbCopies = 0;
        while (getchar() != '\n');
        if (nbCopies <= 0) printf(">>> ERREUR : Le nombre de copies doit etre positif. <<<\n");
    } while (nbCopies <= 0);

    // --- 3.4 CALCULS (Operateurs arithmetiques, math.h) ---
    totalPages = nbPages * nbCopies; // Total pages imprimees
    montantHT = totalPages * prixUnitaire; // Cout brut

    // Structure alternative simple : Remise conditionnelle avec arrondi inferieur
    if (totalPages > SEUIL_REMISE) {
        remise = floorf(montantHT * TAUX_REMISE);
    } else {
        remise = 0.0f;
    }

    netHT = montantHT - remise; // Base taxable
    // floorf : arrondi à l'entier inferieur (troncature) pour les FCFA
    tva = floorf(netHT * TAUX_TVA); // TVA arrondie
    totalTTC = floorf(netHT + tva); // TTC arrondi

    // --- 3.5 AFFICHAGE FORMATE DU TICKET (printf) ---
    printf("\n+-----+\n");
    printf("| FACTURE IMPRESSION - TICKET |\n");
    printf("+-----+\n");
    printf("| Papier : %-28s|\n", libellePapier);
    printf("| Pages doc. : %8d |\n", nbPages);
    printf("| Copies : %8d |\n", nbCopies);
    printf("| Total pages : %8d |\n", totalPages);
    printf("| Prix unitaire : %8.0f FCFA |\n", prixUnitaire);
    printf("|-----|\n");
    printf("| Montant HT : %8.0f FCFA |\n", montantHT);
    printf("| Remise (5%%) : -%7.0f FCFA |\n", remise);

```

```

printf(" Net HT      : %8.0f FCFA      |\n", netHT);
printf(" TVA (19.25%%) : %8.0f FCFA      |\n", tva);
printf("-----|\n");
printf(" TOTAL TTC   : %8.0f FCFA      |\n", totalTTC);
printf("-----+|\n");
printf("      Merci de votre visite !\n");

// --- 3.6 DEMANDE CONTINUATION ---
printf("\nTraiter un autre client ? (1=Oui / 0=Non) : ");
if (scanf("%d", &continuer) != 1) continuer = 0;
while (getchar() != '\n'); // Nettoyage buffer

} while (continuer == 1); // Fin boucle principale

printf("\nFermeture de la caisse. Au revoir !\n");
return 0; // Succes
}

```

Listing 1 – Fichier cyber_gestion.c

3. Explication des points techniques clés (pour l'oral) :

- `#include <math.h>` et `-lm` : La fonction `floorf` (version float de `floor`) nécessite l'édition de liens avec la bibliothèque mathématique (`-lm` avec GCC). Dans Code ::Blocks/Dev-C++, c'est souvent automatique, mais en ligne de commande c'est obligatoire. Placez `-lm` après le nom du fichier source.
- `while(getchar() != '\n');` : Après un `scanf("%d", ...)`, le caractère `\n` (Entrée) reste dans le tampon d'entrée (`stdin`). Si on ne le vide pas, le `scanf` suivant (ou un `getchar/fgets`) le lira immédiatement, sautant la saisie. C'est un piège classique en C.
- `sprintf(libellePapier, ...)` : Permet de construire la chaîne de caractères du libellé pour l'affichage final, évitant de refaire un `switch` dans le `printf` du ticket. Nécessite `stdio.h` (déjà inclus).
- Types `float` vs `int` pour l'argent : On utilise `float` pour les calculs intermédiaires (TVA, remise) mais `floorf` ramène à un entier (FCFA). En gestion réelle, on préférera stocker les montants en centimes (entier `long long`) pour éviter les erreurs de virgule flottante (ex : `0.1 + 0.2 != 0.3`). Ici, `float + floorf` est acceptable au niveau Première.
- Choix `switch` vs `if/else` : Le `switch` est plus lisible et souvent plus performant (table de saut) pour une comparaison d'égalité sur une variable entière (`typePapier`) avec plusieurs cas discrets. L'`if/else` est réservé aux conditions de plage (ex : `totalPages > 100`).

4. Résultats des tests (Sorties console attendues) :

Test A : Type 1, 10 pages, 5 copies.

```
Total pages = 50. Montant HT = 2500. Remise = 0. TVA = 481. TTC = 2981.
```

Test B : Type 2, 20 pages, 6 copies.

Total pages = 120 (> 100). Montant HT = 18000. Remise = `floor`(900) = 900. Net HT = 17100.

TVA = `floor`(17100 * 0.1925) = `floor`(3291.75) = 3291.

TTC = `floor`(17100 + 3291.75) = `floor`(20391.75) = 20391.

Test C : Type 3, 1 page, 1 copie.

Total pages = 1. Montant HT = 100. Remise = 0. TVA = 19. TTC = 119.

Test D (Robustesse) : Saisie type 4 → message erreur → resaisie 1. Saisie pages -5 → erreur → resaisie 10. Saisie copies 0 → erreur → resaisie 2. Le programme ne plante pas et redemande correctement.

9.6 Bilan de l'activité

Cette activité de synthèse a permis de valider les compétences suivantes, piliers du module Programmation procédurale en C en Première TI :

À RETENIR

Compétences validées

1. Modélisation algorithmique : Passage d'un énoncé métier (grille tarifaire, règles de remise, TVA) à un algorithme structuré (variables, constantes, structures de contrôle).
2. Traduction C rigoureuse : Respect de la syntaxe (`#include`, `main`, `,`, accolades), type `fort` (`int`, `float`, `const`), utilisation correcte des bibliothèques standards (`stdio.h`, `math.h`).
3. Maîtrise des structures de contrôle :
 - `do...while` pour la validation de saisie (boucle au moins une fois, condition de sortie en fin).
 - `switch` pour le choix discret (type papier $\in \{1,2,3\}$).
 - `if/else` pour la condition de seuil (remise si > 100).
 - Boucle principale `do...while` pour le traitement itératif multi-clients.
4. Gestion des entrées/sorties : `scanf` avec vérification du code de retour (`!= 1`), vidage du buffer (`getchar()`), `printf` avec spécificateurs de format (`%d`, `%.0f`, `%s`) pour un alignement professionnel.
5. Calculs numériques et arrondi : Utilisation de `floorf` (`math.h`) pour respecter l'unité monétaire (FCFA entier), distinction entre calcul sur `float` et affichage entier.
6. Débogage pas à pas : Pose de points d'arrêt (breakpoints) sur le calcul de `remise` et `tva`, observation des variables locales (Watch window) pour vérifier l'application correcte du seuil de 100 pages et l'arrondi de la TVA (Test B : 3291.75 → 3291).
7. Bonnes pratiques professionnelles : Constantes nommées (`TAUX_TVA`) au lieu de «

nombre magiques », commentaires explicatifs, indentation cohérente (style K&R ou Allman), noms de variables explicites (`montantHT` vs `mht`).

ATTENTION Pièges évités grâce au débogueur

- Oubli du `-lm` : Erreur de liaison `undefined reference to `floorf'` si compilation manuelle sans `-lm` en fin de commande.
- Buffer `stdin` non vidé : Boucle infinie ou saut de saisie si `while(getchar() != '\n');` omis après `scanf`.
- Division entière involontaire : Écrire `5/100` (résultat 0 en entier) au lieu de `0.05f` ou `5.0/100.0`. Ici, l'usage de constantes `float` (`0.05f`) l'évite.
- Arrondi bancaire vs troncature : `printf("%.0f", x)` arrondit à l'entier le plus proche (`0.5 → 1`), alors que la réglementation comptable exige souvent la troncature (`floorf`). Le corrigé utilise `floorf` avant stockage et affichage.
- Oubli d'arrondir la remise : La remise doit elle aussi être tronquée à l'entier inférieur (`floorf`) pour garantir la cohérence du net HT affiché.

LESAVAIS-TU ? Le C au Cameroun : De la maintenance aux fintechs

Le langage C, bien que « bas niveau », reste le cœur des systèmes embarqués et critiques. Au Cameroun, les automates bancaires (GAB) de BICEC, UBA ou Afriland, les compteurs électriques prépayés (ENEO), les terminaux de paiement mobile (Mobile Money, Orange Money) et les box internet CAMTEL tournent sur du firmware C/C++. Maîtriser la gestion mémoire, les entrées/sorties bas niveau et la rigueur du typage statique en Première TI, c'est poser la première pierre pour devenir ingénieur systèmes embarqués ou développeur noyau — des profils très recherchés par l'écosystème Silicon Mountain (Buea) et les industriels de Douala. Ce petit programme de cybercafé simule exactement la logique d'un terminal de point de vente (TPE) : saisie sécurisée, calculs financiers exacts, impression ticket, boucle multi-transactions. Vous venez d'écrire le noyau d'un TPE !

10 MÉTHODES ET EXERCICES RÉSOLUS

11 PROGRAMMATION PROCÉDURALE EN LANGAGE C

LESAVAIS-TU ? Naissance du C et de Unix

Le langage **C** a été créé en 1972 par Dennis Ritchie aux laboratoires Bell (USA) pour réécrire le système d'exploitation Unix. Avant le C, les OS étaient écrits en assembleur, spécifiques à chaque machine. Le C a apporté la **portabilité** : le même code source, recompilé, tourne sur n'importe quel processeur. Au Cameroun, la majorité des logiciels embarqués (compteurs prépayés CAMTEL, bornes de paiement Mobile Money, systèmes de gestion de stock des pharmacies) sont écrits en C ou C++ pour leur rapidité et leur fiabilité. Le C est le « latin » de l'informatique moderne : le maîtriser, c'est comprendre comment la machine exécute vraiment nos algorithmes.

11.1 Langages interprétés vs compilés & structure minimale d'un programme C

MÉTHODE Comment établir la différence entre langage interprété et langage compilé

Pour répondre à une question de comparaison (question fréquente au Probatoire), procède en quatre étapes :

1. **Définis** chaque notion : un **langage compilé** est traduit en une seule fois en langage machine par un **compilateur**, qui produit un fichier exécutable autonome ; un **langage interprété** est traduit ligne par ligne, à chaque exécution, par un **interpréteur**.
2. **Compare** selon des critères précis : moment de la traduction, vitesse d'exécution, détection des erreurs, portabilité du fichier produit.
3. **Cite des exemples** : le C est compilé (compilateur `gcc`) ; le JavaScript est interprété (par le navigateur) ; Python est interprété (CPython).
4. **Conclus** par une phrase de synthèse : le compilé est plus rapide à l'exécution, l'interprété est plus souple pour tester et déboguer.

Réflexe Bac : présente ta réponse sous forme de tableau comparatif, c'est ce que le correcteur attend.

PROPRIÉTÉ Tableau comparatif : Compilé vs Interprété

Critère	Langage compilé (ex. C)	Langage interprété (ex. JavaScript, Python)
Traduction	Par un compilateur (gcc), avant exécution	Par un interpréteur, pendant l'exécution
Produit	Fichier exécutable autonome (.exe, .out)	Aucun exécutable; le code source est requis
Vitesse exécution	Très rapide (code machine natif)	Plus lente (analyse ligne par ligne à la volée)
Détection erreurs	À la compilation (erreurs de syntaxe, types)	À l'exécution (erreurs découvertes au fil de l'eau)
Portabilité	Recompiler le source pour chaque architecture	Portable si l'interpréteur existe sur la machine cible
Exemples	C, C++, Rust, Go	JavaScript, Python, PHP, Bash

EXERCICE RÉSOLU 9 Langages interprétés et compilés

Énoncé. Dans un cybercafé de Douala, le gérant fait développer deux outils : un programme de facturation écrit en C, et une page web animée en JavaScript. (1) Définis les termes langage compilé et langage interprété. (2) Classe chacun des deux langages utilisés et justifie. (3) Donne deux avantages du langage compilé pour le programme de facturation.

SOLUTION

Solution détaillée.

(1) Définitions. Un **langage compilé** est un langage dont le code source est traduit intégralement en langage machine par un programme appelé **compilateur**, avant toute exécution. Le résultat est un **fichier exécutable** (par exemple `facturation.exe`) qui peut être lancé sans le code source ni le compilateur. Un **langage interprété** est un langage dont le code source est traduit et exécuté ligne par ligne, à chaque lancement, par un programme appelé **interpréteur**. Aucun exécutable autonome n'est produit : l'interpréteur doit être présent à chaque exécution.

(2) Classification.

Critère	C (facturation)	JavaScript (page web)
Nature	Langage compilé	Langage interprété
Traducteur	Compilateur (gcc)	Interpréteur du navigateur (V8, SpiderMonkey...)
Moment de traduction	Avant l'exécution, en une fois	Pendant l'exécution, ligne par ligne
Erreurs	Détectées à la compilation	Détectées pendant l'exécution
Fichier livré	Exécutable binaire	Fichier source .js

Le C est compilé car `gcc` transforme le fichier `.c` en exécutable machine. Le JavaScript est interprété car le navigateur lit et exécute le script directement, sans produire d'exécutable.

(3) Avantages du compilé pour la facturation. (a) **Rapidité d'exécution** : le programme est déjà en langage machine, donc plus rapide — utile pour enchaîner les factures en heure de pointe. (b) **Autonomie et confidentialité** : l'exécutable fonctionne sans distribuer le code source, ce qui protège le travail du développeur et les algorithmes métier.

Conclusion. Le C, langage compilé, convient aux applications rapides et autonomes ; le JavaScript, langage interprété, convient aux pages web exécutées par le navigateur.

MÉTHODE

Comment écrire et exécuter un premier programme C

1. **Inclus** la bibliothèque d'entrées/sorties : `#include <stdio.h>` en toute première ligne.
2. **Écris** la fonction principale `int main(void)` : tout programme C commence son exécution dans `main`.
3. **Place** les instructions entre les accolades `{ }` du corps de `main`, chacune terminée par un point-virgule.
4. **Termine** par `return 0;` qui signale au système que le programme s'est bien déroulé.
5. **Exécute** dans l'IDE : sous Code ::Blocks ou Dev-C++, crée un projet console en C, colle le code, puis F9 (Compiler et exécuter) sous Code ::Blocks ou F11 sous Dev-C++.

Structure minimale à connaître par cœur :

```
#include <stdio.h>

int main(void)
{
    printf("Bonjour la Première TI !\n");
    return 0;
}
```

EXERCICE RÉSOLU 10 Premier programme : afficher un message

Énoncé. Le proviseur du lycée de Bafoussam veut qu'un programme affiche à l'écran le message LYCEE DE BAFOUSSAM - ANNEE 2024. (1) Écris le programme C complet réalisant cet affichage. (2) Explique le rôle de chacune des lignes `#include <stdio.h>`, `int main(void)` et `return 0;`. (3) Décris les étapes pour exécuter ce programme sous Code ::Blocks.

SOLUTION

Solution détaillée.

(1) Programme complet.

```
#include <stdio.h>

int main(void)
{
    printf("LYCEE DE BAFOUSSAM - ANNEE 2024\n");
    return 0;
}
```

Vérification ligne par ligne : la directive `#include` est bien la première ligne ; les accolades ouvrante et fermante encadrent le corps de `main` ; l'instruction `printf` se termine par un point-virgule ; la chaîne est entre guillemets doubles ; `\n` provoque le retour à la ligne après l'affichage.

(2) Rôle des lignes.

- `#include <stdio.h>` : directive du préprocesseur qui insère la bibliothèque standard d'entrées/sorties, indispensable pour utiliser `printf`.
- `int main(void)` : déclare la **fonction principale**, point d'entrée du programme ; `int` indique qu'elle renvoie un entier au système, `void` qu'elle ne reçoit aucun paramètre.
- `return 0;` : renvoie la valeur 0 au système d'exploitation, convention qui signifie « exécution réussie ».

(3) Exécution sous Code ::Blocks. Étape 1 : menu File → New → Project → Console application, choisir le langage C. Étape 2 : ouvrir `main.c` et saisir le code. Étape 3 : appuyer sur F9 (Build and run) : le compilateur traduit le source en exécutable puis la console s'ouvre et affiche le message.

Conclusion. Tout programme C repose sur le squelette `#include <stdio.h> + int main(void) + return 0;` ; l'IDE automatise la compilation et l'exécution en une seule touche.

11.2 Types de base, variables, constantes et opérateurs

MÉTHODE Comment déclarer une variable ou une constante en C

1. **Identifie** la nature de la donnée : entier (`int`), réel (`float` ou `double`), caractère (`char`).
2. **Choisis** le type : âge, nombre d'élèves → `int`; moyenne, prix → `float`; note très précise → `double`; une lettre → `char`.
3. **Déclare** en début de bloc : `type nom;` ou avec initialisation `type nom = valeur;` (fortement conseillé).
4. **Constante** : ajoute le mot-clé `const` devant le type, et initialise obligatoirement : `const float TVA = 19.25;` — attention, en C on écrit 19.25 avec un point décimal!
5. **Nomme** correctement : lettres, chiffres et `_`, en commençant par une lettre ; pas d'accent, pas d'espace, pas de mot réservé (`int`, `return`...). Le C distingue majuscules et minuscules : `Note` et `note` sont deux variables différentes.

Types de base à retenir : `int` (entier), `float` (réel simple précision), `double` (réel double précision), `char` (caractère, entre apostrophes : `'A'`).

PROPRIÉTÉ Types de base en C (architecture 32/64 bits typique)

Type	Taille	Domaine approximatif	Usage courant
<code>char</code>	1 octet	-128 à 127 (signé) ou 0 à 255	Caractère ASCII, petit entier
<code>int</code>	4 octets	-2 147 483 648 à 2 147 483 647	Entiers généraux, compteurs
<code>float</code>	4 octets	$\pm 3,4 \times 10^{-38}$ à $\pm 3,4 \times 10^{38}$ (~ 7 chiffres significatifs)	Réels simples (prix, moyennes)
<code>double</code>	8 octets	$\pm 1,7 \times 10^{-308}$ à $\pm 1,7 \times 10^{308}$ (~ 15 chiffres significatifs)	Réels précis (calcul scientifique)

Rappel : en C, la virgule décimale s'écrit point (3.14), jamais virgule (3,14 est une erreur de syntaxe ou deux arguments).

EXERCICE RÉSOLU 11 Déclarations pour la gestion d'une boutique

Énoncé. Une boutique de Yaoundé veut un programme gérant : le nombre de cartons de savon en stock (entier), le prix unitaire d'un carton en FCFA (réel), la lettre de la catégorie du produit (caractère), et le taux de remise fixe de 5 % qui ne doit jamais changer pendant l'exécution. (1) Écris les quatre déclarations C correctes, avec initialisation. (2) Justifie le choix de chaque type. (3) Indique ce qui se passe si l'on tente de modifier le taux de remise plus loin dans le programme.

SOLUTION

Solution détaillée.

(1) Déclarations.

```
int stock = 120;           /* nombre de cartons */
float prixUnitaire = 3500.0; /* prix en FCFA */
char categorie = 'A';      /* lettre de categorie */
const float REMISE = 0.05; /* taux fixe de 5 % */
```

Remarque la syntaxe : le réel s'écrit 3500.0 avec un point décimal (jamais de virgule en C), le caractère 'A' est entre apostrophes simples (les guillemets doubles sont réservés aux chaînes), et la constante est écrite en majuscules par convention.

(2) Justification des types.

- stock : un nombre de cartons est un entier naturel, d'où `int`.
- prixUnitaire : un prix peut avoir une partie décimale (ex. 3500,50 FCFA), d'où `float`.
- categorie : une seule lettre, d'où `char`.
- REMISE : un taux est un réel (0,05), et le mot-clé `const` interdit toute modification : c'est une **constante**.

(3) Modification interdite. Si l'on écrit plus loin `REMISE = 0.10;`, le compilateur refuse et signale une erreur du type `assignment of read-only variable`. C'est précisément l'intérêt de `const` : protéger une valeur contre toute modification accidentelle.

Conclusion. Le choix du type dépend de la nature mathématique de la donnée ; toute valeur fixe du problème doit être déclarée `const` pour sécuriser le programme.

MÉTHODE Comment maîtriser opérateurs, printf et scanf

1. **Opérateurs arithmétiques** : +, -, *, /, et % (reste de la division entière, réservé aux entiers). Piège : la division de deux `int` est entière : `7 / 2` vaut 3, pas 3,5.
2. **Opérateurs de comparaison** : == (égal), !=, <, <=, >, >=. Le résultat vaut 1 (vrai) ou 0 (faux).
3. **Opérateurs logiques** : && (ET), || (OU), ! (NON). Court-circuit : si le premier opérande

détermine le résultat, le second n'est pas évalué.

4. **printf** : `printf("format", variables);` avec les spécificateurs `%d` (int), `%f` (float), `%lf` (double), `%c` (char). Pour deux décimales : `%.2f`.
5. **scanf** : `scanf("format", &variable);` — l'**esperluette** `&` (adresse de la variable) est obligatoire devant chaque variable (sauf pour les tableaux/chaînes).

Réflexe : vérifie que chaque % du format correspond, dans l'ordre, à une variable du bon type.

PROPRIÉTÉ Spécificateurs de format courants

Spécificateur	Type C	Exemple
<code>%d</code> ou <code>%i</code>	int	<code>printf("%d", age);</code>
<code>%f</code>	float	<code>printf("%.2f", prix);</code>
<code>%lf</code>	double	<code>scanf("%lf", &x);</code>
<code>%c</code>	char	<code>printf("%c", lettre);</code>
<code>%s</code>	char * (chaîne)	<code>printf("%s", nom);</code>

EXERCICE RÉSOLU 12 Saisie et calcul de la moyenne de deux notes

Énoncé. On veut un programme qui demande à un élève ses deux notes d'informatique (sur 20), calcule leur moyenne et l'affiche avec deux décimales. (1) Écris le programme C complet. (2) Explique pourquoi on écrit `(note1 + note2) / 2.0` et non `(note1 + note2) / 2`. (3) Donne le résultat affiché si l'utilisateur saisit 12 et 15.

SOLUTION

Solution détaillée.

(1) Programme complet.

```
#include <stdio.h>

int main(void)
{
    float note1, note2, moyenne;

    printf("Entrez la premiere note : ");
    scanf("%f", &note1);
    printf("Entrez la deuxieme note : ");
    scanf("%f", &note2);
```

```
moyenne = (note1 + note2) / 2.0;

printf("La moyenne est : %.2f\n", moyenne);
return 0;
}
```

Vérification : chaque `scanf` porte bien un `&` devant la variable ; le format `%.2f` correspond au type `float` ; `%.2f` impose deux chiffres après la virgule.

(2) Pourquoi `2.0` et non `2`. En C, si les deux opérandes de `/` sont des entiers, la division est entière (la partie décimale est tronquée). Ici `note1 + note2` est un `float`, donc même avec `2` le résultat serait correct par promotion automatique ; mais écrire `2.0` est le réflexe de sécurité qui garantit une division réelle dans tous les cas. Contre-exemple classique : avec deux variables `int`, `(a + b) / 2` donnerait un entier faux (ex. $(12+15)/2 = 13$ au lieu de $13,5$).

(3) Trace d'exécution. Saisies : `note1 = 12.0`, `note2 = 15.0`.

$$\text{moyenne} = \frac{12 + 15}{2} = \frac{27}{2} = 13,5$$

L'écran affiche : `La moyenne est : 13.50`.

Conclusion. Le trio déclaration typée + `scanf` avec `&` + division réelle garantit un calcul de moyenne exact.

11.3 Structures alternatives : if, if...else, switch

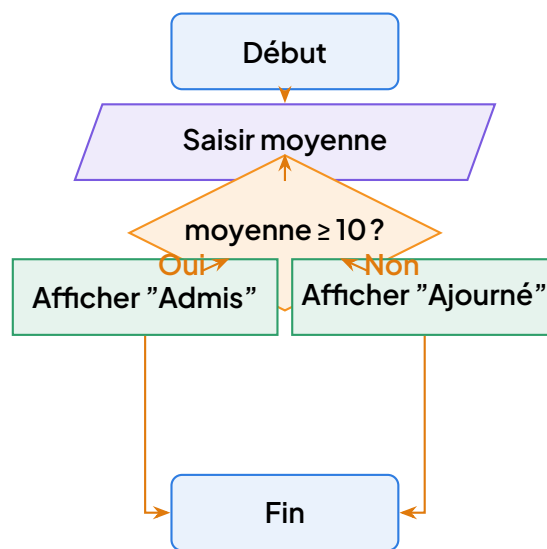
MÉTHODE Comment traduire une décision en C

1. **Analyse** le nombre de cas : un seul test → `if` ; deux cas exclusifs → `if...else` ; plusieurs valeurs précises d'une même variable entière ou caractère → `switch`.
2. **Rédige** la condition entre parenthèses, avec `==` pour tester l'égalité (jamais `=` qui est une affectation!).
3. **Encadre** chaque bloc d'instructions par des accolades `{ }` (obligatoire si plusieurs instructions, recommandé toujours).
4. **switch** : chaque `case` se termine par `break` ; pour éviter l'exécution en cascade (fall-through) ; le cas `default` traite toutes les autres valeurs.
5. **Teste** mentalement chaque branche avec des valeurs limites (ex. exactement 10 pour une moyenne).

Syntaxe du `switch` :

```
switch (choix) {
    case 1:
        printf("Consultation\n");
        break;
    case 2:
```

```
printf("Modification\n");  
break;  
default :  
    printf("Choix invalide\n");  
}
```

EXEMPLE Algorithme : Structure alternative simple (if...else)

Algorithme de la décision d'admission (if...else).

EXERCICE RÉSOLU 13 Décision d'admission selon la moyenne

Énoncé. Au lycée de Ngaoundéré, la décision du conseil de classe est : « Admis » si la moyenne est supérieure ou égale à 10, « Ajourné » sinon. (1) Écris un programme C qui saisit la moyenne et affiche la décision. (2) Trace l'exécution pour les moyennes 10 et 9,75. (3) Réécris le test avec un `switch` est-il possible ? Justifie.

SOLUTION

Solution détaillée.

(1) Programme.

```
#include <stdio.h>  
  
int main(void)
```

```
{  
    float moyenne;  
  
    printf("Entrez la moyenne : ");  
    scanf("%f", &moyenne);  
  
    if (moyenne >= 10) {  
        printf("Admis\n");  
    } else {  
        printf("Ajourne\n");  
    }  
    return 0;  
}
```

La condition `moyenne >= 10` traduit fidèlement « supérieure ou égale à 10 » ; les accolades délimitent clairement les deux branches.

(2) Traces.

- Moyenne = 10 : la condition $10 \geq 10$ est vraie (l'égalité est incluse grâce à `>=`) → affichage `Admis`. C'est la valeur limite : si l'on avait écrit `>` au lieu de `>=`, un élève à 10,00 serait injustement ajourné.
- Moyenne = 9,75 : la condition $9,75 \geq 10$ est fausse → exécution du `else` → affichage `Ajourne`.

(3) Le `switch` convient-il ? Non. Le `switch` ne teste que l'égalité d'une expression entière (`int` ou `char`) avec des constantes entières. Or ici la condition est une inégalité (≥ 10) portant sur un réel (`float`). Il faudrait une infinité de `case`. La structure adaptée est donc bien `if...else`.

Conclusion. Le choix de la structure dépend de la nature du test : inégalité sur un réel → `if...else` ; égalité d'un entier avec quelques valeurs → `switch`.

11.4 Boucles : `for`, `while`, `do...while` et traduction d'algorithmes itératifs

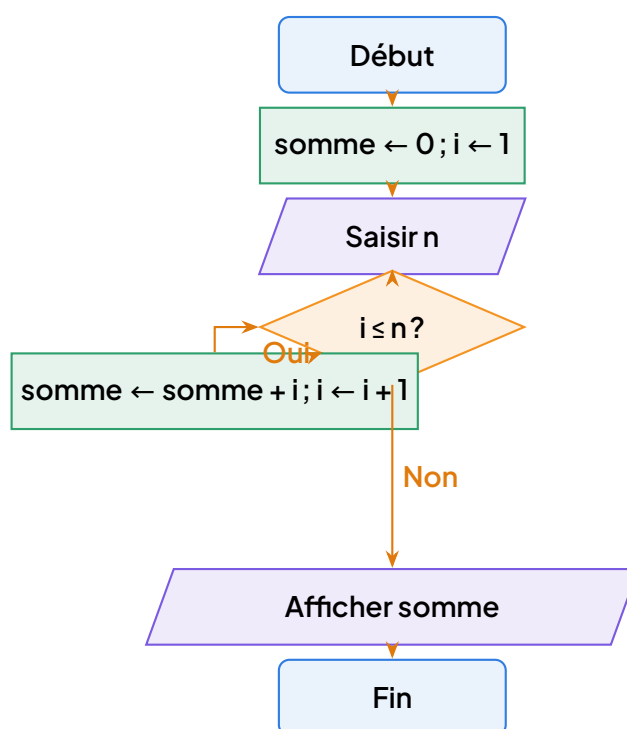
MÉTHODE Comment traduire un algorithme itératif en C

1. **Choisis** la boucle selon l'algorithme : nombre d'itérations connu → `for` ; répétition conditionnelle (tant que...) → `while` ; action à exécuter au moins une fois (saisie avec contrôle) → `do...while`.
2. **for** : `for (initialisation ; condition ; incrément)` — les trois parties sont séparées par des points-virgules.
3. **Initialise** toujours les accumulateurs avant la boucle : somme à 0, produit à 1, compteur à 0.
4. **Garantis** la terminaison : la variable de contrôle doit évoluer vers la condition d'arrêt (sinon boucle infinie).

5. **Vérifie** par une trace sur papier : note les valeurs des variables à chaque tour.

Correspondance algorithme → C : Pour i de 1 à n devient `for (i = 1; i <= n; i++)`;
Tant que (cond) devient `while (cond)`; Répéter ... Jusqu'à (cond) devient `do { ... } while (!cond)`; — attention, le `do...while` continue tant que la condition est vraie, l'inverse du « Jusqu'à »!

EXEMPLE Algorithme : Boucle pour (Somme des n premiers entiers)



Algorithme de la boucle Pour (somme de 1 à n).

EXERCICE RÉSOLU 14 Somme des n premiers entiers et saisie contrôlée

Énoncé. (1) Traduis en C l'algorithme suivant : « Lire un entier $n \geq 1$, calculer et afficher $S = 1 + 2 + \dots + n$ ». (2) Trace l'exécution pour $n = 4$. (3) Écris une saisie contrôlée qui redemande n tant que l'utilisateur entre une valeur strictement inférieure à 1.

SOLUTION

Solution détaillée.

(1) Traduction.

```
#include <stdio.h>

int main(void)
{
    int n, i, somme = 0;

    printf("Entrez n (>= 1) : ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++){
        somme = somme + i;
    }

    printf("S = %d\n", somme);
    return 0;
}
```

L'accumulateur `somme` est initialisé à 0 avant la boucle ; la borne `i <= n` inclut bien le terme `n` ; `i++` fait progresser le compteur, garantissant l'arrêt.

(2) Trace pour $n = 4$.

Tour	i	Condition $i \leq 4$	somme
initial	—	—	0
1	1	vraie	$0 + 1 = 1$
2	2	vraie	$1 + 2 = 3$
3	3	vraie	$3 + 3 = 6$
4	4	vraie	$6 + 4 = 10$
5	5	fausse → sortie	10

Le programme affiche $S = 10$. Vérification par la formule : $S = \frac{n(n+1)}{2} = \frac{4 \times 5}{2} = 10$. Le résultat est cohérent.

(3) Saisie contrôlée avec `do...while`.

```
do{
    printf("Entrez n (>= 1) : ");
    scanf("%d", &n);
} while (n < 1);
```

Le `do...while` exécute la saisie au moins une fois (on ne connaît pas n avant de l'avoir demandé), puis recommence tant que $n < 1$. C'est la traduction exacte de « Répéter... Jusqu'à ce que $n \geq 1$ ».

Conclusion. Le `for` s'impose pour un nombre de tours connu ; le `do...while` est l'outil naturel de la saisie contrôlée.

11.5 Fonctions mathématiques (math.h) et exécution pas à pas (débogage)

PROPRIÉTÉ Fonctions usuelles de <math.h> et <stdlib.h>

Fonction	Bibliothèque	Prototype	Description / Condition
sqrt	math.h	double sqrt(double x);	Racine carrée ($x \geq 0$)
pow	math.h	double pow(double x, double y);	Puissance x^y
cos, sin, tan	math.h	double cos(double x);	Trigonométrie : argument en radians
abs	stdlib.h	int abs(int x);	Valeur absolue entière
fabs	math.h	double fabs(double x);	Valeur absolue réelle
floor, ceil	math.h	double floor(double x);	Arrondi inférieur / supérieur

Important : lors de la compilation en ligne de commande avec `gcc`, il faut lier la bibliothèque mathématique : `gcc prog.c -o prog -lm`. Sous Code ::Blocks / Dev-C++, c'est généralement automatique.

MÉTHODE Comment utiliser les fonctions mathématiques et déboguer pas à pas

1. **Inclus** la bibliothèque : `#include <math.h>` en haut du fichier, en plus de `stdio.h` (et `stdlib.h` pour `abs`).
2. **Fonctions à connaître** : `sqrt(x)` (racine carrée, exige $x \geq 0$), `abs(x)` (valeur absolue d'un entier, `stdlib.h`), `fabs(x)` (réel, `math.h`), `cos(x)` (cosinus, x en radians), `pow(x, y)` (x^y).
3. **Affecte** le résultat : une fonction renvoie une valeur, il faut la récupérer : `r = sqrt(d)`; et non `sqrt(d)`; seul.
4. **Exécution pas à pas** (débogage) sous Code ::Blocks : place un point d'arrêt (clic dans la marge à gauche de la ligne), lance avec F8 (Debug/Start), avance instruction par instruction avec F7 (Next line) et observe les valeurs des variables dans la fenêtre Watches.
5. **Méthode papier** au Bac : dresse un tableau des variables ligne par ligne, comme pour une trace d'algorithme.

EXPÉRIENCE TP guidé : Premier débogage pas à pas sous Code ::Blocks

Objectif. Maîtriser le débogueur pour visualiser l'état de la mémoire à chaque instruction.
Matériel / Logiciel. PC sous Windows, Code ::Blocks installé (compilateur GCC MinGW).
Manipulation.

1. Crée un nouveau projet Console Application en C. Nomme-le DebugDemo.
2. Copie le code du calcul de distance (exemple ci-dessous) dans `main.c`.
3. Clique dans la marge grise à gauche de la ligne `dx = xB - xA;` : un point rouge apparaît (point d'arrêt / breakpoint).
4. Appuie sur F8 (Debug → Start). Le programme s'arrête sur la ligne marquée (flèche jaune).
5. Ouvre la fenêtre Debug → Debugging windows → Watches. Ajoute `xA`, `yA`, `xB`, `yB`, `dx`, `dy`, `d`.
6. Appuie successivement sur F7 (Next line) : observe les valeurs changer dans Watches après chaque instruction.
7. Continue jusqu'à la fin (`return 0;`). Ferme le débogueur (Ctrl+F8).

Observation. Les variables non initialisées contiennent des valeurs « poubelle » avant leur première affectation. Le débogueur le montre clairement. **Interprétation.** Le pas à pas est l'outil indispensable pour comprendre une boucle, repérer une division entière involontaire ou un `scanf` qui ne lit pas ce qu'on croit.

EXERCICE RÉSOLU 15 Distance entre deux points et exécution pas à pas

Énoncé. Un programme doit calculer la distance entre deux points $A(x_A, y_A)$ et $B(x_B, y_B)$ repérés sur une carte : $d = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$. (1) Écris le programme C complet utilisant `math.h`. (2) Exécute-le pas à pas (trace) pour $A(1, 2)$ et $B(4, 6)$. (3) Que se passe-t-il si l'on oublie `#include <math.h>` ?

SOLUTION

Solution détaillée.

(1) Programme complet.

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    float xA, yA, xB, yB, dx, dy, d;

    printf("Coordonnees de A (xA yA) : ");
    scanf("%f %f", &xA, &yA);
```

```

printf("Coordonnees de B (xB yB) : ");
scanf("%f %f", &xB, &yB);

dx = xB - xA;
dy = yB - yA;
d = sqrt(dx * dx + dy * dy);

printf("Distance AB = %.2f\n", d);
return 0;
}

```

Le résultat de `sqrt` est bien récupéré dans `d`; l'argument `dx * dx + dy * dy` est une somme de carrés, donc toujours positive ou nulle : la condition d'application de `sqrt` ($x \geq 0$) est automatiquement respectée.

(2) Exécution pas à pas pour $A(1,2)$ et $B(4,6)$.

Instruction	Calcul	Variables après exécution
saisies	—	$x_A=1, y_A=2, x_B=4, y_B=6$
<code>dx = xB - xA;</code>	$4 - 1$	<code>dx = 3</code>
<code>dy = yB - yA;</code>	$6 - 2$	<code>dy = 4</code>
<code>d = sqrt(...);</code>	$\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25}$	<code>d = 5</code>
<code>printf(...);</code>	—	affiche Distance AB = 5.00

Vérification géométrique : $d = \sqrt{(4-1)^2 + (6-2)^2} = \sqrt{25} = 5$. Le résultat est exact (c'est le célèbre triangle 3-4-5).

(3) Oubli de `#include <math.h>`. Le compilateur ne connaît pas le prototype de `sqrt` : il émet un avertissement (implicit declaration of function 'sqrt') et suppose un retour `int`, ce qui corrompt le résultat (troncature). De plus, à l'édition de liens sous Linux, il faut ajouter l'option `-lm`. Réflexe : toute fonction utilisée exige l'inclusion de sa bibliothèque.

Conclusion. `math.h` fournit les outils de calcul scientifique ; l'exécution pas à pas, au débogueur ou sur papier, est la méthode reine pour valider chaque instruction.

11.6 Mini-projet d'intégration : Calculatrice de notes de classe

EXERCICE RÉSOLU 16

Mini-projet : Gestion des notes d'une classe (intégration)

Énoncé. On veut un programme qui : (1) Demande le nombre d'élèves N (entre 1 et 50, saisie contrôlée). (2) Pour chaque élève, saisit sa note (réel entre 0 et 20, saisie contrôlée). (3) Calcule et affiche : la moyenne de la classe, la note maximale, la note minimale, et le nombre d'élèves ayant une note ≥ 10 (admis).

SOLUTION

Solution détaillée.

Analyse. Besoin d'une boucle pour les N élèves (`for`), d'accumulateurs (somme, max, min, compteur admis), de saisies contrôlées (`do...while`) pour N et chaque note.

Programme complet.

```
#include <stdio.h>

int main(void)
{
    int N, i, admis = 0;
    float note, somme = 0.0, max, min;

    /* 1. Saisie contrôlée de N */
    do {
        printf("Nombre d'eleves (1 a 50) : ");
        scanf("%d", &N);
    } while (N < 1 || N > 50);

    /* 2. Saisie de la première note pour initialiser max et min */
    do {
        printf("Note de l'eleve 1 (0 a 20) : ");
        scanf("%f", &note);
    } while (note < 0 || note > 20);
    somme = note;
    max = note;
    min = note;
    if (note >= 10) admis = 1;

    /* 3. Boucle pour les N-1 élèves restants */
    for (i = 2; i <= N; i++) {
        do {
            printf("Note de l'eleve %d (0 a 20) : ", i);
            scanf("%f", &note);
        } while (note < 0 || note > 20);

        somme = somme + note;
        if (note > max) max = note;
        if (note < min) min = note;
        if (note >= 10) admis++;
    }

    /* 4. Affichage des résultats */
    printf("\n--- BULLETIN DE CLASSE ---\n");
    printf("Moyenne : %.2f\n", somme / N);
    printf("Maximum : %.2f\n", max);
    printf("Minimum : %.2f\n", min);
    printf("Admis : %d sur %d\n", admis, N);

    return 0;
}
```

Points clés vérifiés.

- Initialisation de `max` et `min` avec la première note (pas 0 ou 20 arbitraires).
- Saisie contrôlée `do...while` pour N et chaque note.

- Accumulateur `somme` initialisé à 0, puis première note ajoutée.
- Compteur `admis` incrémenté correctement.
- Division finale `somme / N` : `somme` est `float`, division réelle garantie.

Test rapide (trace mentale). $N = 3$, notes : 12, 8, 15.

- Init : `somme=12`, `max=12`, `min=12`, `admis=1`.
- $i=2$ (note 8) : `somme=20`, `max=12`, `min=8`, `admis=1`.
- $i=3$ (note 15) : `somme=35`, `max=15`, `min=8`, `admis=2`.
- Moyenne = $35/3 \approx 11.67$. `Max=15`, `Min=8`, `Admis=2`. Correct.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Confondre `=` et `==`. Écrire `if (moyenne = 10)` n'est pas un test : c'est une affectation qui met 10 dans `moyenne` et dont le résultat est toujours vrai (non nul). Le test d'égalité exige `==`.
2. Oublier le `&` dans `scanf`. `scanf("%d", n);` au lieu de `scanf("%d", &n);` : le programme compile parfois mais plante à l'exécution (erreur de segmentation / Segmentation fault). L'adresse de la variable est obligatoire pour que `scanf` puisse la modifier.
3. La division entière inattendue. Avec des `int`, `(a + b) / 2` tronque le résultat : `(12+15)/2` donne 13 et non 13,5. Utilise des variables `float` ou écris `/ 2.0` (ou `/ 2.0f`).
4. Oublier `break;` dans un `switch`. Sans `break`, l'exécution « tombe » dans les `case` suivants (fall-through) et affiche plusieurs messages au lieu d'un seul. Seul le `default` final peut s'en passer.
5. Point-virgule parasite après `if` ou `for`. `if (note >= 10);` suivi d'un bloc : le point-virgule crée une instruction vide, le bloc s'exécute toujours, et la condition ne sert plus à rien. De même `for (...);` vide le corps de la boucle. On ne met jamais de `;` entre la condition et l'accolade ouvrante.

Réflexe final : relis ton programme en vérifiant dans l'ordre : `#include` présents, accolades appariées, point-virgule après chaque instruction (mais pas après `if/for/while`), `&` dans les `scanf`, types cohérents avec les formats `%d/%f/%lf`.

EXERCICE 10 Entraînement 1 : Conversion de température

Écrire un programme C qui demande une température en degrés Celsius, la convertit en Fahrenheit ($F = C \times 1,8 + 32$) et en Kelvin ($K = C + 273,15$), puis affiche les trois valeurs avec deux décimales. Prévoir une saisie contrôlée pour refuser les températures inférieures au

zéro absolu ($-273,15^{\circ}\text{C}$).

EXERCICE 11 Entraînement 2 : Table de multiplication

Écrire un programme qui demande un entier n (entre 1 et 20, saisie contrôlée) et affiche sa table de multiplication de $1 \times n$ à $10 \times n$ sous la forme : $1 \times 7 = 7$

$2 \times 7 = 14$

... Utiliser une boucle `for`.

EXERCICE 12 Entraînement 3 : Pair ou Impair & Multiple

Écrire un programme qui saisit un entier x et affiche :

- « Pair » ou « Impair »,
- « Multiple de 3 » ou « Non multiple de 3 »,
- « Multiple de 5 » ou « Non multiple de 5 ».

Utiliser l'opérateur modulo `%` et des structures `if...else` imbriquées ou successives.

CORRIGÉ Exercice 1 : Conversion de température

```
#include <stdio.h>

int main(void)
{
    float celsius, fahrenheit, kelvin;

    do {
        printf("Temperature en Celsius (>= -273.15) : ");
        scanf("%f", &celsius);
    } while (celsius < -273.15);

    fahrenheit = celsius * 1.8 + 32;
    kelvin = celsius + 273.15;

    printf("Celsius      : %.2f\n", celsius);
    printf("Fahrenheit : %.2f\n", fahrenheit);
    printf("Kelvin       : %.2f\n", kelvin);
    return 0;
}
```

CORRIGÉ Exercice 2 : Table de multiplication

```
#include <stdio.h>

int main(void)
{
    int n, i;

    do{
        printf("Entier n (1 a 20) : ");
        scanf("%d", &n);
    } while (n < 1 || n > 20);

    printf("Table de multiplication de %d :\n", n);
    for (i = 1; i <= 10; i++){
        printf("%d x %d = %d\n", i, n, i * n);
    }
    return 0;
}
```

CORRIGÉ Exercice 3 : Pair/Impair & Multiples

```
#include <stdio.h>

int main(void)
{
    int x;

    printf("Entier x : ");
    scanf("%d", &x);

    if (x % 2 == 0)
        printf("Pair\n");
    else
        printf("Impair\n");

    if (x % 3 == 0)
        printf("Multiple de 3\n");
    else
        printf("Non multiple de 3\n");

    if (x % 5 == 0)
        printf("Multiple de 5\n");
    else
        printf("Non multiple de 5\n");

    return 0;
}
```

12 EXERCICES

12.1 Je vérifie mes connaissances

EXERCICE 13 QCM : Compilation et structure de base

Parmi les affirmations suivantes, une seule est exacte. Laquelle ?

1. Un programme C est exécuté directement par le processeur sans étape intermédiaire.
2. Le compilateur traduit le code source en langage machine (fichier objet) ; l'éditeur de liens (linker) assemble ensuite les fichiers objets et les bibliothèques pour produire l'exécutable.
3. La fonction `main()` est optionnelle dans un programme C standard.
4. L'instruction `#include <stdio.h>` définit le contenu de la bibliothèque standard d'entrée/sortie.

EXERCICE 14 Vrai ou Faux justifié : Types et variables

Déterminez si chaque affirmation est vraie (V) ou fausse (F). Justifiez brièvement chaque réponse.

1. Le type `float` occupe 4 octets en mémoire et permet de stocker des entiers très grands sans perte de précision.
2. Une variable déclarée `const double PI = 3.14159;` peut voir sa valeur modifiée par une instruction d'affectation ultérieure dans le même bloc.
3. L'opérateur `%` (modulo) ne s'applique qu'à des opérandes de type entier (`int`, `short`, `long`).
4. En C, l'instruction `a = b = c = 5;` est syntaxiquement incorrecte car l'affectation n'est pas associative.

EXERCICE 15 Définitions et distinctions

Donnez une définition précise et concise pour chacun des termes suivants :

1. Langage compilé vs langage interprété.
2. Variable locale vs variable globale (portée et durée de vie).
3. Instruction de préprocesseur (exemple : `#define`, `#include`).

4. Débogage pas à pas (step over, step into, point d'arrêt).

EXERCICE 16 Calculs directs et priorités opératoires

Évaluez les expressions C suivantes en respectant les règles de priorité et d'associativité. Indiquez le type et la valeur du résultat. On suppose `int a = 7, b = 3; double x = 2.5;`.

1. `a / b * b + a % b`
2. `++a * b--` (valeur de l'expression et valeurs finales de `a` et `b`)
3. `(int)(x * 2) + 10 / 3`
4. `a > b && x < 5.0 || a == 10`

12.2 Je m'entraîne

EXERCICE 17 Table de multiplication avec boucle `for`

Écrivez un programme C complet qui :

1. Demande à l'utilisateur de saisir un entier `n` compris entre 1 et 20 (boucler tant que la saisie est invalide).
2. Affiche la table de multiplication de `n` de `1 × n` à `10 × n` en utilisant une boucle `for`, avec le format : `1 x 7 = 7`.

EXERCICE 18 Résolution d'équation du second degré — traduction algorithme vers C

Traduisez en langage C l'algorithme suivant :

```
Algorithme RacinesSecondDegre
Variables a, b, c, delta, x1, x2 : Réel
Début
    Afficher "Entrez a, b, c : "
    Lire a, b, c
    Si a <> 0 Alors
        delta <- b*b - 4*a*c
        Si delta > 0 Alors
            x1 <- (-b - Racine(delta)) / (2*a)
            x2 <- (-b + Racine(delta)) / (2*a)
            Afficher "Deux racines : ", x1, " et ", x2
        Sinon Si delta = 0 Alors
            x1 <- -b / (2*a)
            Afficher "Racine double : ", x1
        Sinon
```

```

        Afficher "Pas de racine réelle"
    Fin Si
Sinon
    Afficher "Ce n'est pas une équation du second degré (a = 0)"
Fin Si
Fin

```

Utilisez les fonctions de `math.h` (`sqrt`). Gérez l'affichage avec 3 décimales.

EXERCICE 19 Correction d'erreur logique : Somme des carrés avec `while`

Le programme ci-dessous est censé calculer la somme $S = \sum_{k=1}^N k^2$ pour un entier N saisi par l'utilisateur. Il contient une erreur logique. Identifiez l'erreur, expliquez-la et proposez le code corrigé.

```

#include <stdio.h>

int main() {
    int N, k = 1;
    long somme = 0; // long pour éviter débordement rapide

    printf("Entrez N (entier positif) : ");
    scanf("%d", &N);

    while (k <= N) {
        somme = somme + k * k;
        // ERREUR ICI : l'incrément de k est mal placée ou absente ?
    }

    printf("Somme des carrés de 1 à %d = %ld\n", N, somme);
    return 0;
}

```

EXERCICE 20 Menu restaurant avec `switch` et réduction

Concevez un programme C qui simule une commande au restaurant « Chez Maman » à Yaoundé.

1. Affichez le menu suivant :

Code	Plat	Prix (FCFA)
1	Ndolé + Bobolo	2500
2	Poisson braisé + Plantain	3000
3	Poulet DG	3500
4	Eru + Water Fufu	2800
0	Terminer la commande	—

2. Permettez au client de saisir plusieurs codes (un par un) jusqu'à ce qu'il entre 0.
3. Utilisez une instruction `switch` pour ajouter le prix du plat au total.
4. Si le total final dépasse 5000 FCFA, appliquez une réduction de 10%.
5. Affichez le détail (liste des plats choisis, sous-total, réduction, total à payer).

12.3 Je me prépare au Probatoire

EXERCICE 21 Probatoire 2023 — Gestion de notes d'une classe

Un enseignant du Lycée de Biyem-Assi souhaite automatiser le calcul des statistiques de sa classe de 40 élèves maximum pour une matière donnée. Écrire un programme C structuré (utilisation de fonctions) qui permet de :

1. Saisir le nombre d'élèves N ($1 \leq N \leq 40$). Re-saisir si invalide.
2. Saisir les notes de chaque élève (réelles entre 0 et 20). Re-saisir toute note hors intervalle.
3. Stocker les notes dans un tableau `float notes[40]`.
4. Calculer et afficher :
 - La moyenne de la classe.
 - La note maximale et la note minimale.
 - Le nombre d'élèves ayant une note ≥ 10 (admis) et le nombre d'élèves ayant une note < 10 (ajournés).
 - L'écart-type de la série : $\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$.
5. Afficher la liste des notes triées par ordre croissant (utiliser un tri simple : bulle ou sélection).

Contraintes : Décomposer le programme en fonctions : `saisirNotes`, `calculerMoyenne`, `calculerEcartType`, `trierNotes`, `afficherStatistiques`. Le `main` ne fait qu'appeler ces fonctions.

EXERCICE 22 Probatoire 2022 — Simulation de guichet automatique (ATM) CAMTEL

On veut simuler le fonctionnement simplifié d'un distributeur automatique de billets (DAB) pour un client de la CAMTEL. Le client a un solde initial de 150 000 FCFA et un code PIN à 4 chiffres (initialisé à 1234). Le programme doit présenter un menu en boucle jusqu'à ce que l'utilisateur choisisse « Quitter » :

1. Vérifier le solde : Affiche le solde actuel.
2. Retirer de l'argent : Demande le montant (multiple de 5000 FCFA, $\leq$ solde, $\leq$ 100 000 FCFA par transaction). Déduit du solde si valide.
3. Déposer de l'argent : Demande le montant (multiple de 1000 FCFA, > 0). Ajoute au solde.
4. Changer le PIN : Demande l'ancien PIN, puis le nouveau PIN (2 fois pour confirmation). Met à jour si l'ancien est correct et les deux nouveaux correspondent.
5. Quitter : Affiche « Merci d'avoir utilisé les services CAMTEL » et termine.

Gérer les erreurs de saisie (montants invalides, PIN incorrect, choix de menu invalide) avec des messages clairs et redemander la saisie. Utiliser `switch` pour le menu principal. Utiliser des fonctions pour chaque opération.

EXERCICE 23 Probatoire 2024 — Calculatrice scientifique modulaire

Développer un programme C qui implémente une calculatrice scientifique offrant les opérations suivantes via un menu `switch` dans une boucle `do...while`:

1. Addition ($a + b$)
2. Soustraction ($a - b$)
3. Multiplication ($a \times b$)
4. Division (a/b) — gérer la division par zéro.
5. Puissance (a^b) — utiliser `pow` de `math.h`.
6. Racine carrée ($\sqrt{a}$) — utiliser `sqrt`, gérer $a < 0$.
7. Cosinus ($\cos(a)$) — argument en degrés, conversion en radians ($\text{rad} = \text{deg} \times \pi/180$).
8. Quitter.

Exigences techniques :

- Chaque opération est codée dans une fonction séparée (ex : `double addition(double a, double b);` `double racine(double a)` renvoie -1 et affiche une erreur si $a < 0$).
- Le programme redemande les opérandes pour chaque opération (sauf racine et cosinus : un seul opérande).
- L'affichage des résultats se fait avec 4 décimales (`%.4f`).
- Définir une constante PI avec `#define PI 3.14159265359`.
- Le code doit être proprement indenté, commenté et compilable sous Code ::Blocks / Dev-C++ (GCC).

13 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : Compilation et structure de base

Réponse correcte : b.

- a) FAUX. Le processeur n'exécute que du langage machine (suite d'instructions binaires). Un programme C (code source) doit d'abord être **compilé** : le compilateur le traduit en fichier objet, puis l'éditeur de liens produit l'exécutable. Le code source n'est jamais exécuté « directement ».
- b) VRAI. C'est exactement la chaîne de compilation : `fichier.c` → préprocesseur (traitement des directives `#include`, `#define`) → compilateur (fichier objet `.o` ou `.obj`) → éditeur de liens (linker) qui assemble les fichiers objets et les bibliothèques (ex. `stdio`) → exécutable (`.exe` sous Windows).
- c) FAUX. La fonction `main()` est **obligatoire** : c'est le point d'entrée de tout programme C standard, la première fonction appelée au lancement. Sans elle, l'éditeur de liens échoue avec une erreur du type « undefined reference to main ».
- d) FAUX. `#include <stdio.h>` ne définit rien : c'est une directive du préprocesseur qui insère le contenu du fichier d'en-tête `stdio.h` (déclarations de `printf`, `scanf`, etc.) dans le code source avant compilation. La définition des fonctions, elle, se trouve dans la bibliothèque liée par l'éditeur de liens.

CORRIGÉ Exercice 2 — Vrai ou Faux justifié : Types et variables

1. FAUX. `float` occupe bien 4 octets, mais c'est un type flottant (virgule flottante, environ 7 chiffres significatifs). Il ne stocke pas les grands entiers « sans perte » : au-delà de $2^{24} \approx 16,7$ millions, la précision se dégrade. Pour les grands entiers, on utilise `long` ou `long long`.
2. FAUX. Le mot-clé `const` rend la variable **non modifiable** : toute affectation ultérieure (`PI = 3.14;`) provoque une erreur de compilation. C'est d'ailleurs une bonne pratique : le compilateur protège la valeur.
3. VRAI. L'opérateur modulo `%` n'accepte que des opérandes entiers. Écrire `5.5 % 2` est une erreur de compilation. Pour les réels, il faudrait utiliser `fmod()` de la bibliothèque `math.h`.
4. FAUX. L'affectation est **associative à droite** : `a = b = c = 5;` s'évalue comme `a = (b = (c = 5));`. C'est parfaitement valide : les trois variables reçoivent la valeur 5.

CORRIGÉ Exercice 3 — Définitions et distinctions

1. Langage compilé : le code source est traduit une fois pour toutes en langage machine par un compilateur, produisant un exécutable autonome et rapide (ex. C, C++). Langage interprété : le code est traduit et exécuté ligne par ligne à chaque exécution par un interpréteur, sans exécutable autonome (ex. JavaScript dans le navigateur, Python). Conséquence pratique : un programme compilé est en général plus rapide, mais il faut recompiler après chaque modification du source.
2. Variable locale : déclarée à l'intérieur d'une fonction ou d'un bloc ; sa portée est limitée à ce bloc et sa durée de vie s'arrête à la fin du bloc. Variable globale : déclarée en dehors de toute fonction ; visible par toutes les fonctions du fichier et vivant pendant toute l'exécution du programme. On préfère les variables locales, plus sûres et plus lisibles.
3. Instruction de préprocesseur : directive commençant par #, traitée avant la compilation proprement dite. `#include <stdio.h>` insère un fichier d'en-tête ; `#define PI 3.14159` effectue une substitution textuelle : chaque occurrence de `PI` est remplacée par `3.14159` dans le source. Attention : une directive `#define` ne se termine pas par un point-virgule.
4. Débogage pas à pas : exécution contrôlée du programme via le débogueur de l'IDE (Code::Blocks, Dev-C++). Un **point d'arrêt** (breakpoint) suspend l'exécution sur une ligne ; step over (« Next line ») exécute la ligne courante sans entrer dans les fonctions appelées ; step into (« Step into ») entre dans le corps de la fonction appelée. On observe alors l'évolution des variables dans la fenêtre des watches, ce qui permet de localiser précisément l'instruction fautive.

CORRIGÉ Exercice 4 — Calculs directs et priorités opératoires

Données : `int a = 7, b = 3; double x = 2.5;`

1. `a / b * b + a % b` : /, * et % ont la même priorité et s'évaluent de gauche à droite. Division entière $7/3 = 2$, puis $2 * 3 = 6$, puis $7 \% 3 = 1$, enfin $6 + 1 = 7$. Résultat : int, valeur 7.
2. `++a * b--` : pré-incrémentation $\Rightarrow$ a devient 8 avant usage ; post-décrémentation $\Rightarrow$ b vaut 3 au moment du calcul puis devient 2. Expression : $8 * 3 = 24$. Résultat : int, valeur 24 ; valeurs finales : a = 8, b = 2.
3. `(int)(x * 2) + 10 / 3` : `x * 2 = 5.0`, le cast (conversion forcée) donne `(int)5.0 = 5` ; $10 / 3 = 3$ (division entière entre int) ; $5 + 3 = 8$. Résultat : int, valeur 8.
4. `a > b && x < 5.0 || a == 10` : a > b est vrai (1), x < 5.0 est vrai (1), donc $1 \&\& 1 = 1$; a == 10 est faux (0) ; $1 || 0 = 1$. Résultat : int, valeur 1 (vrai). Rappel : && est prioritaire sur ||, et les opérateurs de comparaison sont prioritaires sur les deux.

Point de vigilance : en C, $7 / 3$ donne 2 (division entière entre `int`) et non 2,33 ; c'est l'erreur la plus fréquente au Probatoire. Pour obtenir un quotient réel, il faut qu'au moins un opérande soit flottant : $7.0 / 3$ ou `(double)a / b`.

CORRIGÉ Exercice 5 — Table de multiplication avec boucle `for`

```
#include <stdio.h>

int main() {
    int n, i;

    /* Saisie contrôlée : on boucle tant que n est hors de [1, 20] */
    do {
        printf("Entrez un entier n entre 1 et 20 : ");
        scanf("%d", &n);
        if (n < 1 || n > 20) {
            printf("Erreur : n doit être compris entre 1 et 20.\n");
        }
    } while (n < 1 || n > 20);

    /* Affichage de la table de n */
    printf("\nTable de multiplication de %d : \n", n);
    for (i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", i, n, i * n);
    }

    return 0;
}
```

Explications : la boucle `do...while` garantit au moins une saisie et recommence tant que `n` est invalide (le test est fait après le corps, ce qui convient parfaitement à une saisie contrôlée). La boucle `for` est adaptée car le nombre d'itérations (10) est connu à l'avance : initialisation `i = 1`, condition `i <= 10`, incrémentation `i++` sont regroupées sur une seule ligne. Le format `%d x %d = %d` produit exactement l'affichage demandé (ex. `1 x 7 = 7`).

CORRIGÉ Exercice 6 — Résolution d'équation du second degré

```
#include <stdio.h>
#include <math.h> /* pour sqrt() */

int main() {
    double a, b, c, delta, x1, x2;

    printf("Entrez a, b, c : ");
    scanf("%lf %lf %lf", &a, &b, &c);
```

```

if (a != 0) {
    delta = b * b - 4 * a * c;
    if (delta > 0) {
        x1 = (-b - sqrt(delta)) / (2 * a);
        x2 = (-b + sqrt(delta)) / (2 * a);
        printf("Deux racines : %.3f et %.3f\n", x1, x2);
    } else if (delta == 0) {
        x1 = -b / (2 * a);
        printf("Racine double : %.3f\n", x1);
    } else {
        printf("Pas de racine réelle\n");
    }
} else {
    printf("Ce n'est pas une equation du second degre (a = 0)\n");
}

return 0;
}

```

Vérification mentale : pour $a = 1$, $b = -3$, $c = 2$, on a $\Delta = 9 - 8 = 1 > 0$, $x_1 = \frac{3-1}{2} = 1$ et $x_2 = \frac{3+1}{2} = 2$: le programme affiche Deux racines : 1.000 et 2.000.

Points de vigilance :

- Pour un double, scanf exige %lf (et non %f, réservé à float); en revanche printf accepte %f pour un double.
- sqrt() prend et renvoie un double; ne jamais l'appeler sur un delta négatif (d'où le test préalable), sinon le résultat est indéfini.
- Les parenthèses de $(2 * a)$ sont indispensables : $-b / 2 * a$ calculerait $(-b/2) \times a$, ce qui est faux.
- %.3f impose l'affichage à 3 décimales.
- Sous GCC en ligne de commande, penser à lier la bibliothèque mathématique avec l'option -lm (automatique sous Code ::Blocks).

CORRIGÉ Exercice 7 — Correction d'erreur logique : Somme des carrés

Erreur identifiée : la variable de boucle `k` n'est jamais incrémentée dans le corps du `while`. Comme `k` reste égal à 1, la condition `k <= N` demeure vraie indéfiniment : le programme boucle à l'infini (et `somme` augmente sans cesse jusqu'au débordement de capacité). L'instruction `k++`; manque à la fin du corps de la boucle. Il s'agit d'une erreur logique : le programme compile sans erreur, mais ne produit pas le résultat attendu.

Code corrigé :

```

#include <stdio.h>

int main() {
    int N, k = 1;

```

```

long somme = 0; /* long pour eviter un debordement rapide */

printf("Entrez N(entier positif) : ");
scanf("%d", &N);

while (k <= N) {
    somme = somme + k * k;
    k++; /* INCREMENTATION : indispensable pour progresser vers N */
}

printf("Somme des carres de 1 a %d = %ld\n", N, somme);
return 0;
}

```

Vérification : pour $N = 3$, $S = 1 + 4 + 9 = 14$; le programme affiche bien 14. On retrouve la formule $S = \frac{N(N+1)(2N+1)}{6}$: pour $N = 3$, $\frac{3 \times 4 \times 7}{6} = 14$. Noter le format `%ld` pour afficher un long. Point de vigilance : toute boucle `while` doit contenir une instruction qui fait évoluer la condition vers la sortie, sous peine de boucle infinie. C'est précisément le type d'erreur que l'exécution pas à pas (débugueur) permet de repérer : on verrait `k` rester bloqué à 1.

CORRIGÉ Exercice 8 — Menu restaurant « Chez Maman »

```

#include <stdio.h>

int main() {
    int code;
    int total = 0;
    double reduction = 0.0, aPayer;

    printf("=== CHEZ MAMAN - YAOUNDE ===\n");
    printf("1. Ndole + Bobolo ..... 2500 FCFA\n");
    printf("2. Poisson braise + Plantain 3000 FCFA\n");
    printf("3. Poulet DG ..... 3500 FCFA\n");
    printf("4. Eru + Water Fufu ..... 2800 FCFA\n");
    printf("0. Terminer la commande\n");

    do {
        printf("\nVotre choix (0 pour terminer) : ");
        scanf("%d", &code);
        switch (code) {
            case 1:
                printf("-> Ndole + Bobolo ajoute.\n");
                total += 2500;
                break;
            case 2:
                printf("-> Poisson braise + Plantain ajoute.\n");
                total += 3000;
                break;
            case 3:
                printf("-> Poulet DG ajoute.\n");
                total += 3500;

```

```

        break;
    case 4:
        printf("-> Eru + Water Fufu ajoute.\n");
        total += 2800;
        break;
    case 0:
        break; /* fin de commande */
    default:
        printf("Code invalide, recommencez.\n");
    }
} while (code != 0);

if (total > 5000) {
    reduction = total * 0.10; /* reduction de 10 pour cent */
}
aPayer = total - reduction;

printf("\n===== FACTURE =====\n");
printf("Sous-total   : %d FCFA\n", total);
printf("Reduction    : %.0f FCFA\n", reduction);
printf("Total a payer : %.0f FCFA\n", aPayer);
printf("Merci et a bientot chez Maman !\n");

return 0;
}

```

Explications : la boucle `do...while` permet plusieurs commandes jusqu'à la saisie de 0. Chaque case du `switch` ajoute le prix au `total` grâce à l'affectation composée `+=`; le `break` est indispensable pour éviter l'exécution en cascade des cas suivants (piège classique du `switch`). Le `default` intercepte les codes hors menu. La réduction ne s'applique que si `total > 5000`. Exemple de trace : Ndolé (2500) + Poulet DG (3500) = 6000 > 5000, réduction de 600, à payer 5400 FCFA.

CORRIGÉ Exercice 9 — Probatoire 2023 : Gestion de notes d'une classe

Programme structuré en fonctions :

```

#include <stdio.h>
#include <math.h>

#define MAX 40

/* Saisie contrôlée des N notes dans le tableau */
void saisirNotes(float notes[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        do {
            printf("Note de l'eleve %d (0 a 20) : ", i + 1);
            scanf("%f", &notes[i]);
            if (notes[i] < 0 || notes[i] > 20) {
                printf("Erreur : note hors intervalle.\n");
            }
        }
    }
}

```

```
    } while (notes[i] < 0 || notes[i] > 20);
}
}

double calculerMoyenne(float notes[], int n){
    double somme = 0;
    int i;
    for (i = 0; i < n; i++){
        somme += notes[i];
    }
    return somme / n;
}

double calculerEcartType(float notes[], int n, double moyenne){
    double sommeCarres = 0;
    int i;
    for (i = 0; i < n; i++){
        sommeCarres += (notes[i] - moyenne) * (notes[i] - moyenne);
    }
    return sqrt(sommeCarres / n);
}

/* Tri à bulles croissant */
void trierNotes(float notes[], int n) {
    int i, j;
    float tmp;
    for (i = 0; i < n - 1; i++){
        for (j = 0; j < n - 1 - i; j++){
            if (notes[j] > notes[j + 1]) {
                tmp = notes[j];
                notes[j] = notes[j + 1];
                notes[j + 1] = tmp;
            }
        }
    }
}

void afficherStatistiques(float notes[], int n) {
    double moyenne = calculerMoyenne(notes, n);
    double ecartType = calculerEcartType(notes, n, moyenne);
    float max = notes[0], min = notes[0];
    int admis = 0, i;

    for (i = 0; i < n; i++){
        if (notes[i] > max) max = notes[i];
        if (notes[i] < min) min = notes[i];
        if (notes[i] >= 10) admis++;
    }

    printf("\n===== STATISTIQUES =====\n");
    printf("Moyenne   : %.2f / 20\n", moyenne);
    printf("Note max   : %.2f\n", max);
    printf("Note min   : %.2f\n", min);
    printf("Admis (>=10) : %d\n", admis);
    printf("Ajournes   : %d\n", n - admis);
    printf("Ecart-type  : %.2f\n", ecartType);
}
```

```

int main() {
    float notes[MAX];
    int N, i;

    do {
        printf("Nombre d'eleves (1 a 40) : ");
        scanf("%d", &N);
    } while (N < 1 || N > MAX);

    saisirNotes(notes, N);
    afficherStatistiques(notes, N);

    trierNotes(notes, N);
    printf("\nNotes triees (croissant) :\n");
    for (i = 0; i < N; i++) {
        printf("%.2f ", notes[i]);
    }
    printf("\n");

    return 0;
}

```

Barème indicatif (sur 10) : saisie contrôlée de N et des notes : 2 pts ; déclaration et usage corrects du tableau : 1 pt ; fonctions moyenne et écart-type (formule exacte, `sqrt`) : 2,5 pts ; max/min/admis/ajournés : 1,5 pts ; tri correct : 2 pts ; structure en fonctions et clarté du `main` : 1 pt.

Points de vigilance : passer la moyenne en paramètre à `calculerEcartType` (ou la recalculer à l'intérieur) ; ne jamais dépasser l'indice $N-1$ (les indices vont de 0 à $N-1$) ; le tri s'effectue après les statistiques pour ne pas fausser la lecture des données saisies ; `%f` pour lire un `float` avec `scanf` ; un tableau passé en paramètre est modifiable dans la fonction (passage par adresse implicite), c'est pourquoi `trierNotes` modifie réellement le tableau du `main`.

CORRIGÉ

Exercice 10 — Probatoire 2022 : Simulation de guichet automatique CAMTEL

```

#include <stdio.h>

/* Variables globales simples (solde et PIN partagés) */
double solde = 150000.0;
int pin = 1234;

void verifierSolde(void) {
    printf("Solde actuel : %.0f FCFA\n", solde);
}

void retirer(void) {
    double montant;
    printf("Montant a retirer : ");
    scanf("%lf", &montant);
}

```

```
    if (montant <= 0 || (int)montant % 5000 != 0){
        printf("Erreur : le montant doit etre un multiple de 5000.\n");
    } else if (montant > 100000){
        printf("Erreur : plafond de 100000 FCFA par transaction.\n");
    } else if (montant > solde){
        printf("Erreur : solde insuffisant.\n");
    } else {
        solde -= montant;
        printf("Retrait effectue. Nouveau solde : %.0f FCFA\n", solde);
    }
}

void deposer(void){
    double montant;
    printf("Montant a deposer : ");
    scanf("%lf", &montant);
    if (montant <= 0 || (int)montant % 1000 != 0){
        printf("Erreur : le montant doit etre un multiple de 1000.\n");
    } else {
        solde += montant;
        printf("Depot effectue. Nouveau solde : %.0f FCFA\n", solde);
    }
}

void changerPin(void){
    int ancien, nouveau1, nouveau2;
    printf("Ancien PIN : ");
    scanf("%d", &ancien);
    if (ancien != pin) {
        printf("Erreur : ancien PIN incorrect.\n");
        return;
    }
    printf("Nouveau PIN : ");
    scanf("%d", &nouveau1);
    printf("Confirmez le nouveau PIN : ");
    scanf("%d", &nouveau2);
    if (nouveau1 == nouveau2 && nouveau1 >= 1000 && nouveau1 <= 9999){
        pin = nouveau1;
        printf("PIN modifie avec succes.\n");
    } else {
        printf("Erreur : les deux PIN ne correspondent pas ou sont invalides.\n");
    }
}

int main(){
    int choix;
    do{
        printf("\n===== DAB CAMTEL =====\n");
        printf("1. Verifier le solde\n");
        printf("2. Retirer de l'argent\n");
        printf("3. Deposer de l'argent\n");
        printf("4. Changer le PIN\n");
        printf("5. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);

        switch (choix){
```

```

        case 1: verifierSolde(); break;
        case 2: retirer(); break;
        case 3: deposer(); break;
        case 4: changerPin(); break;
        case 5:
            printf("Merci d'avoir utilise les services CAMTEL.\n");
            break;
        default:
            printf("Choix invalide, recommencez.\n");
    }
} while (choix != 5);

return 0;
}

```

Barème indicatif (sur 10) : menu en boucle avec `switch` et `default` : 2 pts; retrait avec les trois contrôles (multiple de 5000, plafond 100 000, solde suffisant) : 2,5 pts; dépôt contrôlé : 1,5 pts; changement de PIN (ancien correct + double confirmation) : 2 pts; fonctions par opération et messages clairs : 2 pts.

Points de vigilance : tester le multiple avec `% ((int)montant % 5000 != 0)` — le modulo exige des entiers, d'où le cast; vérifier le plafond avant de débiter le solde; le `default` du `switch` gère les choix hors menu; chaque `case` se termine par `break`; le `return`; dans `changerPin` permet de quitter immédiatement la fonction si l'ancien PIN est faux.

CORRIGÉ

Exercice 11 — Probatoire 2024 : Calculatrice scientifique modulaire

```

#include <stdio.h>
#include <math.h>

#define PI 3.14159265359

double addition(double a, double b) { return a + b; }
double soustraction(double a, double b) { return a - b; }
double multiplication(double a, double b) { return a * b; }

double division(double a, double b) {
    if (b == 0) {
        printf("Erreur : division par zero impossible.\n");
        return 0;
    }
    return a / b;
}

double puissance(double a, double b) { return pow(a, b); }

double racine(double a) {
    if (a < 0) {
        printf("Erreur : racine d'un nombre negatif.\n");
        return -1;
    }
    return sqrt(a);
}

```

```
}

double cosinusDegre(double a){
    double rad = a * PI / 180.0; /* conversion degres -> radians */
    return cos(rad);
}

int main(){
    int choix;
    double a, b;

    do{
        printf("\n=== CALCULATRICE SCIENTIFIQUE ===\n");
        printf("1. Addition      2. Soustraction\n");
        printf("3. Multiplication  4. Division\n");
        printf("5. Puissance      6. Racine carree\n");
        printf("7. Cosinus (degres) 8. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);

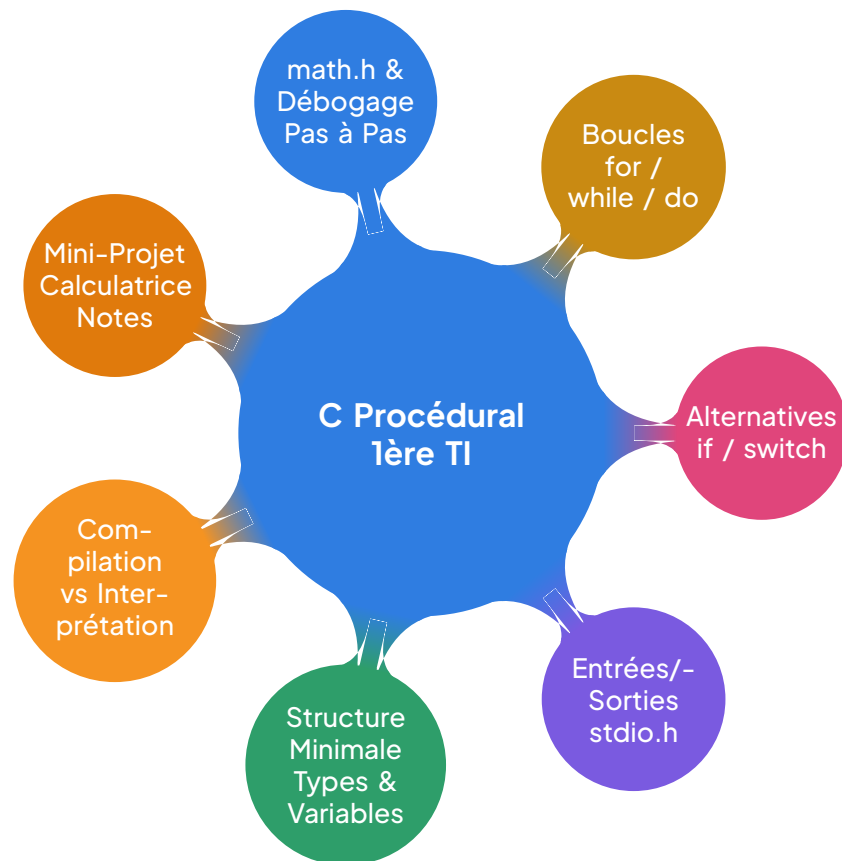
        switch(choix){
            case 1:
                printf("a et b : "); scanf("%lf %lf", &a, &b);
                printf("Resultat : %.4f\n", addition(a, b));
                break;
            case 2:
                printf("a et b : "); scanf("%lf %lf", &a, &b);
                printf("Resultat : %.4f\n", soustraction(a, b));
                break;
            case 3:
                printf("a et b : "); scanf("%lf %lf", &a, &b);
                printf("Resultat : %.4f\n", multiplication(a, b));
                break;
            case 4:
                printf("a et b : "); scanf("%lf %lf", &a, &b);
                if (b != 0) {
                    printf("Resultat : %.4f\n", division(a, b));
                } else {
                    division(a, b); /* affiche le message d'erreur */
                }
                break;
            case 5:
                printf("a et b : "); scanf("%lf %lf", &a, &b);
                printf("Resultat : %.4f\n", puissance(a, b));
                break;
            case 6:
                printf("a : "); scanf("%lf", &a);
                if (a >= 0){
                    printf("Resultat : %.4f\n", racine(a));
                } else {
                    racine(a); /* affiche le message d'erreur */
                }
                break;
            case 7:
                printf("Angle en degres : "); scanf("%lf", &a);
                printf("Resultat : %.4f\n", cosinusDegre(a));
                break;
        }
    } while(choix != 8);
}
```

```
        case 8 :  
            printf("Au revoir !\n");  
            break;  
        default :  
            printf("Choix invalide.\n");  
    }  
} while (choix != 8);  
  
return 0;  
}
```

Vérifications : $\cos(60^\circ) = \cos(\pi/3) = 0,5$: le programme affiche 0.5000. $\sqrt{16} = 4$: affichage 4.0000. $2^{10} = 1024$: affichage 1024.0000.

Barème indicatif (sur 10) : menu `switch` dans `do...while` : 2 pts ; fonctions séparées pour chaque opération : 2 pts ; gestion division par zéro et racine négative : 2 pts ; conversion degrés/radians avec `#define PI` : 1,5 pts ; format `%.4f` et saisies `%lf` : 1,5 pts ; compilation propre, indentation, commentaires : 1 pt.

Points de vigilance : `cos()` attend des radians, d'où la conversion obligatoire $\text{rad} = \text{deg} \times \pi/180$; ne pas oublier `#include <math.h>` ni l'option de liaison `-lm` sous GCC en ligne de commande (automatique sous Code ::Blocks) ; `%lf` dans `scanf` pour les `double` ; la double vérification (dans `main` et dans la fonction) est une bonne pratique de programmation défensive : la fonction reste sûre même si elle est appelée ailleurs sans contrôle préalable.

14 FICHE BILAN

Carte mentale : Les 7 piliers de la programmation C en Première TI

FICHE BILAN L'essentiel à connaître pour l'examen**14.1 Définitions Claires****DÉFINITION** Langage Compilé vs Interprété

Un langage compilé (C, C++) est traduit une seule fois en code machine (exécutable .exe) par un compilateur (GCC, Clang) avant l'exécution. Un langage interprété (Python, JS, PHP) est lu et exécuté ligne par ligne par un interpréteur à chaque lancement. Le C offre rapidité d'exécution et bas niveau ; l'interprété offre portabilité immédiate et développement rapide.

DÉFINITION Structure Minimale d'un Programme C

Tout programme C valide possède une fonction `main` (point d'entrée) et inclut les en-têtes nécessaires (`#include`). Syntaxe canonique :

```
#include <stdio.h> // Entrées/Sorties standard
#include <stdlib.h> // Utilitaires généraux (EXIT_SUCCESS)

int main(void) {
    // Corps du programme : déclarations, instructions
    printf("Bonjour lere TI !\n");
    return EXIT_SUCCESS; // ou return 0;
}
```

DÉFINITION Variable et Constante

Une variable est une zone mémoire nommée, de type fixe, dont la valeur peut changer. Une constante (`const` ou `#define`) a une valeur figée à la compilation.

14.2 Types de Base, Opérateurs & Syntaxe Essentielle (À Connaître Par Cœur)

Type / Catégorie	Syntaxe / Rôle	Format printf/scanf
Entier signé	<code>int a;</code> (généralement 4 octets)	<code>%d</code> ou <code>%i</code>
Entier court	<code>short b;</code> (2 octets)	<code>%hd</code>
Entier long	<code>long c;</code> (4 ou 8 octets)	<code>%ld</code>
Réel simple	<code>float x;</code> (4 octets, ≈ 7 ch. sign.)	<code>%f</code> (sortie), <code>%f</code> (entrée)
Réel double	<code>double y;</code> (8 octets, ≈ 15 ch. sign.)	<code>%lf</code> (entrée), <code>%f</code> ou <code>%lf</code> (sortie)
Caractère	<code>char c;</code> (1 octet, code ASCII)	<code>%c</code>
Chaîne de caractères	<code>char s[50];</code> (tableau terminé par <code>\0</code>)	<code>%s</code>
Constante symbolique	<code>#define PI 3.14159</code> (préprocesseur)	Pas de format (remplacement textuel)
Constante typée	<code>const float TAUX = 0.15f;</code>	Selon le type sous-jacent
Opérateurs Arith.	<code>+ - * / %</code> (modulo entier seulement)	Priorités classiques : <code>*</code> / <code>%</code> avant <code>+</code> -
Opérateurs Comp.	<code>== != < ></code> <code><= >=</code> → renvoient <code>int</code> (0 ou 1)	Utilisés dans conditions <code>if/while</code>
Opérateurs Logiques	<code>&&</code> (ET), <code> </code> (OU), <code>!</code> (NON)	Court-circuit : <code>&&</code> évalue 2ème op. si 1er vrai
Affectation Composée	<code>+= -= *= /=</code> <code>%=</code>	<code>a += 3</code> $\equiv$ <code>a = a + 3</code>
Incrémentation	<code>++a</code> (pré), <code>a++</code> (post)	Différence cruciale dans expressions!

14.3 Méthodes Express

MÉTHODE Traduire un Algorithme vers C (Checklist)

1. Includes : `stdio.h` (toujours), `math.h` (si $\sqrt{\quad}$, `cos`, `pow`), `stdlib.h` (si `EXIT_SUCCESS`, `abs`).
2. Variables : Déclarer au début du bloc (C89/C90 strict) ou près de l'usage (C99+). Initialiser systématiquement (`int n = 0;`).
3. Si / Sinon : `if (cond) {...} else {...}`. Accolades obligatoires si >1 instruction. `else if` pour chaîner.
4. Switch : Uniquement sur entier (`int`, `char`). `case val: ... break; default: ...`. Oublier `break` → fall-through (piège!).
5. Boucle `for` : Compteur connu. `for (init; cond; pas) {...}`.
6. Boucle `while` : Condition testée avant (0 à n fois). Idéal « TantQue » algo.
7. Boucle `do...while` : Corps exécuté au moins une fois. Condition testée après. Idéal « Répéter...Jusqu'à » algo (inverser la condition!).
8. `scanf` : Toujours passer l'adresse `&var` (sauf tableau/chaîne `nomTableau` qui est déjà adresse). Vérifier le retour (`== 1`) pour robustesse.
9. `math.h` : Linker avec `-lm` (GCC) ou cocher « Link math library » dans `Code::Blocks`. `sqrt`, `pow`, `cos` attendent/retournent `double`.

MÉTHODE Débogage Pas à Pas dans Code ::Blocks / Dev-C++

1. Compiler en mode Debug (gros bouton « Debug » / F8, pas « Run »).
2. Poser un point d'arrêt (breakpoint) : clic dans la marge gauche → point rouge.
3. Lancer le débogueur (F8). L'exécution s'arrête au point d'arrêt.
4. F7 (Step Into) : entre dans la fonction. F6 (Step Over) : exécute la ligne sans entrer. `Ctrl+F7` (Step Out) : sort de la fonction.
5. Surveiller la fenêtre Watches (Espions) : ajouter `variable` pour voir sa valeur évoluer.
6. Corriger, recompiler, relancer.

À RETENIR

Checklist Avant le Bac — 10 Points de Vigilance

- ☐ Je distingue compilation (C → exe) et interprétation (ligne à ligne), et je cite un exemple de chaque.
- ☐ J'écris la structure main correcte (`int main(void){... return 0;}`) et les `#include` nécessaires.
- ☐ Je déclare avant usage : `int n=0; float x=0.0f; char c='A';` (initialisation = 0 bug en moins).
- ☐ Je choisis le bon `%d`, `%f`, `%lf`, `%c`, `%s` dans `printf` ET `scanf` (attention `%lf` pour double en entrée!).
- ☐ Dans `scanf`, je mets l'esperluette `&` devant les variables scalaires (`&n`, `&x`), jamais devant un tableau/chaîne.
- ☐ Je structure mes `if/else` avec des accolades `{}` même pour une ligne (lisibilité, évite erreur ; parasite).
- ☐ Dans `switch`, je mets `break` à chaque `case` (sauf fall-through volontaire commenté) et un `default`.
- ☐ Je choisis la boucle adaptée : `for` (nb connu), `while` (0 ou n fois, test entrée), `do-while` (1 fois minimum, test sortie).
- ☐ Pour `math.h` : j'inclus `<math.h>`, j'utilise `double` (ou `cast`), et je linke la lib `math` (`-lm` ou option IDE).
- ☐ Je sais déboguer pas à pas : point d'arrêt (F5/F9), F7 (pas à pas détaillé), F6 (pas à pas principal), fenêtre Watch.



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+